计算机系统笔记

HNU《计算机系统》（CS:APP 体系）课程学习笔记

作者：long6177 | 撰写时间：2026 春季学期

- csapp相关配置和introduction
  - 0.1 相关配置
  - 0.2 Introduction
  - 0.3 Study Methods
- 第一章 计算机系统漫游
  - 1.1 信息 = 位(bit) + 上下文
  - 1.2 程序的生命周期
  - 1.3 了解编译系统工作机制的益处
  - 1.10 小结
- 第二章 信息的表示与处理
  - 2.1 信息存储
  - 2.2 整数表示
  - 2.3 整数运算
  - 2.4 浮点数表示
  - 2.5 小结
- 第三章 程序的机器级表示
  - 3.1 ATT 汇编语言
  - 3.2 数据格式
  - 3.3 访问信息
  - 3.4 操作数与指示符
  - 3.5 数据传送指令
  - 3.6 算术与逻辑操作
  - 3.7 控制
  - 3.8 call指令
  - 3.9 循环指令和switch语句
  - 3.10 过程
  - 3.11 数组分配与访问
  - 3.12 结构体和联合
  - 3.13 理解黑客攻击和防御
- 第四章 链接器
  - 4.1 C语言的符号变量定义相关
  - 4.2 链接时的强弱符号
  - 4.3 链接器的工作流程
  - 4.4 ELF

    - 4.5 链接第一步 -- 符号解析
    - 4.6 链接第二步 -- 重定位
    - 4.7 动态链接
- 第五章 内存系统
    - 5.1 计算机的存储
    - 5.2 存储器层次结构
- 第六章 虚拟存储器
    - 6.1 虚存抽象
    - 6.2 虚存作为缓存工具
    - 6.3 虚存作为内存管理工具
    - 6.4 虚存作为内存保护工具
    - 6.5 虚拟内存的地址翻译
- 第七章 异常与进程
    - 7.1 异常
    - 7.2 进程
    - 7.3 进程的执行

csapp相关配置和introduction

- 0.1 相关配置
  - 0.1.1 终端工具
  - 0.1.2 64位支持32位程序
  - 0.1.3 64位运行32程序
- 0.2 Introduction
  - 0.2.1 前言
  - 0.2.2 背景知识
  - 0.2.3 阅读此书
  - 0.2.4 32位架构 vs 64位架构
- 0.3 Study Methods
  - 根据C代码解读汇编

0.1 相关配置

环境: VMware17.6.3 + Ubuntu24.04桌面版

GNOME Terminal查看环境

```
long@ubuntu2404:~/Desktop/test$ uname -m # 查看系统架构
x86_64  # i686 代表32位系统

long@ubuntu2404:~/Desktop/test$ lsb_release -a  # Ubuntu版本
No LSB modules are available.
Distributor ID: Ubuntu
Description:    Ubuntu 24.04.4 LTS
Release:    24.04
Codename:   noble

long@ubuntu2404:~/Desktop/test$ dpkg --print-architecture # 查看dpkg当前架构
amd64   # 当前是64位架构

long@ubuntu2404:~/Desktop/test$ dpkg --print-foreign-architectures # 输出空表示当前系统未开启32位支持

long@ubuntu2404:~/Desktop/test$ gcc -v
Using built-in specs.
COLLECT_GCC=gcc
COLLECT_LTO_WRAPPER=/usr/libexec/gcc/x86_64-linux-gnu/13/lto-wrapper
OFFLOAD_TARGET_NAMES=nvptx-none:amdgcn-amdhsa
OFFLOAD_TARGET_DEFAULT=1
Target: x86_64-linux-gnu
```

```
Configured with: ../src/configure -v --with-pkgversion='Ubuntu 13.3.0-6ubuntu2~24.04.1' --with-bugurl=file:///usr/share/doc/gcc-13/README.Bugs --enable-languages=c,ada,c++,go,d,fortran,objc,obj-c++,m2 --prefix=/usr --with-gcc-major-version-only --program-suffix=-13 --program-prefix=x86_64-linux-gnu- --enable-shared --enable-linker-build-id --libexecdir=/usr/libexec --without-included-gettext --enable-threads=posix --libdir=/usr/lib --enable-nls --enable-bootstrap --enable-clocale=gnu --enable-libstdcxx-debug --enable-libstdcxx-time=yes --with-default-libstdcxx-abi=new --enable-libstdcxx-backtrace --enable-gnu-unique-object --disable-vtable-verify --enable-plugin --enable-default-pie --with-system-zlib --enable-libphobos-checking=release --with-target-system-zlib=auto --enable-objc-gc=auto --enable-multiarch --disable-werror --enable-cet --with-arch-32=i686 --with-abi=m64 --with-multilib-list=m32,m64,mx32 --enable-multilib --with-tune=generic --enable-offload-targets=nvptx-none=/build/gcc-13-EldibY/gcc-13-13.3.0/debian/tmp-nvptx/usr,amdgcn-amdhsa=/build/gcc-13-EldibY/gcc-13-13.3.0/debian/tmp-gcn/usr --enable-offload-defaulted --without-cuda-driver --enable-checking=release --build=x86_64-linux-gnu --host=x86_64-linux-gnu --target=x86_64-linux-gnu --with-build-config=bootstrap-lto-lean --enable-link-serialization=2
Thread model: posix
Supported LTO compression algorithms: zlib zstd
gcc version 13.3.0 (Ubuntu 13.3.0-6ubuntu2~24.04.1)

long@ubuntu2404:~/Desktop/test$ gcc -m32 # 查看是否支持32位编译
gcc: fatal error: no input files
compilation terminated.
```

0.1.1 终端工具

gcc--指挥工具链调用

安装: `sudo apt install gcc`

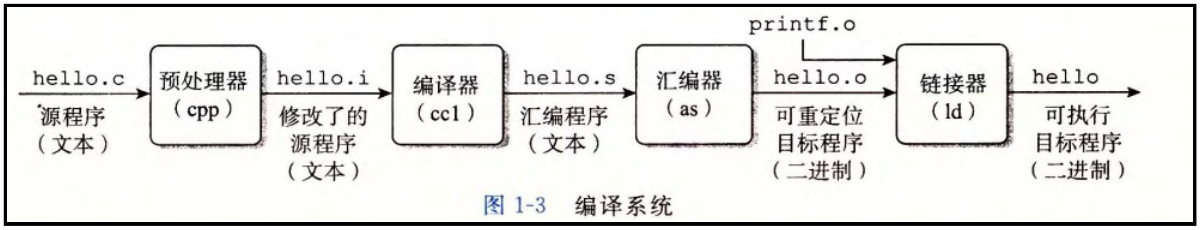


图 1-3　编译系统

GCC是一个驱动程序, 用来控制整个编译流程, 能自动调用as, ld等

```
gcc
 ├── cpp    (预处理器)
 ├── cc1    (C编译器)
 ├── as     (汇编器)
 └── ld     (链接器)
```

执行`gcc hello.c`, 实际发生的是:

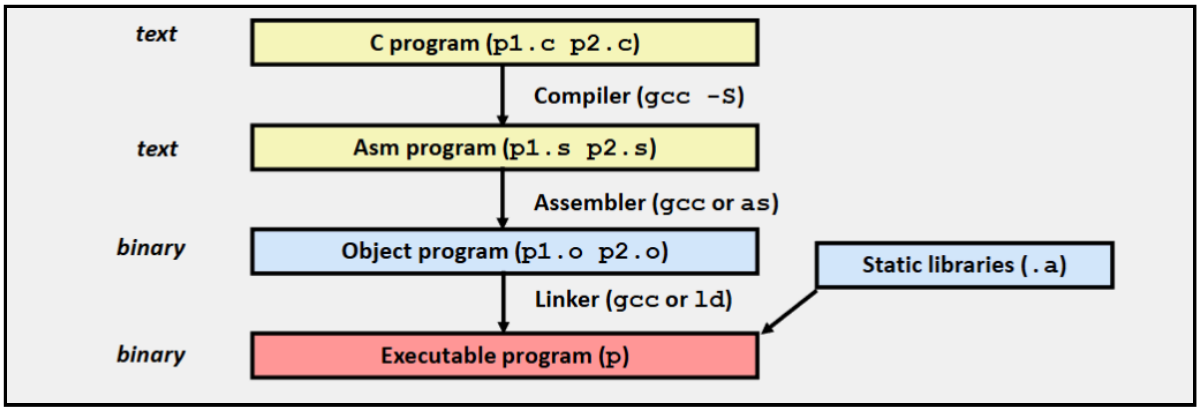
```
cpp hello.c -> hello.i
cc1 hello.i -> hello.s # .s汇编代码文件
as hello.s -> hello.o
ld hello.o -> a.out # .out 可执行程序(ELF)
```

若单独进行编译系统的某一步, 可以:

1. `gcc -E hello.c -o hello.i`: 预处理器(cpp), 将头文件插入到程序文本中去
2. `gcc -S hellp.i -o hello.s`: 编译器(ccl), 将代码翻译成汇编语言
3. `gcc -c hello.s -o hello.o`: 汇编器(as, Assembler), 将汇编语言转换为二进制目标代码
4. `gcc hello.o -o hello.out`: 链接器(ld, Linker), 查找库并打包, 生成*可执行文件(Executable and Linkable Format, ELF)*

`gcc`的`-S`, `-E`和`-c`都可直接操作`hello.c`文件.
并且`-S`和`-c`可以不指定需要生成的文件(不需要`-o <file>`), 自动生成同名不同后缀的文件, 例如:

```
gcc -S file.c # 生成汇编
gcc -c file.c # 生成目标文件
gcc file.c -o a.out # 生成可执行文件
```



关于可执行文件

Linux的可执行文件不依赖后缀, 因此不管是`hello.out`还是`hello`, 都是可执行文件

而Windows以来的是`.exe`后缀, 根据这个后缀来判断是不是ELF

as--汇编器(Assembler)

安装

```
sudo apt install binutils
```

as用于将**汇编代码**(`.s`)转为**机器码**(`.o`)

```
as hello.s -o hello.o
```

生成的`.o`还不能运行, 需要进行后续的**链接**

ld--链接器(Linker)

安装

```
sudo apt install binutils
```

作用:

- 把多个`.o`文件**链接**成一个可执行文件
- 处理符号解析

- 生成ELF文件

```
ld hello.o -o hello
```

直接用`ld`很复杂, 需要执行很多东西, 比如: `ld hello.o -lc`表示链接libc

gdb--调试工具

`~/.gdbinit`配置

```
set debuginfod enabled off # 关闭gdb联网和下载调试信息询问
```

常见操作

1. 断点(BreakPoints)
   `b _start`: 设置入口断点
   `b *_start+1`: 入口的下一步设置断点
   `info b`: 查看所有断点
2. 运行(run)
   `r`或`run`: 程序运行
   `si`: 单步执行
   `n`: 下一步
3. 查看寄存器
   需要在程序开始运行后进行查看
   `info registers`: 查看当前架构下的寄存器
4. 查看变量内存
   `x/4xb &value1`
   1. `x`: examine 查看
   2. `4`: 4个元素
   3. `x`: 16进制 `d`是十进制, `t`是二进制
   4. `b`: byte, `h`是两字节(half word), `w`是四字节(word), `g`是八字节(giant word)

编译时的`-g`参数

`-g`的作用是**在可执行文件中加入调试符号信息（debug symbols）**
这些信息会**存储在 ELF 文件的**`debug section`中。

初学gdb来调试汇编程序(`.s`文件)时, 可以在汇编时(as)加上`-g`参数, 这样能使得能够显示机器执行对应的汇编指令, 更便于初学者对汇编程序的学习

`-g`用在哪？

1. `gcc -g test.c -o test`
2. `as -g 1007.s -o 1007.o`
3. `ld`没有`-g`, h或者说不必要

调试汇编代码

1. `x` 查看内存
    1. `&` 后面跟变量的名称来查看变量地址对应的内存, 即查看变量
    2. `$` 后面跟寄存器名称来根据该寄存器地址查看内存
2. `p` 查看值
    1. `print $eax` 查看寄存器的值,或简写 `p $eax`
    2. `p/x $eax` 以十六进制查看, `d` 为十进制, `t` 为二进制
    3. `p $eax $ebx` 查看多个
    4. 对于汇编里面的变量, `print` 时要加上该变量定义时的类型
3. `where` 查看当前执行到哪一行了
4. `%eip` 是程序指针寄存器, 它指向**即将执行的指令的地址**(指令实际上以机器码形式存在)

    ```
    (gdb) x/10i $eip
    => 0x8048000 <_start>:    nop
    0x8048001 <_start+1>:    movl $1,%eax
    0x8048006 <_start+6>:    movl $0,%ebx
    0x804800b <_start+11>:   int $0x80
    ```

    这里的 `i` 表示 `instruction` 指令的意思,表示的是一种x查看内存的显示格式, `x/10i` 会将内存内容解释为**机器指令, 并显示对应的汇编代码**
5. `disassemble`: 反汇编
    1. `disassemble`: 反汇编当前函数
    2. `disassemble 函数名`: 反汇编指定函数
    3. `disassemble 0x08048000, 0x08048010`: 反汇编指定函数的指定地址

gdb操作总览

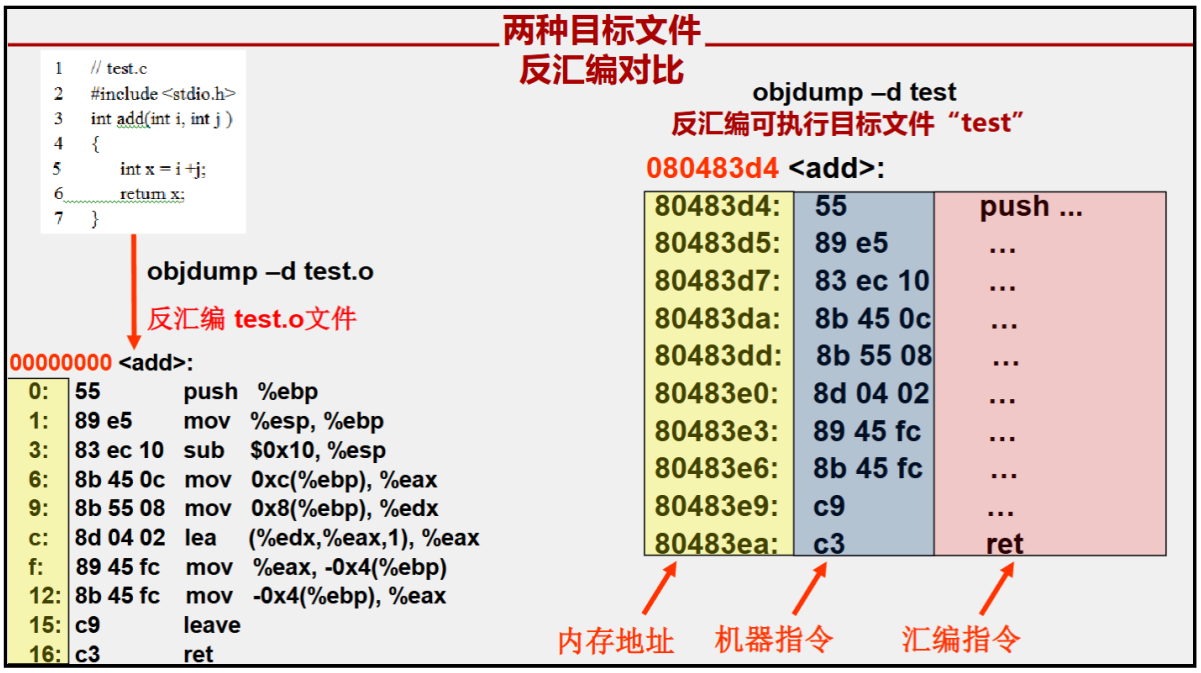
IDE 操作	GDB 指令	指令说明
启动调试	`r` (run)	运行程序，直到遇到断点/程序结束/崩溃；支持传参：`r 参数1 参数2`
暂停程序	`ctrl + c`	程序运行中强行暂停，进入调试界面
单步执行（逐行，不进函数）	`n` (next)	单步走一行代码，遇到函数调用时不进入函数内部
单步执行（逐行，进函数）	`s` (step)	单步走一行代码，遇到函数调用时进入函数内部
跳出当前函数	`finish`	执行完当前函数，返回到调用该函数的下一行
继续运行（到下一个断点）	`c` (continue)	从暂停处继续运行，直到下一个断点/程序结束
设置行断点	`b 行号` 或 `b 函数名`	例：`b 8`（第8行断点）、`b sum`（sum函数入口断点）

IDE 操作	GDB 指令	指令说明
设置条件断点	`b 行号/函数名 if 条件`	例：`b 8 if i==5`（仅当i=5时触发第8行断点）
查看所有断点	`info b`(info breakpoints)	列出所有断点的编号、位置、命中次数
删除断点	`delete 断点编号` 或 `d 断点编号`	例：`delete 1`（删除编号1的断点）；`d` 不带编号删除所有断点
禁用/启用断点	`disable 编号`/`enable 编号`	临时禁用断点（不删除），需要时再启用
查看变量值	`p 变量名`(print)	例：`p sum`、`p a[i]`、`p len`；支持表达式：`p a[i]+10`
实时监控变量	`display 变量名`	每次单步调试时，自动显示该变量的值（类似 IDE 的变量监控窗口）
取消变量监控	`undisplay 编号`	例：`undisplay 1`（取消编号1的监控变量）
查看调用栈	`bt`(backtrace)	显示当前程序的调用栈（哪个函数调用了哪个函数，定位崩溃位置）
切换栈帧	`frame 编号`	例：`frame 0`（切换到崩溃的函数栈帧）、`frame 1`（切换到调用者栈帧）
查看当前代码上下文	`l`(list)	显示当前执行位置附近的代码（默认显示10行）；`l 行号` 显示指定行附近代码
查看内存内容（内存窗口）	`x/格式 内存地址`	例：`x/10xw 0xffffcd00`（以4字节16进制显示10个内存单元）；`x *_start+1`
修改变量值（实时编辑）	`set var 变量名=值`	例：`set var len=5`、`set var a[0]=100`（调试时临时修改，无需重新编译）
配置 GDB 显示格式	`set print pretty on`	格式化输出结构体/数组（默认紧凑显示，开启后更易读）
配置代码显示行数	`set listsize 行数`	例：`set listsize 20`（每次list显示20行代码，默认10行）
查看寄存器值（寄存器窗口）	`info registers`	显示所有寄存器的值；`p $rax`（查看单个寄存器rax的值）
退出调试	`q`(quit)	退出 GDB

objdump--反汇编工具(重要)

什么是反汇编

反汇编(Disassembly) 是将机器码转换为汇编指令的过程, 将编译器生成的可执行文件/目标文件还原为汇编指令, 以便于调试和理解程序的运行原理



可以用在用gdb调试时, 用`disassemble`指令来反汇编当前函数或指定函数.
也可以用下面的objdump指令来反汇编可执行文件或目标文件

安装

```
sudo apt install binutils
```

作用

- **反汇编**可执行文件(ELF, .out)
- 查看机器指令
- 分析ELF

```
objdump -d a.o # 查看目标文件
objdump -d a # 查看可执行文件
```

按32位模式反汇编时, 最好加上参数`-d -m i386`.
`objdump -d -m i386 a.out`

0.1.2 64位支持32位程序

Ubuntu的机制是: **Multiarch**, 一个64位系统可以安装64位库 和 32位库

```
/usr/lib/x86_64-linux-gnu    (64位)
/usr/lib/i386-linux-gnu      (32位)
```

开启32位架构

1. `sudo dpkg --add-architecture i386`: 允许apt安装32位软件包
2. `sudo apt update`: apt 需要重新下载 i386 软件包列表

安装32位运行库

1. `sudo apt install libc6:i386`: 安装后系统会有`/lib/i386-linux-gnu/libc.so.6`

安装32位编译支持

1. `sudo apt install gcc-multilib g++-multilib`: 安装32位程序编译必须组件

验证32位程序

1. `dpkg --print-foreign-architectures`: 应该输出`i386`
2. `dpkg -l | grep libc6`: 应该看到`libc6: amd64`和`libc6:i386`, 表示64位和32位库同时存在

0.1.3 64位运行32程序

每个都可`file <file-name>`来查看架构, 希望看到: `ELF 32-bit LSB executable`

32位编译c程序

```
gcc -m32 test.c -o test32
```

编译32位汇编程序`.s`

1. `as --32 1007.s -o 1007.o`: 汇编
2. `ld -m elf_i386 1007.o -o 1007`: 链接

需要更详细的调试时可以加入`-g`:

1. `as --32 -g 1007.s -o 1007.o`
2. `ld -m elf_i386 1007.o -o 1007`

ld链接缺少参数导致报错

当运行`ld -m elf_i386 1007.o -o 1007`时, 可能会报错, 说没有找到某标准函数的定义, 所以需要的指定链接标准库, 例如: [有两种方式]

1. `ld -m elf_i386 1007.o -o 1007 -lc -dynamic-linker` : 链接C标准库
2. `gcc -m32 1007.o -o 1007` : 直接用gcc来链接, gcc会自动调用ld并且链接标准库

0.2 Introduction

0.2.1 前言

本书面向想**通过学习计算机系统的内在运作而写出更好程序**的人
本书解释了所有计算机系统的本质概念, 并展示这些概念是如何实实在在影响应用程序的正确性, 性能和实用性的.

0.2.2 背景知识

本书前几章揭示了C语言程序和他们对应的机器语言程序之间的交互作用. 机器语言示例都是运行在x86-64处理器上的GNU GCC编译器生成的

```
高级语言(C语言) --> 编译器(ccl) --> 汇编语言 --(根据指令集)-->
汇编器(as) --> 机器码 --> 放到内存(RAM) --> 处理器(CPU) --> 执行指令
```

0.2.3 阅读此书

本书包含很多**练习题和作业题**
所有用到的源程序代码可在CS:APP主页获取

0.2.4 32位架构 vs 64位架构

本书要学习的是典型的 GNU AT&T语法的x86汇编

x86-32 与 x86-64 寄存器区别

x86-32只有八个通用寄存器

32位寄存器	16位	8位	常见用途
eax	ax	al / ah	返回值
ebx	bx	bl / bh	参数/通用
ecx	cx	cl / ch	循环计数
edx	dx	dl / dh	IO
esi	si	—	栈顶
edi	di	—	栈帧
ebp	bp	—	source index
esp	sp	—	dest index

x86-64寄存器数量增加到16个

64位	32位	16位	8位	\	64位	32位	16位	8位
rax	eax	ax	al	\	r8	r8d	r8w	r8b
rbx	ebx	bx	bl	\	r9	r9d	r9w	r9b
rcx	ecx	cx	cl	\	r10	r10d	r10w	r10b
rdx	edx	dx	dl	\	r11	r11d	r11w	r11b
rsi	esi	si	sil	\	r12	r12d	r12w	r12b
rdi	edi	di	dil	\	r13	r13d	r13w	r13b
rbp	ebp	bp	bpl	\	r14	r14d	r14w	r14b
rsp	esp	sp	spl	\	r15	r15d	r15w	r15b

0.3 Study Methods

根据C代码解读汇编

在阅读汇编代码时, 如何将**其中的寄存器与C语言的源代码中的变量联系起来是很关键的

一般建议在阅读汇编代码时, **准备一个表格来记录寄存器中的值和对应的变量名**, 以此来跟踪寄存器中变量和数值的变化, 尤其当寄存器被反复使用来保存不同的C语言变量时, 这样的表格对于判断特定寄存器中变量和其实时数值显得尤为重要

寄存器	变量	初始值
`%eax`	result	1
`%edx`	n	n

第一章 计算机系统漫游

- 1.1 信息 = 位(bit) + 上下文
- 1.2 程序的生命周期
- 1.3 了解编译系统工作机制的益处
- 1.10 小结

本章节通过**跟踪hello程序的生命周期开始对系统的学习**----从他被程序员创建开始, 到在系统上运行, 输出简单的消息, 然后终止.
沿着这个过程, **本章简要介绍一些逐步出现的关键概念, 专业术语和组成部分.**

```c
// hello.c
#include <stdio.h>
int main()
{
    printf("hello, world\n");
    return 0;
}
```

1.1 信息 = 位(bit) + 上下文

#	i	n	c	l	u	d	e	SP	<	s	t	d	i	o	.
35	105	110	99	108	117	100	101	32	60	115	116	100	105	111	46
h	>	\n	\n	i	n	t	SP	m	a	i	n	(	)	\n	{
104	62	10	10	105	110	116	32	109	97	105	110	40	41	10	123
\n	SP	SP	SP	SP	p	r	i	n	t	f	(	"	h	e	l
10	32	32	32	32	112	114	105	110	116	102	40	34	104	101	108
l	o	,	SP	w	o	r	l	d	\	n	"	)	;	\n	SP
108	111	44	32	119	111	114	108	100	92	110	34	41	59	10	32
SP	SP	SP	r	e	t	u	r	n	SP	0	;	\n	}	\n	
32	32	32	114	101	116	117	114	110	32	48	59	10	125	10	

图 1-2 hello.c 的 ASCII 文本表示

`hello.c` 程序是以字节(byte)序列的方式存储在文件中, 这样只有ASCII字符构成的文件称为**文本文件**, 其他所有文件称为**二进制文件**

系统中所有信息 (包括磁盘文件, 内存中的程序, 内存中存放的用户数据...) 都是由一串比特(bit)表示的. 区分不同数据对象的唯一方法是**读到这些数据对象是的上下文**

> C语言是用作Unix系统的程序语言被开发出来的.
> C语言是**系统级编程的首选**, 但对于应用级程序的编写, C语言缺乏非常有用的抽象的显示支持,
> **C++和Java**等针对应用级程序的新程序语言更能胜任

1.2 程序的生命周期

`hello.c` 首先要被编译, 生成 `可执行目标文件`

```
gcc -o hello hello.c
```

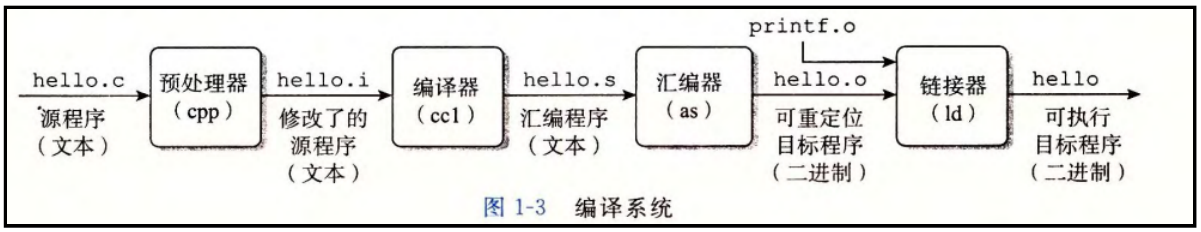


图 1-3　编译系统

分四个阶段, `cpp, ccl, as, ld`共同构成了**编译系统(compilation system)**

- 编译系统 = 一整套把高级语言 → 可执行程序的工具链
- gcc = 前端总指挥（调用后面所有工具）
- as = 汇编器（汇编 → 机器码）
- ld = 链接器（把多个机器码文件拼在一起）

1.3 了解编译系统工作机制的益处

1. 优化程序性能
   e.g. 一个switch语句比多个if-else语句高效的多
2. 理解链接时出现的错误
   一些最令人困扰的**程序错误往往都与链接器操作有关**
3. 避免安全漏洞
   e.g. 缓冲区溢出错误

1.10 小结

计算机系统是由**硬件和系统软件组成**的, 他们共同协作以运行应用程序. 计算机内部的信息**被表示为一组组的位**, 他们一句**上下文**有不同的解释方式. 程序被其他程序**(编译系统)**翻译成不同的形式, 开始时是ASCII文本, 然后被编译器(as)和链接器(ld)翻译成二进制可执行文件

处理器(CPU)读取并解释存放在内存里的二进制指令. 因为计算机花费了**大量的时间**在内存, I/O 设备和CPU寄存器之间复制数据, 所以将系统中的**存储设备划分成层次结构** ---- CPU寄存器在顶部, 接着是多层的硬件高速缓存存储器(L1, L2, ...), DRAM 主存, 和磁盘存储器. 在层次模型中, 位于更高层的存储设备, 比低层的存储设备**要更快**, 单位比特造价也更高. 层次结构中较高层次的存储设备可以作为较低层次设备的高速缓存. 通过理解和运用这种存储层次结构的知识, 程序员可以优化C程序的性能.

操作系统内核是**应用程序和硬件之间的媒介**. 它提供了三个基本的抽象: (1) **文件**是对 I/O 设备的抽象; (2) **虚拟内存**是对主存和磁盘的抽象; (3) **进程**是处理器, 主存和 I/O 设备的抽象

最后网络提供了计算机系统之间通信的手段. 从特殊的系统角度看, **网络就是一种 I/O 设备.**

第二章 信息的表示与处理

- 2.1 信息存储
  - 2.1.1 十六进制表示
  - 2.1.2 字(word)数据大小
  - 2.1.3 寻址和字节顺序
  - 2.1.4~2.1.5 表示字符串, 代码
  - 2.1.6~2.1.9 布尔代数, 位级运算, 逻辑运算, 移位
- 2.2 整数表示
  - 2.2.1 有符号数(补码)和无符号数之间转换
  - 2.2.2 C语言有符号数和无符号数
  - 2.2.3 扩展一个数字的位表示
  - 2.2.4 截断数字
- 2.3 整数运算
  - 2.3.1 无符号数加法
  - 2.3.2 有符号数(补码)加法
  - 2.3.3 整数乘法
- 2.4 浮点数表示
  - 2.4.1 二进制小数
  - 2.4.2 IEEE 浮点表示
  - 2.4.3 舍入(rounding)
  - 2.4.4 浮点运算
- 2.5 小结

2.1 信息存储

大多数计算机使用*8-bits的块*, 或者*字节(byte)*, 作为最小的**可寻址**的内存单元, 而不是单独的一个位(bit).

机器级程序将内存视为一个非常大的字节数组, 成为**虚拟内存(virtual memory)**
内存的每个字节都有一个唯一的数字来表示, 称为他的**地址(addr)**, 所有可能地址的集合就成为**虚拟地址空间(virtual address space)**

实际运行时, 存储空间会被划分为可管理的单元, 来存放不同的**c程序对象(program object)**, 即*程序数据, 指令和控制信息*.

> **C语言的指针**
> C语言的指针是一个重要特性, 指针有两个方面:

1. *值*: 表示某个对象的位置, 即**地址(address)**
2. *类型*: 表示那个位置上所存储对象的类型

2.1.1 十六进制表示

十六位进制(hexadecimal)的每两位数可构成代表一个字节(byte, 8-bits).

十六进制数字	0	1	2	3	4	5	6	7
十进制值	0	1	2	3	4	5	6	7
二进制值	0000	0001	0010	0011	0100	0101	0110	0111
十六进制数字	8	9	A	B	C	D	E	F
十进制值	8	9	10	11	12	13	14	15
二进制值	1000	1001	1010	1011	1100	1101	1110	1111

图 2-2　十六进制表示法。每个十六进制数字都对 16 个值中的一个进行了编码

2.1.2 字(word)数据大小

每台计算机都有一个*字长(word size)*, 指明**指针数据**(即代表地址)的标称大小, 对于一个字长是w位的机器而言, 虚拟地址的范围是$(0, 2^W - 1)$, w通常是32或64

C语言的数据类型字长

C声明		字节数	
有符号	无符号	32位	64位
[signed] char	unsigned char	1	1
short	unsigned short	2	2
int	unsigned	4	4
long	unsigned long	4	8
int32_t	uint32_t	4	4
int64_t	uint64_t	8	8
char *		4	8
float		4	4
double		8	8

2.1.3 寻址和字节顺序

对于跨越多字节的程序对象, 要关注两个规则:

1. 该对象的地址是什么
2. 在内存中如何排列这些字节

字节排列两种方式[e.g. 对`0x01234567`而言, 最高有效字节是指`01`]:

1. **大端法**: 最高有效字节在最前面
2. **小端法**: 最低有效自己在最前面

无论大小端, 对象的地址都是**最前面字节的addr**

2.1.4~2.1.5 表示字符串, 代码

不同机器类型会使用不兼容的指令和编码方式.

2.1.6~2.1.9 布尔代数, 位级运算, 逻辑运算, 移位

一个bit的布尔代数运算

~	
0	1
1	0

&	0	1
0	0	0
1	0	1

|	0	1
0	0	1
1	1	1

^	0	1
0	0	1
1	1	0

图 2-7 布尔代数的运算。二进制值 1 和 0 表示逻辑值 TRUE 或者 FALSE，而运算符 ~、&、| 和^分别表示逻辑运算 NOT、AND、OR 和 EXCLUSIVE-OR

可以扩展到**位向量**的运算, 即向量参数的对应元素之间的布尔代数运算

C语言支持**按位布尔运算**, `| 或  & 与  ~ 取反  ^ 异或`
C语言**逻辑运算**: `|| 或  &$ 与  !取反`

C语言**移位运算**:

操作	值
参数 x	[01100011] [10010101]
x << 4	[0011*0000*] [0101*0000*]
x >> 4（逻辑右移）	[*0000*0110] [*0000*1001]
x >> 4（算术右移）	[*0000*0110] [*1111*1001]

2.2 整数表示

对于一个二进制数[向量$\vec{x} = [x_{w-1}, x_{w-2}, \ldots, x_0]$]:

1. 若当做无符号数, 进行$\sum_{i=0}^{w-1} x_i 2^i$运算, 就是$B2U_w(\vec{x})$, 该函数是**双射**, 无符号数的取值范围是$(0, 2^w - 1)$
2. 若当做有符号数(无论正数还是负数), 最高位解释为**负权**, 进行$-x_{w-1}2_{w-1} + \sum_{i=0}^{w-2} x_i 2^i$, 就是$B2T_w(\vec{x})$, 这就是**补码**, 该函数也是一个双射, 补码代表的整数的取值范围是$(-2^{w-1}, 2^{w-1} - 1)$

补码的数学定义: 对于w位系统, 任何整数x的补码表示为: $补码(x) = x \bmod 2^w$
例子(4位):

```
x = +3:  补码(3) = 3 mod 16 = 3  = 0011
x = -3:  补码(-3) = -3 mod 16 = 13 = 1101
x = +7:  补码(7) = 7 mod 16 = 7  = 0111
x = -8:  补码(-8) = -8 mod 16 = 8 = 1000
```

2.2.1 有符号数(补码)和无符号数之间转换

大多数C语言的*强制类型转换*, 结果就是**保持位值不变, 只是改变了解释这些位的方式.**
据此可定义有符号数和无符号数之间的转换[二进制这个串是不变的]:

$$T2U_w(x) \doteq B2U_w(T2B_w(x_t))$$
$$U2T_w(x) \doteq B2T_w(U2B_w(x_t))$$

实际有推导得:

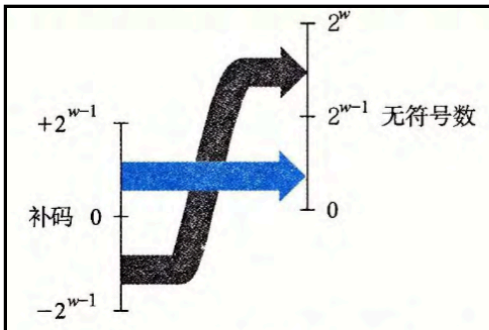


图 2-17 从补码到无符号数的转换。函数 $T2U$ 将负数转换为大的正数

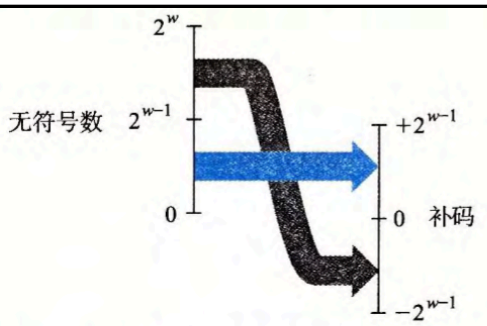


图 2-18 从无符号数到补码的转换。函数 $U2T$ 把大于 $2^{w-1}-1$ 的数字转换为负值

$$U2T_w(u) = \begin{cases} u, & u \le TMax_w = 2^{w-1}-1 \\ u-2^w, & u > TMax_w \end{cases}$$

$$T2U_w(x) = \begin{cases} x+2^w, & x < 0 \\ x, & x \ge 0 \end{cases}$$

2.2.2 C语言有符号数和无符号数

大多数C语言的*强制类型转换*, 结果就是**保持位值不变, 只是改变了解释这些位的方式.**
据此可定义有符号数和无符号数之间的转换[二进制这个串是不变的]:

对于同时包含有符号数和无符号数的表达式, C语言会隐式的将有符号参数强制类型转换为无符号数, 然后进行表达式求值

2.2.3 扩展一个数字的位表示

1. **无符号数**:
   直接使用**零扩展**, 即在高位补零.
2. **有符号数(补码)**:
   需要进行**符号扩展**, 即在高位补符号位(负数为1, 非负数为0).
   这样进行拓展的原因是, 经证明, 符号扩展后的有符号数(补码)的值和原来是一样的, 这在计算机中能带来很多方便

2.2.4 截断数字

1. **截断无符号数**
   令 $\vec{x}$ 等于位向量 $[x_{w-1}, x_{w-2}, \cdots, x_0]$, 而 $\vec{x}'$ 是将其截断为 k 位的结果:
   $\vec{x}' = [x_{k-1}, x_{k-2}, \cdots, x_0]$。令 $x = B2U_w(\vec{x})$, $x' = B2U_k(\vec{x}')$。则 $x' = x \bmod 2^k$。**[推导可见csapp, P57]**
2. **截断有符号数(补码)**
   令 $\vec{x}$ 等于位向量 $[x_{w-1}, x_{w-2}, \cdots, x_0]$, 而 $\vec{x}'$ 是将其截断为 k 位的结果:
   $\vec{x}' = [x_{k-1}, x_{k-2}, \cdots, x_0]$。令 $x = B2U_w(\vec{x})$, $x' = B2T_k(\vec{x}')$。则
   $x' = U2T_k(x \bmod 2^k)$。
   在这个公式中, $x \bmod 2^k$ 将是 0 到 $2^k - 1$ 之间的一个数。对其应用函数 $U2T_k$ 产生的效果是把最高有效位 x_{k-1} 的权重从 2^{k-1} 转变为 -2^{k-1}。举例来看, 将数值 $x = 53191$ 从 int 转换为 short。由于 $2^{16} = 65536 \ge x$, 我们有 $x \bmod 2^{16} = x$。但是, 当我们把这个数转换为 16 位的补码时, 我们得到 $x' = 53191 - 65536 = -12345$。

2.3 整数运算

> 在机器里, 由于**数据的位数都是有限的**, 因此我们不能基于普通的算数加法去看待机器的运算, 我们需要**基于整数数据的表示方法(补码, 截断)定义一些特殊的'加法', '乘法'运算**以便我们去看待机器里面的数据处理, 另一方面也要知晓这些定义在机器上的加法乘法和算术的加法乘法有什么不一样的地方, 这样才能保证以后写出正确的无漏洞的程序

2.3.1 无符号数加法

为定义参数x和y定义运算$+_w^u$, 其中$0 \leq x, y \leq 2^w$, 该操作就是**把整数和x+y截断为w位得到的结果, 再把结果看做是一个无符号数**
对满足 $0 \leq x, y < 2^w$ 的 x 和 y 有:

$$x +_w^u y = \begin{cases} x + y, & x + y < 2^w \\ x + y - 2^w, & 2^w \leq x + y < 2^{w+1} \end{cases}$$

只可能溢出一位, 因此可以看作是从$w+1$位到w位的截断:

$$x + y = (x + y) \bmod 2^w$$

2.3.2 有符号数(补码)加法

在计算机这个机器中, 加法运算并不区分无符号数和有符号数, 都是对两个二进制数(无论是无符号的二进制还是有符号补码)进行位级加法(**即二级制串就和十进制串一个性质**). 取得的结果如果有最高位进位则直接截断, 然后返回结果, 只是对结果的解释方式不同.
综上,定义计算补码的加法可以**当做先执行无符号数的加法, 然后将结果按补码来解释**:

$$\begin{aligned} x +_w^t y &= U2T_w(T2U_w(x) +_w^u T2U_w(y)) \\ &= U2T_w[(x_{w-1}2^w + x + y_{w-1}2^w + y) \bmod 2^w] \\ &= U2T_w[(x + y) \bmod 2^w] \end{aligned}$$

进一步分情况讨论, 对满足 $-2^{w-1} \leq x, y \leq 2^{w-1} - 1$ 的整数 x 和 y, 有:

$$x +_w^t y = \begin{cases} x + y - 2^w, & 2^{w-1} \leq x + y (\text{正溢出}) \\ x + y, & -2^{w-1} \leq x + y < 2^{w-1} (\text{正常}) \\ x + y + 2^w, & x + y < -2^{w-1} (\text{负溢出}) \end{cases}$$

该公式用于判断普通的十进制数之间的加法结果

接下来, 假如只用二进制补码之间进行加法运算, 则可以直接按以下这样:

$$[x + y]_{补} = [x]_{补} + [y]_{补}$$
$$[x - y]_{补} = [x]_{补} + [-y]_{补}$$

即按正常的二进制运算(包括符号位), 最后如果有进位, 就截断, 如果没有, 就直接返回结果
其中计算$-y$的补码, 直接将y的补码取反(包括符号位), 然后加1即可

例题

用二进制值把表格补充完整
解答:

				溢出情况
`[10100]`	`[10001]`	[100101]	[00101]	负溢出

x	y	$x+y$	$x +_5^t y$	溢出情况
[11000]	[11000]	[110000]	[10000]	正常
[10111]	[01000]	[11111]	[11111]	正常
[01100]	[00100]	[10000]	[10000]	正溢出

具体计算：

- 第一行：
  - `x = [10100]: -16 + 4 = -12`
  - `y = [10001]: -16 + 1 = -15`
  - `x + y = [100101]: -32 + 5 = -27`
  - `x +_5^t y = [00101]: 5`
- 第二行：
  - `x = [11000]: -16 + 8 = -8`
  - `y = [11000]: -16 + 8 = -8`
  - `x + y = [110000]: -32 + 16 = -16`
  - `x +_5^t y = [10000]: -16`
- 第三行：
  - `x = [10111]: -16 + 7 = -9`
  - `y = [01000]: 8`
  - `x + y = [11111]: -1`
  - `x +_5^t y = [11111]: -1`
- 第四行：
  - `x = [01100]: 12`
  - `y = [00100]: 4`
  - `x + y = [10000]: -16`
  - `x +_5^t y = [10000]: -16`

2.3.3 整数乘法

有符号和无符号数的乘法都定义为对二进制串乘积多位结果 (最多$2W$位) 的截断w位, 只取低w位, 计算机用的是**同一条乘法指令**

1. **无符号乘法**
   对满足$0 \leq x, y \leq UMax_w$的 x 和 y 有:

$$x *_w^u y = (x \cdot y) \bmod 2^w$$

2. **补码乘法**
   对满足$0 \leq x, y \leq UMax_w$的 x 和 y 有:

$$x *_w^t y = U2T((x \cdot y) \bmod 2^w)$$

对于整数乘法这一运算而言, 如果一个整数**乘以2的幂**, 那么大多数编译器通常会聪明直接这样的运算转为**移位操作**.

2.4 浮点数表示

2.4.1 二进制小数

类似于十进制小数, 二进制小数可表示为:

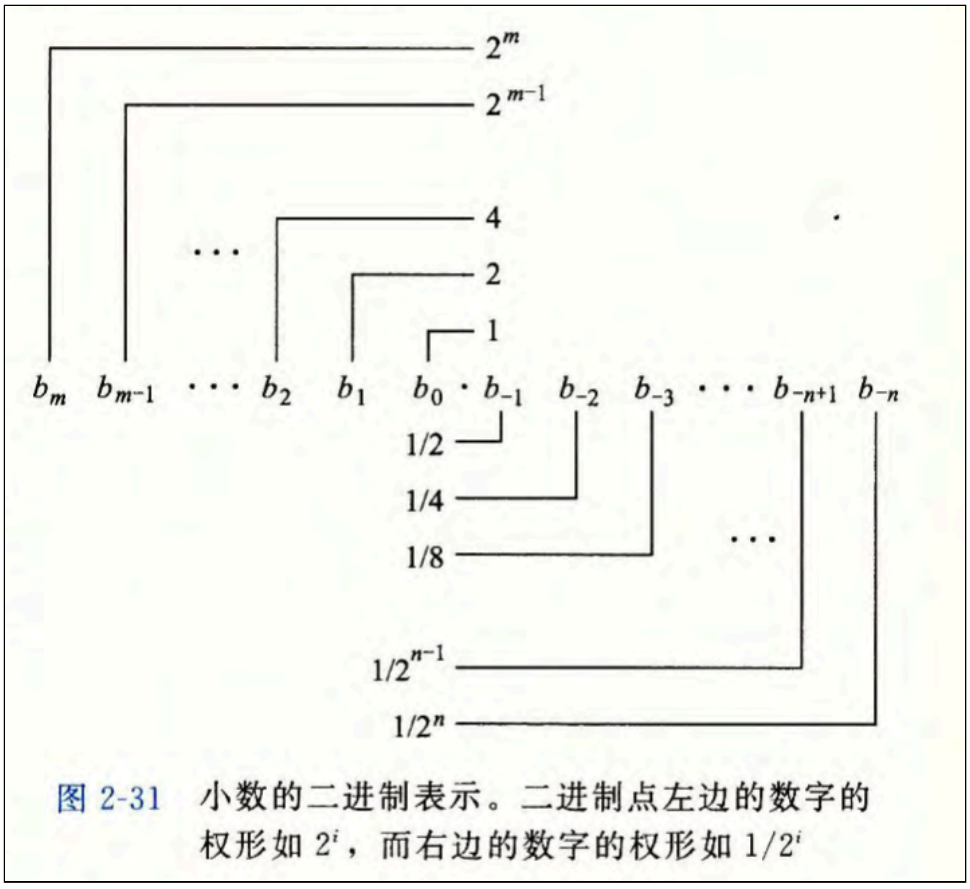


图 2-31 小数的二进制表示。二进制点左边的数字的权形如 2^i，而右边的数字的权形如 $1/2^i$

二进制小数点左移一位, 相当于这个小数在十进制上被2除;右移一位, 相当于这个小数在十进制上乘以2. 由于有限长度的编码, 二进制小数表示的精度有限

2.4.2 IEEE 浮点表示

IEEE 浮点标准用$V = (-1)^s \times M \times 2^E$的形式来表示一个数

- **符号(sign)** s决定小数的正负, 用一个单独的符号位来编码
- **尾数(significand)** M是一个二进制小数, 他的范围是$1\ 2 - \eta$或者是$0\ 1 - \eta$
- **阶码(exponent)** E的作用是对浮点数加权, 这个权重是2的E次幂(也可能是负数)

位分配

- **k位的阶码字段** $exp = e_k \dots e_1 e_0$编码阶码E
- **n位的小数字段**$frac = f_{n-1} \dots f_1 f_0$编码尾数$M$, 但是编码出来的值也依赖于阶码字段的值是否等于0.

以单精度格式(k=8位阶码, n=23位尾数)为例, 讨论编码如何转为小数V

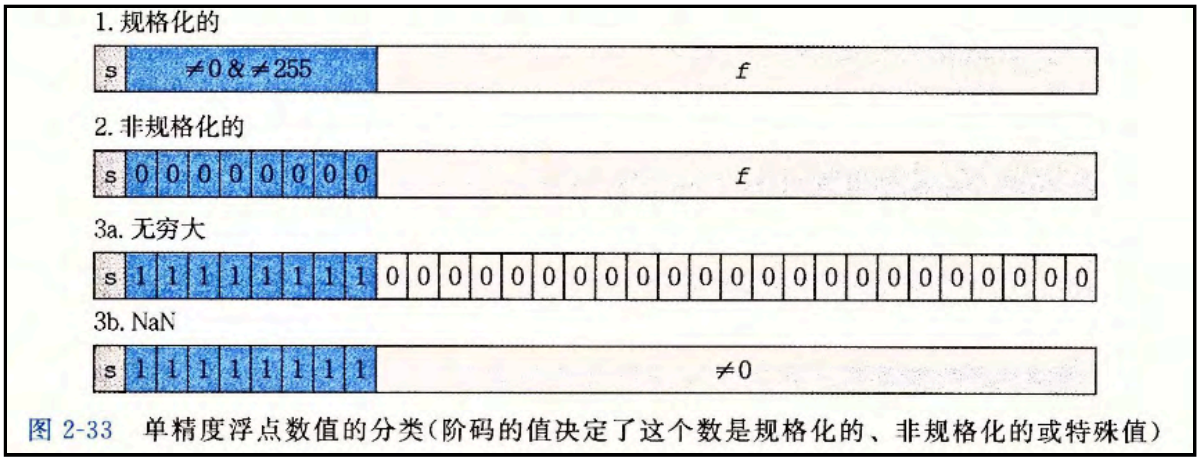


图 2-33 单精度浮点数值的分类(阶码的值决定了这个数是规格化的、非规格化的或特殊值)

1. **规格化的值**(exp的位既不全为0也不全为1时)
   此时: $E = e - Bias$, 其中e为$e_{k-1} \dots e_1 e_0$, $Bias = 2^{k-1} - 1$
   $M = 1 + f$, 其中小数值$f = f_{n-1} f_{n-2} \dots f_0$, 也就是二进制$0.f_{n-1} \dots f_1 f_0$, 表示的是$[1, 2)$范围的小数
2. **非规格化的值**(exp的位全为0)
   此时: $E = 1 - Bias$, 其中e为$e_{k-1} \dots e_1 e_0$, $Bias = 2^{k-1} - 1$
   $M = f$, 其中$f = f_{n-1} f_{n-2} \dots f_0$, 表示的是小于1的小数
3. **特殊值1**(当exp的位全为1, 且frac的位全为0时)
   此时整个编码**表示无穷值**, s=0时是+无穷, s=1时是-无穷
4. **特殊值2**(exp的位全为1, frac的位不全为0时)
   结果只被称为*NaN (Not a Number)*, 即一些不能表示的值

单精度浮点数和双精度浮点数的区别在于阶码字段和小数字段的位数不同, 但他们的编码方式是一样的
单精度浮点数: e=8位, f=23位, Bias=127
双精度浮点数: e=11位, f=52位, Bias=1023

2.4.3 舍入(rounding)

IEEE 浮点格式定义了四种舍入模式

- **向偶数舍入(向最近舍入)**
  对十进制而言, 四舍六入, 五向偶数舍入
- **向0舍入**
  正数向下舍入, 负数向上舍入
- **向上舍入**
  都向上舍入, 无论正负
- **向下舍入**
  都向下舍入, 无论正负

向偶数舍入(向最近舍入)

综上所述, 如果以形式`1.RR..RDD..D`表示浮点数(R表示有效位, 或保留位, 而D表示舍去位), Roundings to nearest/even舍入规则就是:

1. 如果`DD..D` < `10..0`, 则**向下舍入**
2. 如果`DD..D` > `10..0`, 则**向上舍入**
3. 如果`DD..D` = `10..0`, 则**向最近偶数舍入**, 细则如下:

- 即此时把最低有效位(LSB)为1时当做奇数, 为0时当做偶数
- 如果`RR..R` = `XX..0` (X表示任意值, 0或1), 则向下舍入
- 如果`RR..R` = `XX..1`, 则向上舍入

例如

舍入到 1/4 (小数点右边两位)				
Value	Binary Rounded	Rounded Value	Action	
2 3/32	10.00011_2	10.00_2	(<1/2—down)	2
2 3/16	10.00110_2	10.01_2	(>1/2—up)	2 1/4
2 7/8	10.11100_2	11.00_2	(1/2—up)	3
2 5/8	10.10100_2	10.10_2	(1/2—down)	2 1/2

2.4.4 浮点运算

IEEE标准指定了一个简单的规则, 来**确定浮点数诸如加法和乘法这样的算术运算的结果。**把浮点值x和y看成实数, 而某个运算⊙定义在实数上, 计算将产生Round(x⊙y), 这是**对实际运算的精确结果进行舍入后的结果**

对于机器而言, 浮点数的乘法和加法(定义运算符为`+^f`和`+^f`) 就变成了:

$$x +^f y = Round(x + y)$$
$$x *^f y = Round(x * y)$$

浮点加法

两个浮点数再进行加法运算$(-1)^{s_1} M_1 2^{E_1} + (-1)^{s_2} M_2 2^{E_2}$时, 需要分一下步骤:

1. **对阶**
   先对阶码E_1和E_2进行对齐, **小阶向大阶对齐**(假设$E_1 < E_2$)
   - 尾数M_1需要右移(小数点左移)$E_2 - E_1 = \Delta E$(数值不变但精度变差)
   - 同时小阶码E_1要增加ΔE, 已达到和E_2相等
2. **加尾数**
   对齐后的尾数M_1和M_2进行加法运算
3. **规格化**
   尾数加法后要进行规格化, 是的尾数在$[1, 2)$范围内.
4. **舍入**
   根据舍入模式, 对结果进行舍入.

问: 为什么对阶时要小阶向大阶对齐?
如果选择**大阶向小阶对齐**, 则需要将大阶数的尾数左移, 此时可能会导致**尾数的高位数字丢失**, 相较于地位的丢失, 高位丢失可能会导致更大的精度损失.

浮点乘法

两个浮点数再进行乘法运算$(-1)^{s_1} M_1 2^{E_1} \times (-1)^{s_2} M_2 2^{E_2}$时, 需要分一下步骤:

1. 符号位异或($s = s_1 \oplus s_2$)得到乘积的符号位s
2. 尾数相乘($M = M_1 \times M_2$)得到乘积的尾数M
3. 阶码相加($E = E_1 + E_2$)得到乘积的阶码E

4. 调整: 如果$M \geq 2$, 则M右移(小数点左移)一位, E增加1, 直到$M < 2$; 然后将M舍入到frac小数字段的位数范围

2.5 小结

数据被编码成位(bit, 比特), 通常组织成 *字节序列* . 有*不同的编码方式*来表示整数, 实数和字符串. 不同计算机模型的在编码数字和多字节数据中的字节顺序时使用不同的约定.

C语言的设计可以保通不同*字长(word_size)* 和数字编码的实现. 64位的机器逐渐普及, 突破32位程序具有的 *4GB(2^{32})* 地址限制.

大多数机器对整数使用*补码编码*, 而对浮点数使用*IEEE 标准 754 编码*. 在位级上理解这些编码, 并**能理解机器上定义的算数运算的数学特性**, 对于想要编写的程序**能在全部数值上正确运行的程序员**来说, 是很重要的

在相同长度的无符号和有符号整数之间进行强制类型转换是, 大多数C语言实现遵循的原则是底层的位模式不变, 在补码机器上, 对于一个w位的值, 这种行为是由函数$T2U_w$和$U2T_w$来描述的. C语言隐式的强制类型转换会出现许多程序员无法预计的结果, 常常导致程序错误.

第三章 程序的机器级表示

- 3.1 ATT 汇编语言
- 3.2 数据格式
- 3.3 访问信息
- 3.4 操作数与指示符
- 3.5 数据传送指令
  - 函数的栈帧
- 3.6 算术与逻辑操作
- 3.7 控制
  - 3.7.1 条件码
  - 3.7.2 访问条件码
  - 3.7.3 条件分支
  - 3.7.4 条件传送指令(cmov)
- 3.8 call指令
  - 3.8.1 调用前
  - 3.8.2 执行`call sum`指令
  - 3.8.3 `sum`函数主体执行
  - 3.8.4 补充: `leave`指令
- 3.9 循环指令和switch语句
  - 3.9.1 循环结构
  - 3.9.2 switch语句
- 3.10 过程
  - 3.10.1 控制转移
  - 3.10.2 参数转移
  - 3.10.3 递归过程
  - 3.10.4 Swap函数--引用传递
- 3.11 数组分配与访问
  - 3.11.1 一维数组访问
  - 3.11.2 二维数组访问
- 3.12 结构体和联合
  - 3.12.1 结构体(struct)
  - 3.12.2 联合(union)
- 3.13 理解黑客攻击和防御
  - 3.13.1 SQL注入攻击
  - 3.13.2 直接修改可执行文件
  - 3.13.3 改变程序返回地址
  - 3.13.4 缓冲区溢出攻击

1. 编译器是高级语言和汇编语言之间的翻译工具, 它将高级语言程序翻译成汇编语言程序
2. 高级语言编写的程序可以依靠不同的编译器在不同的机器上编译和执行，而汇编语言程序只能在特定的机器上执行
3. 汇编代码是机器代码的文本表示

3.1 ATT 汇编语言

1. ATT 汇编语言是一种基于 AT&T 语法的汇编语言
   ATT 汇编是GCC, objdump 和其他工具的默认输出格式
2. 本片假定代码运行在 IA-32(Intel Architecture 32-bit) 架构上

3.2 数据格式

图为C语言数据类型的IA-32架构下的表示

C语言声明	Intel数据类型	汇编代码后缀	大小/字节
char	字节	b	1
short	字	w	2
int	双字	l	4
long int	双字	l	4
long long int	—	—	4
char *	双字	l	4
float	单精度	s	4
double	双精度	l	8
long double	扩展精度	t	10/12

3.3 访问信息

一个 IA-32 中央处理器(CPU)包含一组**8个存储32位数据的寄存器**



以上寄存器中有名字的判断都是可以单独访问的.
通常情况下, 前六个寄存器被看成通用寄存器;

1. `%esp` 和 `%ebp` 是栈指针和基址寄存器, 分别用于栈的访问和函数调用的参数传递
2. `%eip` 是程序指针寄存器, 它指向**下一条将要执行的指令的地址**(指令实际上以机器码形式存在)

3.4 操作数与指示符

类型	格式	操作数值	名称
立即数	$Imm	Imm	立即数寻址
寄存器	E_a	R[E_a]	寄存器寻址
存储器	Imm	M[Imm]	绝对寻址
存储器	(E_a)	M[R[E_a]]	间接寻址
存储器	Imm(E_β)	M[Imm+R[E_β]]	(基址+偏移量)寻址
存储器	(E_β,E_i)	M[R[E_β]+R[E_i]]	变址寻址
存储器	Imm(E_β,E_i)	M[Imm+R[E_β]+R[E_i]]	变址寻址
存储器	(,E_i,s)	M[R[E_i]·s]	比例变址寻址
存储器	Imm(,E_i,s)	M[Imm+R[E_i]·s]	比例变址寻址
存储器	(E_β,E_i,s)	M[R[E_β]+R[E_i]·s]	比例变址寻址
寄存器	Imm(E_β,E_i,s)	M[Imm+R[E_β]+R[E_i]·s]	比例变址寻址

其中, **比例变址寻址** 是比较是最通用的格式, 经常用于高级语言中数组的访问, 有四个部分, 分别是 立即数偏移Imm, 基址寄存器E_β, 变址寄存器E_i, 比例因子s, 这里s必须是1, 2, 4, 8中的一个.

3.5 数据传送指令

指令	效果	描述
MOV S,D	D←S	传送
movb	传送字节	传送字节
movw	传送字	传送字
movl	传送双字	传送双字

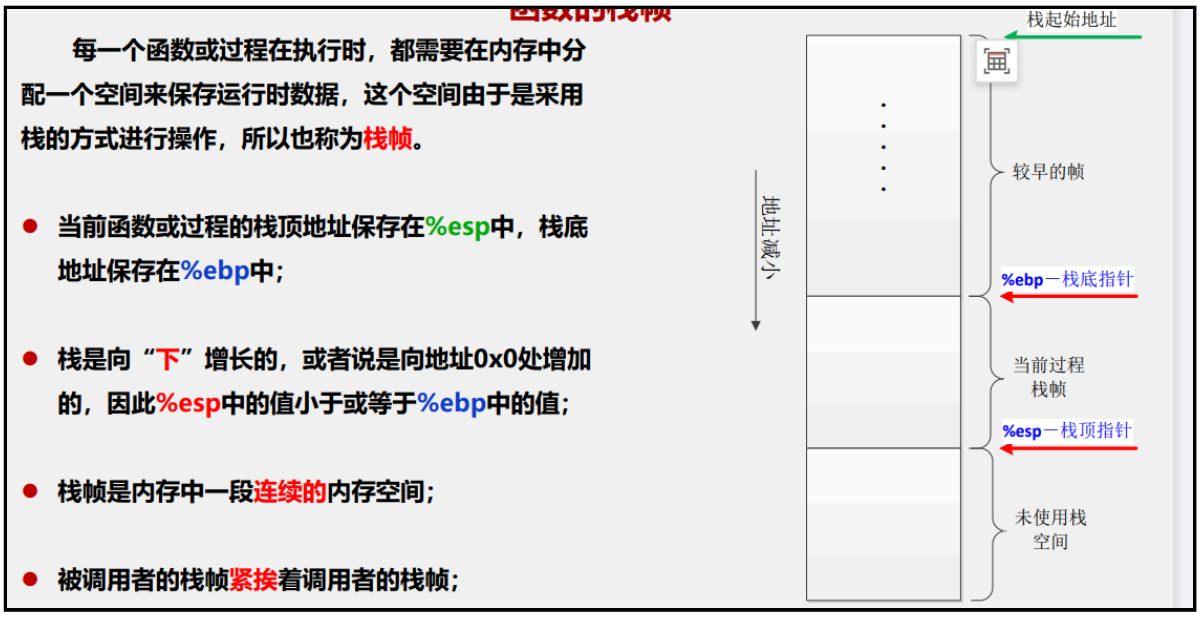
指令	效果	描述
MOVS S,D	D←符号扩展(S)	传送符号扩展的字节
movsbw	将做了符号扩展的字节传送到字	
movsbl	将做了符号扩展的字节传送到双字	
movswl	将做了符号扩展的字传送到双字	

指令	效果	描述
MOVZ S,D	D←零扩展(S)	传送零扩展的字节
movzbw	将做了零扩展的字节传送到字	
movzbl	将做了零扩展的字节传送到双字	
movzwl	将做了零扩展的字传送到双字	

1. IA32对MOV传送指令有一个限制, 即**传送指令的两个操作数同时是存储器**.
2. MOVS和MOVZ指令都是讲一个较小的源数据复制到一个较大的源数据位置, 区别在于**进行符号拓展和零拓展**

函数的栈帧

每个函数或过程在执行是, 都需要**在内存中分配一个空间**来保存运行时数据, 这个空间由于是采用栈的方式进行操作, 所以也称为**栈帧**



栈从**栈底到栈顶**对应内存空间的**从高到低**, 即栈底的地址是最大的, 栈顶的地址是最小的

指令	效果	描述
pushl S	`R[%esp]←R[%esp]-4` `M[R[%esp]]←S`	将双字压栈
popl D	`D←M[R[%esp]]` `R[%esp]←R[%esp]+4`	将双字出栈

pushl指令的功能就是把数据压入程序栈, popl指令的功能就是把数据从栈中弹出
他们后面都只跟一个操作数压入的数据源和弹出的数据目的寄存器.

等价

1. `pushl %eax` 的行为等价于:

```
subl $4, $esp    # 栈顶指针减4, 即栈增长
movl %eax, ($esp) # 源寄存器的值放入%esp指向的内存
```

2. `popl %ebx` 的行为等价于:

```
movl ($esp), %ebx # %esp指向的内存中取出值放到%ebx寄存器
addl $4, $esp    # 栈顶指针加4, 即栈缩小
```

3.6 算术与逻辑操作

指令	效果	描述
leal S,D	D←&S	加载有效地址

指令	效果	描述
INC D	D←D+1	加1
DEC D	D←D-1	减1
NEG D	D←-D	取负
NOT D	D←~D	取补(按位取反)
ADD S,D	D←D+S	加
SUB S,D	D←D-S	减
IMUL S,D	D←D*S	乘
XOR S,D	D←D^S	异或
OR S,D	D←D|S	或
AND S,D	D←D&S	与
SAL k,D	D←D<<k	左移(等同于SHL)
SHL k,D	D←D<<k	左移
SAR k,D	$D \leftarrow D >>_a k$	算术右移
SHR k,D	$D \leftarrow D >>_l k$	逻辑右移

以上的算术与逻辑操作指令, 除了leal("load effective address")指令, 其余其他指令都会有不同大小操作数的变体, 例如: ADD类有`addb, addw, addl`三个变体, 分别对应字节, 字, 双字.

`SAL`之类的移位指令的第一个操作数, 即一个立即数(也可以是寄存器`%cl`), 只能是一个字节, 如果该操作数为指出, 则默认值为1, 例如`sal %eax`的行为等价于`sal $1, %eax`

**C语言中的移位只有`<<`和`>>`两种, 具体是逻辑移位还是算术移位取决于被移位的变量的数据类型

1. 有符号数: 算术移位
2. 无符号数: 逻辑移位
3. **加载有效地址指令leal**
   例如: 假如`R[%edx]=x`那么`leal 7(%edx, %edx, 4), %eax`, 就是把`%eax`的值设为`5x+7`
   leal的目的操作数必须是一个寄存器**
4. **一元操作和二元操作**
   一元操作的一个操作数既是源也是目的, 可以是一个寄存器也可以是一个存储器位置
   二元操作的第二个操作数既是源也是目的, 可以是寄存器或存储器位置

5. **移位操作**
   先给出移位量k, 然后给出操作数D, k可以是一个立即数, 或者放在单字节寄存器元素 **%cl** 中. k在机器语言中用单个字节编码
6. **特殊的算术操作**

指令	效果	描述
imull S	`R[%edx]:R[%eax]←S*R[%eax]`	有符号全64位乘法
mull S	`R[%edx]:R[%eax]←S*R[%eax]`	无符号全64位乘法
cltd	`R[%edx]:R[%eax]←SignExtend(R[%eax])`	转为四字(32→64位)
idivl S	`R[%edx]←R[%edx]:R[%eax]modS` `R[%eax]←R[%edx]:R[%eax]÷S`	有符号除法
divl S	`R[%edx]←R[%edx]:R[%eax]modS` `R[%eax]←R[%edx]:R[%eax]÷S`	无符号除法

上面的`imull`指令有**单操作数**和**多操作数**的区别:

1. **单操作数**指令`imull S`是**有符号全64位乘法**, 它把S(寄存器/内存/立即数)和寄存器R[%eax]中的值相乘, 并把结果的64位结果存储在寄存器R[%edx]:R[%eax]中, 规定了必须是这些寄存器
2. **多操作数**指令`imull S,D`是**有符号全32位乘法**, 它把S(寄存器/内存/立即数)和D(寄存器)中的值相乘, 并把结果的32位结果(**溢出则截断**)存储在D中.

`imull`指令甚至有三操作数的形式:

```
imull Imm,reg/M,reg

# 运算: result = Imm * D
```

其余`mull`和`idivl`和`divl`指令, 都是只有**单操作数的形式**

3.7 控制

目前为止, 只学习了**直线代码**的行为, 对于C语言的某些语句结构, 比如条件语句, 循环语句和分支语句等, 机器代码提供**两种基本的低级机制**来实现有条件的行为:

1. 测试数据值
2. 根据测试结果改变控制流和测试流

3.7.1 条件码

除了八个整数寄存器, CPU还维护一组单个位的条件码(Condition Code)寄存器, **他们描述最近的算术和逻辑操作的属性**, 算术与逻辑指令以及后面的测试与比较指令都会设置条件码, 这些条件码可以用来检查操作结果的属性

- **CF**: 进位标志位, 最近的操作是最高位产生进位, 可以用来检查**无符号操作数的溢出**
- **ZF**: 零标志位, 最近的操作结果为0
- **SF**: 负号标志位, 最近的操作结果为负数
- **OF**: 溢出标志位, 最近的操作导致一个补码溢出(正溢出或负溢出)

IA-32架构下, 条件码存储在寄存器EFLAGS(%eflags)中:

- EFLAGS中的关键位:
  - CF(Carry Flag, 进位标志): **位0。**
  - ZF(Zero Flag, 零标志): **位6。**
  - SF(Sign Flag, 符号标志): **位7。**
  - OF(Overflow Flag, 溢出标志): **位11。**

对于操作$t = a + b$, 可用C语言表达式来表示条件码的设置:

```
CF = (unsigned) t < (unsigned) a     无符号溢出
ZF = (t == 0)                        零
SF = (t < 0)                         负数
OF = (a<0 == b<0) && (t<0> != a<0)   有符号溢出
```

比较与测试指令

还有两类特殊的指令, 分别是**比较指令**和**测试指令**, 他们都是**只设置条件码**, 而不改变任何数据(寄存器)., CMP指令行为与SUB指令相同, 只是不改变目的操作数的值.
TEST指令行为与AND指令相同, 只是不改变目的操作数的值.

指令	基于	描述
CMP S_2,S_1	S_1-S_2	比较
cmpb	Compare byte	比较字节
cmpw	Compare word	比较字
cmpl	Compare double word	比较双字
TEST S_2,S_1	S_1&S_2	测试
testb	Test byte	测试字节
testw	Test word	测试字
testl	Test double word	测试双字

例如`testl %eax, %eax`可以用来测试`%eax`是正数还是零还是负数, 通过设置`ZF`, `SF`标志位来实现.

3.7.2 访问条件码

条件码通常不会直接读取, 而是有三种常见使用方法:

1. 根据条件码某个组合, 将一个字节设置为0或1 -- **SET类指令**
2. 根据条件码实现有条件的跳转 -- **J类指令**
3. 有条件的传送数据

SET类指令

指令	同义名(synonym)	效果	设置条件
sete D	setz	D←ZF	相等/**零**
setne D	setnz	D←~ZF	不等/**非零**
sets D		D←SF	负数
setns D		D←~SF	非负数
setg D	setnle	D←~(SF^OF)& ~ZF	大于(有符号>)
setge D	setnl	D←~(SF^OF)	大于或等于(有符号>=)
setl D	setnge	D←SF^OF	小于(有符号<)
setle D	setng	D←(SF^OF)|ZF	小于或等于(有符号<=)
seta D	setnbe	D←~CF& ~ZF	超过(无符号>)
setae D	setnb	D←~CF	超过或等于(无符号>=)
setb D	setnae	D←CF	低于(无符号<)
setbe D	setna	D←CF|ZF	低于或等于(无符号<=)

J类指令

跳转(Jump)h指令会导致当前执行位置切换到一个全新的位置. 在汇编代码中, 通常用一个**标号(Label)**来指明这些跳转的目的地.

```
1  movl $0, %eax
2  jmp .L1              # 无条件跳转
3  movl (%eax), %edx
4 .L1
5  popl %eax
```

跳转的目的地有两种表示方式

1. **直接跳转**: 直接用标号作为跳转目标
2. **间接跳转**: `*`号后面跟一个操作数指示符
   例如: `jmp *%eax`, 寄存器的值直接作为跳转目标
   `jmp *(%eax)`, 存储器M[R[%eax]]的值作为跳转目标

`*`的含义
`*`号表示间接寻址, 他用于指示后面的操作数是一个间接地址, `jmp *%eax`表示跳转到%eax中存储的地址, `jmp *0x8048000`的含义是: **读取内存地址** `0x8048000`中存储的值，然后将程序跳转到该值所表示的地址。

label标号的含义
标号实际上就是地址的别名, 例如, 设`exit`是一个标号

- 那么`jmp exit`的含义就是: 将程序跳转到标号`exit`所表示的地址.
- `jmp *exit`的含义就是: 读取**内存地址** `exit`中存储的值, 然后将程序跳转到该值所表示的地址.

```
.section .data

    value:
        .int 0x87654321
.section .text

.global _start
 _start:
   nop
    movl $0x0804807e,%eax
    subl $1,%eax
loop:
    incl %ebx
    # jmp loop
    # jmp *%eax # %eax的值直接付给%eip
    # jmp *0x08049010 # 把该地址对应的内存赋值给%eip
    jmp exit  # exit是地址，直接跳转到该地方
    incl %eax
    # jmp *exit    # exit这个地址对应内存赋值给%eip
    movl 1,%eax  #

exit:
    movl $1, %eax
   movl $0, %ebx
   int $0x80
```

指令	同义名	跳转条件	描述
jmp Lable		1	直接跳转
jmp *Operand		1	间接跳转
je Lable	jz	ZF	相等/**零**
jne Lable	jnz	~ZF	不等/**非零**
js Lable		SF	负数
jns Lable		~SF	非负数
jg Lable	jnle	~(SF^OF)& ~ZF	大于(有符号>)
jge Lable	jnl	~(SF^OF)	大于或等于(有符号>=)
jl Lable	jnge	SF^OF	小于(有符号<)
jle Lable	jng	(SF^OF)|ZF	小于或等于(有符号<=)
ja Lable	jnbe	~CF& ~ZF	超过(无符号>)
jae Lable	jnb	~CF	超过或等于(无符号>=)
jb Lable	jnae	CF	低于(无符号<)
jbe Lable	jna	CF|ZF	超过或等于(无符号<=)

3.7.3 条件分支

机器指令一般是结合跳转指令来实现C语言中的条件分支.
下面是一个例子

```
int abs_diff(int x, int y)
{
      if(x < y)
        return y - x;
      else
        return x - y;
}
```

以上代码编译后生成的汇编代码如下: [`body` 部分]

```
...
  movl 0x8(%ebp), %edx  # get x
  movl 0xc(%ebp), %eax  # get y
  cmpl %eax, %edx       # compare x:y
  jge .L2               # if >=, goto "x-y"
  subl %edx, %eax       # y-x
  jmp .L3               # goto 'done'
.L2:
  subl %eax, %edx       # "x-y"
  movl %edx, %eax       # set result as return value
.L3
  ...
```

汇编代码通过比较两个操作数来设置条件码, 然后根据条件码体现出来的两个操作数的大小关系来决定跳转目标

C语言 goto语句

C语言有 `goto` 语句, 与汇编代码的风格很相似, 虽然C语言的 `goto` 语句比较low, 但是观察这样的语句能更好的理解汇编代码的执行流程.

```
int abs_diff(int x, int y){
    int result;
    if (x>=y)
        goto x_ge_y;
    result = y-x;
    goto done;
x_ge_y:
    result = x-y;
done:
    return result;
}
```

重要题型: 由汇编代码推断C代码

挖空的C代码如下, 需要根据汇编代码来推断出空缺的部分

```
int test(int x, int y){
    int val = ___;
    if (____){
        if (____)
      val = ____;
        else
            val = ____;
    } else if (____)
        val = ____;

    return val;
}
```

汇编代码:

```
  movl 0x8(%ebp), %eax
  movl 0xc(%ebp), %edx
  cmpl $-3, %eax
  jge .L2
  cmpl $edx, %eax
  jle .L3
  imull %edx, %eax
  jmp .L4
.L3:
  leal (%edx,%eax), %eax
  jmp .L4
.L2:
  cmpl $2, %eax
  jg .L5
  xorl %edx, %eax
  jmp .L4
.L5:
  subl %edx, %eax
.L4:
  ...
```

有两种解法, 下面只介绍第一种

```
int test(int x, int y){
    int val = x - y;
    if (x < -3){
        if (x > y)
            val = x * y;
        else
            val = x + y;
    } else if (x <= 2)
        val = x ^ y;

    return val;
}
```

3.7.4 条件传送指令(cmov)

现代处理器为了提高流水线(Pipelining)技术, 指令的处理分为几个步骤(取值, 译码, ...), 通过**重叠连续指令**的步骤来提高性能(指令并行执行).
但是**遇到条件分支**时, **J指令由于不知道要不要跳转**, 可能导致并行出错
条件传送指令这一策略能够**更好的匹配**现在处理器的流水线设计, 避免了预测错误的代价, 流水线效率更高



采用优化编译方式(`-O1`)可以得到采用**条件传送指令**代替**J跳转指令**的汇编代码.
条件数据传送方式将可能的所有结果都预先计算出来, 这样使得控制流不依赖于数据, 保持处理器流水线的满工作状态

编译命令

1. `gcc -S -O1 -g -m32 abs_diff.c -o abs_diff.s`
2. `gcc -c -O1 -g -m32 abs_diff.c -o abs_diff.o`

条件传送的代价

1. 条件测试结果的每一种情况**都要预先计算出来**, 无论后面是否需要
2. 一些特定的情况如果没有限制直接计算会引发**非法操作**

因此, 并不是所有情况都适合用条件传送指令

cmov类指令

指令	条件码组合	描述
cmove/cmovz	ZF	相等/零
cmovne/cmovnz	~ZF	不等/非零
cmovs	SF	负数
cmovns	~SF	非负数
cmovg/cmovnle	~(SF^OF)& ~ZF	大于
cmovge/cmovge	~(SF^OF)	大于或等于
cmovl/cmovnge	SF^OF	小于
cmovle/cmovng	(SF^OF)	ZF
cmova/cmovnbe	~CF& ~ZF	高于
cmovae/cmovnb	~CF	高于或相等

指令	条件码组合	描述
cmovb/cmovc	CF	低于
cmovbe/cmovna	CF	ZF

3.8 call指令

以一个汇编代码中`call sum`语句为例, 来讲清楚`call`函数过程中都发生了什么, 汇编代码标号为`sum`的片段如下

```
0x401040 <sum>:
 0x401040:   55          push   %ebp
 0x401041:   89 e5       mov    %esp,%ebp
 0x401043:   8b 45 0c    mov    0xc(%ebp),%eax
 0x401046:   03 45 08    add    0x8(%ebp),%eax
 0x401049:   5d          pop    %ebp
 0x40104a:   c3          ret
```

3.8.1 调用前

如果`sum`函数有参数, 则需要先将参数压入栈中(参数从右到左压入), 此时栈结构如下: **[假设向上为内存地址增长方向, 即栈减小方向]**

```
        [调用者的其他数据]
        [参数2]
栈顶 → [参数1]
```

3.8.2 执行`call sum`指令

1. **把返回地址压入栈中保存**
   相当于执行了一个`pushl %eip`指令, 把当前指令的地址压入栈中, 作为返回地址, 返回地址即为调用`call sum`后的下一条指令地址

```
        [调用者的其他数据]
        [参数2]
        [参数1]
栈顶 → [返回地址]
```

2. **跳转到被调用函数的入口**
   相当于执行了一个`jmp sum`指令, 把`sum`函数的地址付给`%eip`, 从而跳转到`sum`函数的入口

```
        [调用者的其他数据]
        [参数2]
        [参数1]
栈顶 → [返回地址]
```

3.8.3 `sum`函数主体执行

```
# setup阶段
push %ebp
mov %esp, %ebp

# body阶段
mov 0xc(%ebp), %eax
add 0x8(%ebp), %eax

# finish阶段
pop %ebp
ret
```

函数体执行一般分为三个阶段:

- `setup`: 函数入口处的代码, 用于设置函数的栈帧
- `body`: 函数的主要代码
- `finish`: 函数退出处的代码, 用于恢复调用者的栈帧

1. **setup阶段**
   `push %ebp` 把当前的`%ebp`值压入栈中, 作为函数的栈帧指针, 即保存调用者的`%ebp`值, 即上一帧的`%ebp`值

```
        [调用者的其他数据]
        [参数2]
        [参数1]
        [返回地址]
栈顶 → [旧的%ebp所指的地址]
```

`mov %esp, %ebp` 把当前的`%esp`值赋值给`%ebp`, 即让`%ebp`指向当前栈顶

```
                [调用者的其他数据]
                [参数2]
                [参数1]
                [返回地址]
栈底、栈顶 →  [旧的%ebp所指的地址]
```

2. **body阶段**
   函数体执行, 主要用于计算函数的返回值,
   分别执行`mov 0xc(%ebp), %eax`和`add 0x8(%ebp), %eax`指令, 即取参数1, 并把参数2加到参数1(%eax)上
3. **finish阶段**
   函数退出, 要把调用者的栈帧恢复, 并返回调用者
   `pop %ebp`, 弹出`[旧的%ebp所指的地址]`, 即恢复调用者的`%ebp`值

```
        [调用者的其他数据]
        [参数2]
        [参数1]
栈顶 →  [返回地址]
        [旧的%ebp所指的地址]
```

`ret`的效果等同于`pop %eip`，即把当前栈顶存的`[返回地址]`弹给`%eip`，即跳转到返回地址，即返回调用者

3.8.4 补充: `leave`指令

有时候`finish`阶段的代码可以简化为一个指令`leave`，比如:

```
sum:
    pushl   %ebp
    movl    %esp, %ebp
    # sub $0x4, %esp

    movl    0xc(%ebp), %eax
    addl    0x8(%ebp), %eax

    leave
    ret
```

`leave`指令是IA-32架构中的一个**复合**指令，他等价于执行:

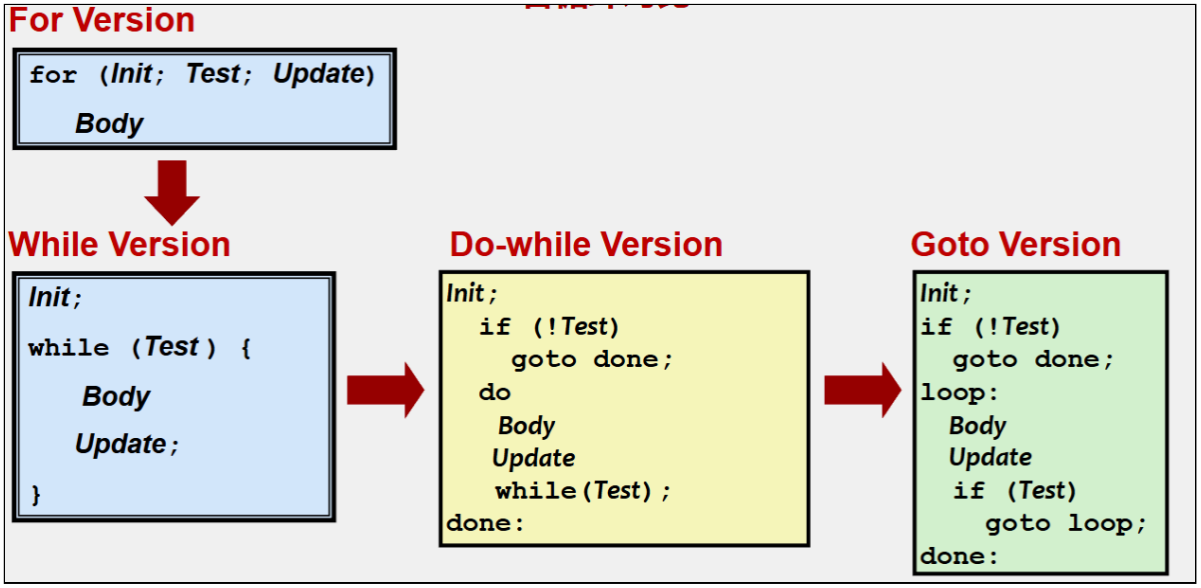
```
mov %ebp, %esp # 将栈顶指针恢复到栈帧底部
pop %ebp       # 恢复调用者的%ebp值
```

注意这里的`mov %ebp, %esp`指令，在这个例子中看似多余了，但是一般的函数`setup`阶段，在执行`mov %esp, %ebp`指令后，有时还需要执行`sub $0x4, %esp`指令，为函数的局部变量分配栈空间，这时候，`leave`中的`mov %ebp, %esp`指令就显得必要了

3.9 循环指令和switch语句

3.9.1 循环结构

C语言有`do-while`，`while`和`for`三种循环结构
但是大多数编译器只有一种形式的循环，基本都是按照`do-while`的形式产生出来的，其他的循环方式会先被编译器转换成`do-while`的形式，再变成汇编代码



3.9.2 switch语句

C语言提供`switch`语句来便捷的实现多分支的条件判断, 同时也会在合适的情况下应用**跳转表(Jump table)机制**来提高跳转的效率.

下面是一个很典型的`switch`语句的例子

```
long switch_eg1(long x, long y, long z){
  long w=1;    // 需要w初始化的case只有3和6,
  // 其他不需要w初始化, 此处汇编器也会聪明的进行优化
  switch(x){
    case 1:
      w = y * z;
      break;
    case 2:     // 无break, 该片段执行后会(fall through)继续执行case 3的片段
      w = y / z;
    case 3:
      w += z;   // 这些`+=`自操作符隐含了需要将w作为源操作数之一
      break;
    case 5:
    case 6:
      w -= z;
      break;
    default:
      w = 2;
  }
  return w;
}
```

Jump table跳转表

关于上面的switch语句, 如果不用跳转表编译, 由于**每个case的代码长度不一样, 所有分支代码段的其实地址之间不是等距的**, 此时处理器要根据索引值(本例子属于变量`x`)的不同去取不同的case指令会很难, 比较繁琐, 效率不高(可直接用gcc编译上述代码查看为什么低下, 每次jmp之前都要进行cmpl判断, 并且必须先判断前面的case分支)

可以用**跳转表**来优化, 跳转表的核心好处就是: **能够只用一条jmp指令**, 利用索引值(变量`x`)来直接跳转到不同的case代码段, 从而提高效率.

```
long switch_eg(long x, long y,
long z)
{
    long w = 1;
    switch(x) {
      . . .
    }
    return w;
}
```

Jump Table

```
jtab:  Targ0
       Targ1
       Targ2
         •
         •
         •
       Targn-1
```

Jump table

```
.section  .rodata
  .align 4
.L4:
  .long    .L8 # x = 0
  .long    .L3 # x = 1
  .long    .L5 # x = 2
  .long    .L9 # x = 3
  .long    .L8 # x = 4
  .long    .L7 # x = 5
  .long    .L7 # x = 6
```

```
setup:
switch_eg: 注意: 此处W没有进行初始化
pushl      %ebp              # Setup
movl       %esp,%ebp         # Setup
movl       8(%ebp),%eax      # %eax = x
cmpl       $6,%eax           # Compare x:6
ja         .L8               # If unsigned '>' goto default
jmp        *.L4(,%eax,4)     # Goto *JTab[x]
```

跳转表的每个表项存储的都是地址, 每个地址指向一个case代码段的入口, 这些地址表项在内存中是相邻的
以`jmp *.L4(,%eax,4)`为例, 从中可以看出, `.L4`就是跳转表的起始地址, `4`表示每个表项占`4`个字节, 该指令根据`x`的值算偏移量, 从而匹配到想跳转的表项, `*`表示间接寻址, 用表项所存的地址作为跳转目标, 直接跳转到需要执行的case片段

由上可知, 当**开关(case)的数量比较多, 并且值的范围跨度不大时(不是非常离散)**, 这时候就会使用跳转表来优化, 不然取值效率会很低

3.10 过程

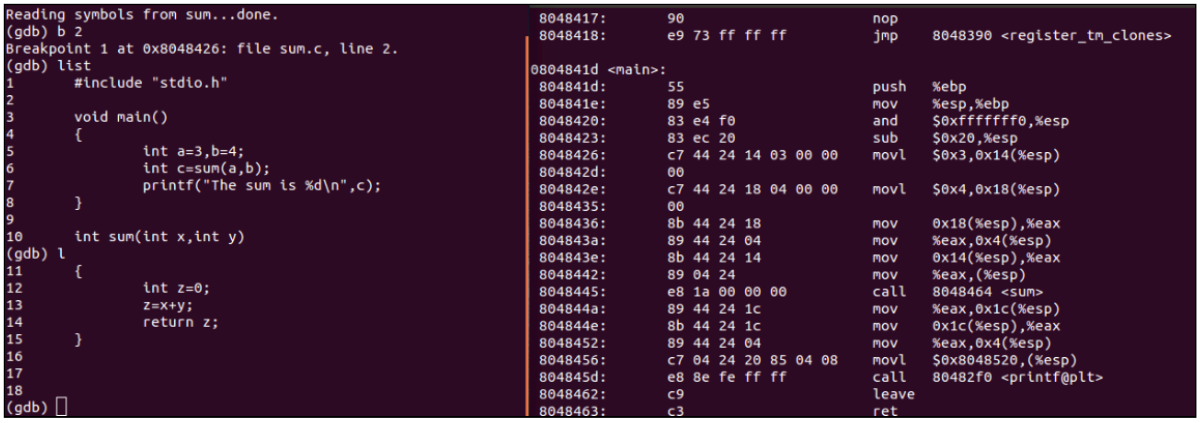
高级语言中, 一个主函数里可以有多个嵌套的子函数, 帮助主函数完成一些特定的功能, 这些函数也成为**过程**
由于处理器用来运算的资源(寄存器等)是有限的, 因此在过程被调用是, **调用者(主函数)**需要将控制和数据交给**被调用者(子函数)**, 同时还要在被调用者执行完毕后, 能够正确的返回到调用者继续执行, 这就需要**过程调用机制**来实现这些功能

3.10.1 控制转移

本节课以看3.8 call指令, 看完就能清楚的知道汇编中的**栈帧结构**和**转移控制**是怎样的

3.10.2 参数转移

接下来以一个sum函数的调用(**值传递**)的汇编代码为例, 来讲解**参数转移**的过程, 也涉及到关于`main`栈帧准备的一些细节




```
<sum>:
    push   %ebp
    mov    %esp, %ebp
    sub    $0x10, %esp
    mov    $0x0, -0x4(%ebp)
    movl   0xc(%ebp), %eax
    movl   0x8(%ebp), %edx
    addl   %edx, %eax
    mov    %eax, -0x4(%ebp)
    mov    %-0x4(%ebp), %eax
    leave
    ret
```

main栈帧准备的细节

main栈帧的建立过程中, 有一个`and $0xfffffff0, %esp`指令
该指令的效果是将`%esp`的值的低4位(二进制)强制清零, 这样使得栈顶的值是16的整数倍, **IA32体系的硬性规范+硬件/库函数要求**

sum函数的参数传递

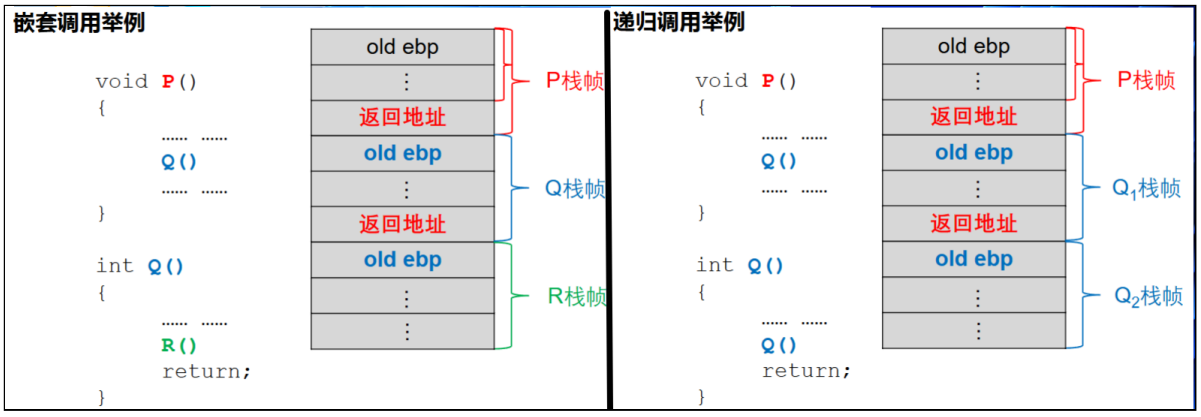
在`call sum`之前, 也就是sum(被调用者)的栈帧建立之前, 会有:

```
mov 0x18(%esp), %eax
mov %eax, 0x4(%esp)
mov 0x14(%esp), %eax
mov %eax, (%esp)
```

以上的汇编就是sum函数的参数传递, 他们会把main局部变量**拷贝一份**, 分别放到`(esp)`和`0x4(%esp)`位置(这里是值传递, 所以要进行拷贝, 存副本, 后续会以`swap`函数为例来说明关于引用传递, 能够在被调用者的帧中直接修改原变量的值)

后续sum栈帧建立后, 就会通过访问`0x8(%ebp)`和`0xc(%ebp)`来访问这两个参数, 这是因为sum建立时, 有两个push过程, 此时`%esp`相较原来减小了`0x8`, 后续又有`mov %esp, %ebp`指令, 此时`%ebp`来到了`%esp`位置, 因此此时的`0x8(%ebp)`和`0xc(%ebp)`就分别对应了之前的`(esp)`和`0x4(%esp)`位置

3.10.3 递归过程



寄存器的值保存惯例

寄存器是有限的, 在被调用者(Q)被调用之前, 如果某些寄存器存有调用者(P)稍后需要用到的内容, 但这些寄存器需要被Q使用, 则需要将这些内容保存起来.
在IA-32架构下, 约定了**调用者(P)**和**被调用者(Q)**的寄存器使用惯例:

1. `%eax`, `%ecx`, `%edx`属于**由调用者保存**的寄存器, 调用发生时, P负责将它们先压栈保存, 再产生Q的栈帧
2. `%ebx`, `%esi`, `%edi`属于**由被调用者保存**的寄存器, Q被调用后, 在这些寄存器被覆盖前, 要将它们在Q的栈帧里面保存, Q返回前进行恢复

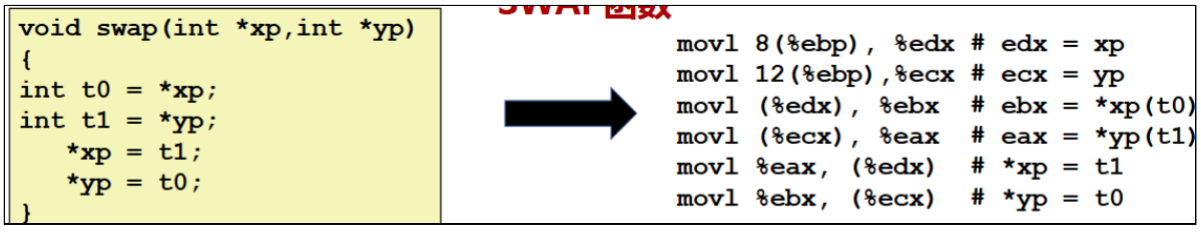
递归调用例子--阶乘

```
int rfact(int n){
  int result;
  if (n <= 1)
    result = 1;
  else
    result = n * rfact(n-1);
  return result;
}
```

以上代码编译后生成的汇编代码如下:

```
#Argument: n at %ebp+8
#Registers: n in %ebx, result in %eax
1   rfact:
2       pushl   %ebp                #save old %ebp
3       movl    %esp, %ebp          #set %ebp as frame pointer
4       pushl   %ebx                #save callee-save register %ebx(调用者保存寄存
器)
5       subl    $4, %esp            #allocate 4 bytes on stack
6       movl    8(%ebp), %ebx       #get n
7       movl    $1, %eax            #result = 1
8       cmpl    $1, %ebx            #compare n:1
9       jle     .L53                 #if <=, goto done
10      leal    -1(%ebx), %eax      #compute n-1
11      movl    %eax, (%esp)        #store at top of stack
12      call    rfact               #call rfact(n-1)
13      imull   %ebx, %eax          #compute result=return_value*n
14  .L53:                           #done:
15      addl    $4, %esp            #deallocate 4 bytes from stack
16      popl    %ebx                #restore %ebx (需要用到时进行恢复)
17      popl    %ebp                #restore %ebp
18      ret                         #Return result
```

3.10.4 Swap函数--引用传递



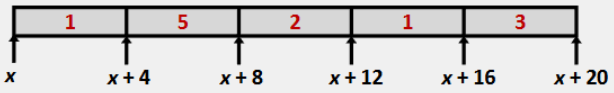
3.11 数组分配与访问

C语言中数组元素的访问可以通过对**指针(地址)**的运算来实现, 这样的较为简单的实现方式使得编译器对**C语言数组的机器代码翻译很容易**

数组的核心元素是**元素的个数**和**元素的数据类型**:

- **T A[L]:** type **T**, length **L**,
- 标识符 A可用于表示数组的首地址，也可以通过 Type T*转换成数组首地址

```
int val[5];
```



Reference	Type	Value
val[4]	int	3
val	int *	x
val+1	int *	x + 4
&val[2]	int *	x + 8
val[5]	int	??

3.11.1 一维数组访问

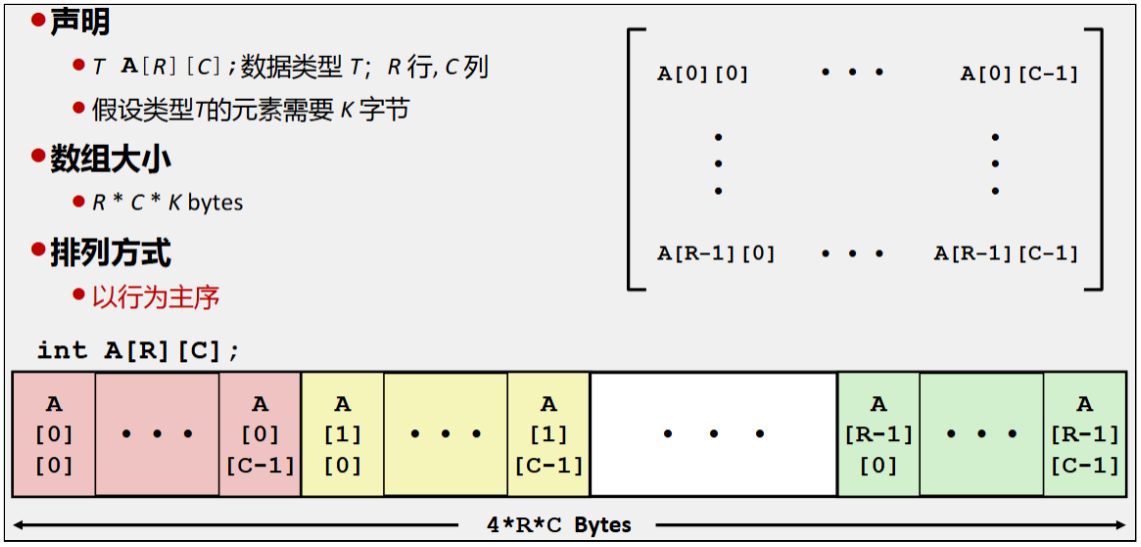
设K是每个元素占用的字节数, x_A是数组A的基地址, 那么访问数组元素`A[i]`的地址可以通过以下公式计算得到:

$$A[i] = x_A + i \times K$$

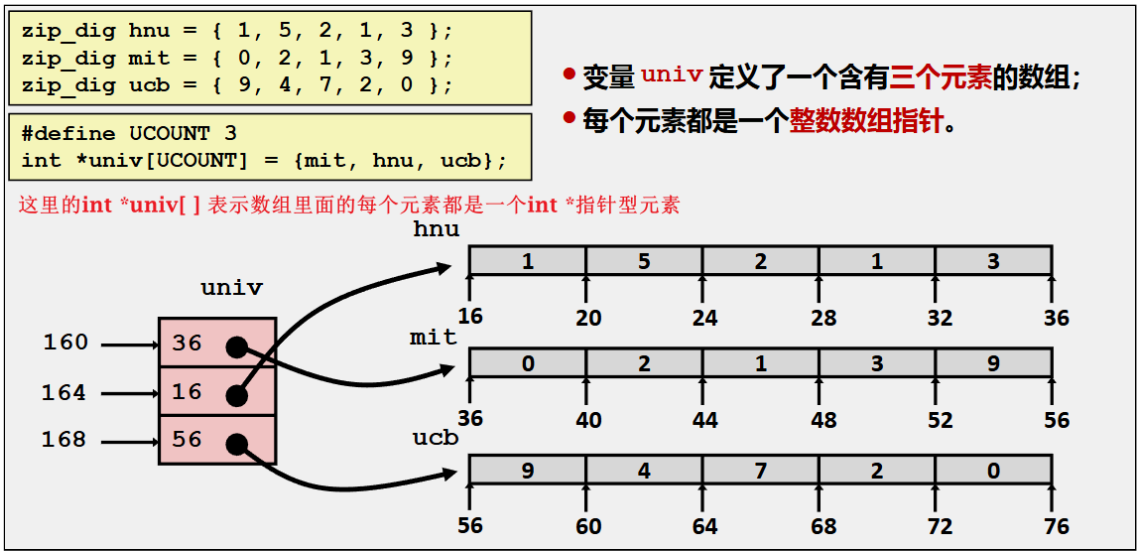
3.11.2 二维数组访问

二维数组可以理解有两种形式:

1. **以行为主序的二维数组**, 实质上是一维数组



2. **指针嵌套数组**



$$A[i][j] = x_A + (i \times C \times K) + j \times K$$
$$= x_A + (i \times C + j) \times K$$

访问A[i][j]的汇编如下:

```
# A at %ebp+8, i at %ebp+12, j at %ebp+16
movl 12(%ebp), %eax
leal (%eax, %eax, 2), %eax
movl 16(%ebp), %edx
sall $2, %edx                    # compute j*4
addl 8(%ebp), %edx               # compute xA+4j
movl (%edx, %eax, 4), %eax       # read from M[xA+12i+4j]
```

对于第二种, 汇编代码如下:

```
movl 8(%ebp), %eax           # index
movl univ(,%eax,4), %edx     # p = univ[index]  (指针必定是4byte, 32位系统)
movl 12(%ebp), %eax          # dig
movl (%edx, %eax, 4), %eax   # p[dig]
```

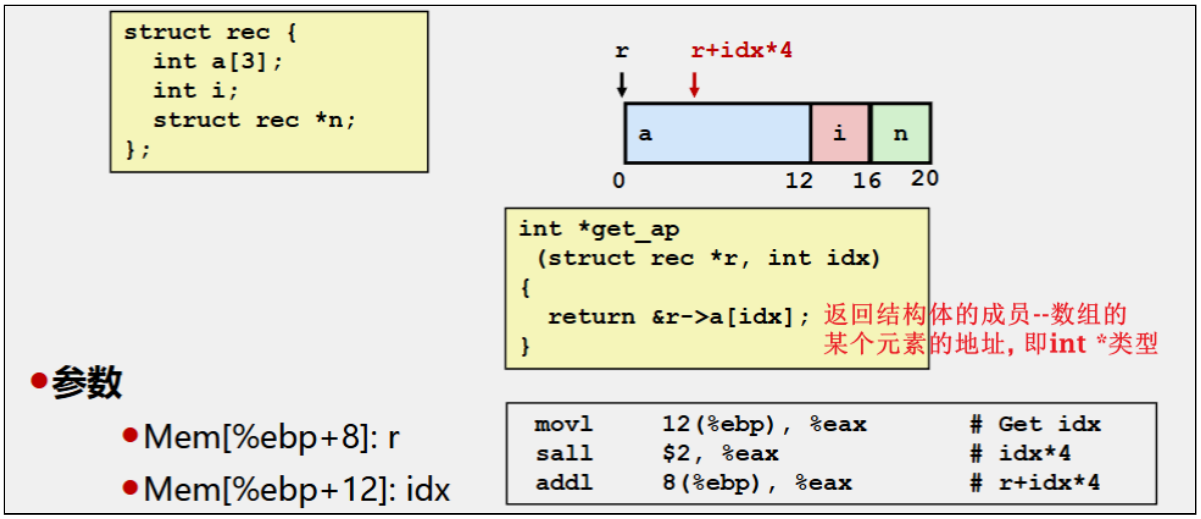
3.12 结构体和联合

C语言中的结构体(struct)和联合(union)都是**用户定义的数据类型**, 他们都可以包含多个成员, 但是他们的内存布局

3.12.1 结构体(struct)

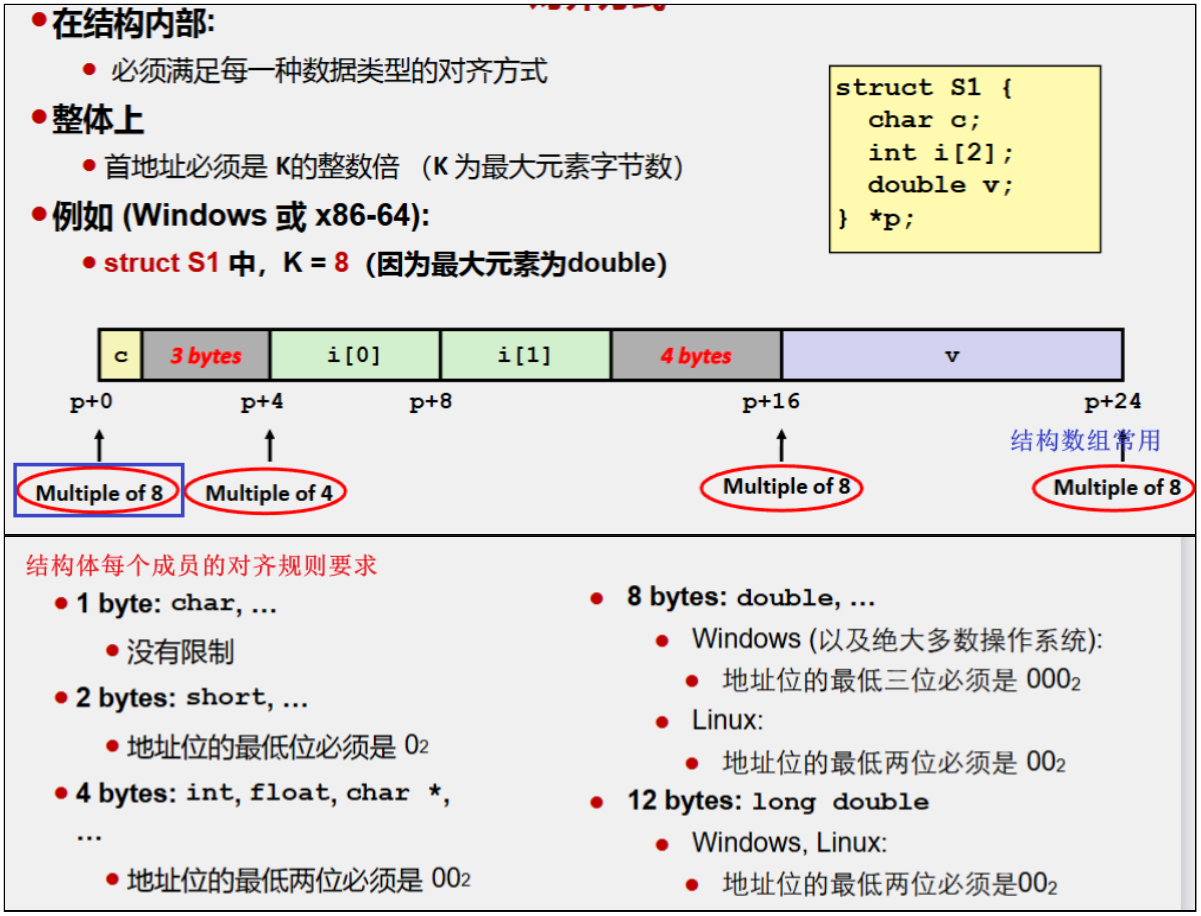
C语言中结构体内存连续分配, 可以包含不同数据类型的成员

下面是一个结构体成员访问的例子



结构体的内存对齐

结构体的内存对齐有整体上和局部上两方面规则



结构体每个成员的对齐规则要求

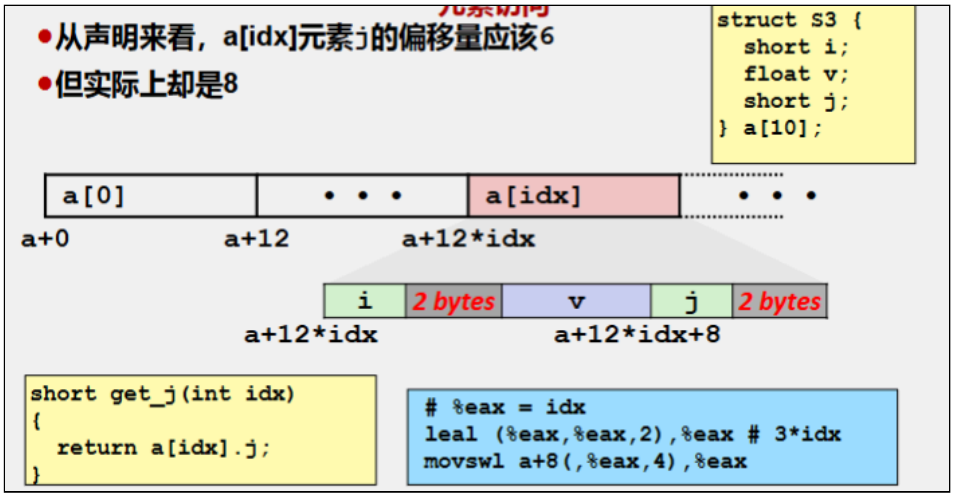
- 1 byte: char, ...
  - 没有限制
- 2 bytes: short, ...
  - 地址位的最低位必须是 0_2
- 4 bytes: int, float, char *, ...
  - 地址位的最低两位必须是 00_2
- 8 bytes: double, ...
  - Windows (以及绝大多数操作系统):
    - 地址位的最低三位必须是 000_2
  - Linux:
    - 地址位的最低两位必须是 00_2
- 12 bytes: long double
  - Windows, Linux:
    - 地址位的最低两位必须是00_2

结构对齐三大原则:

1. 结构的首地址必须是最大元素字节数的整数倍
2. 结构中的每个数据的地址必须是自身字节数的整数倍

3. **结构的总体长度必须是最大元素字节数的整数倍**

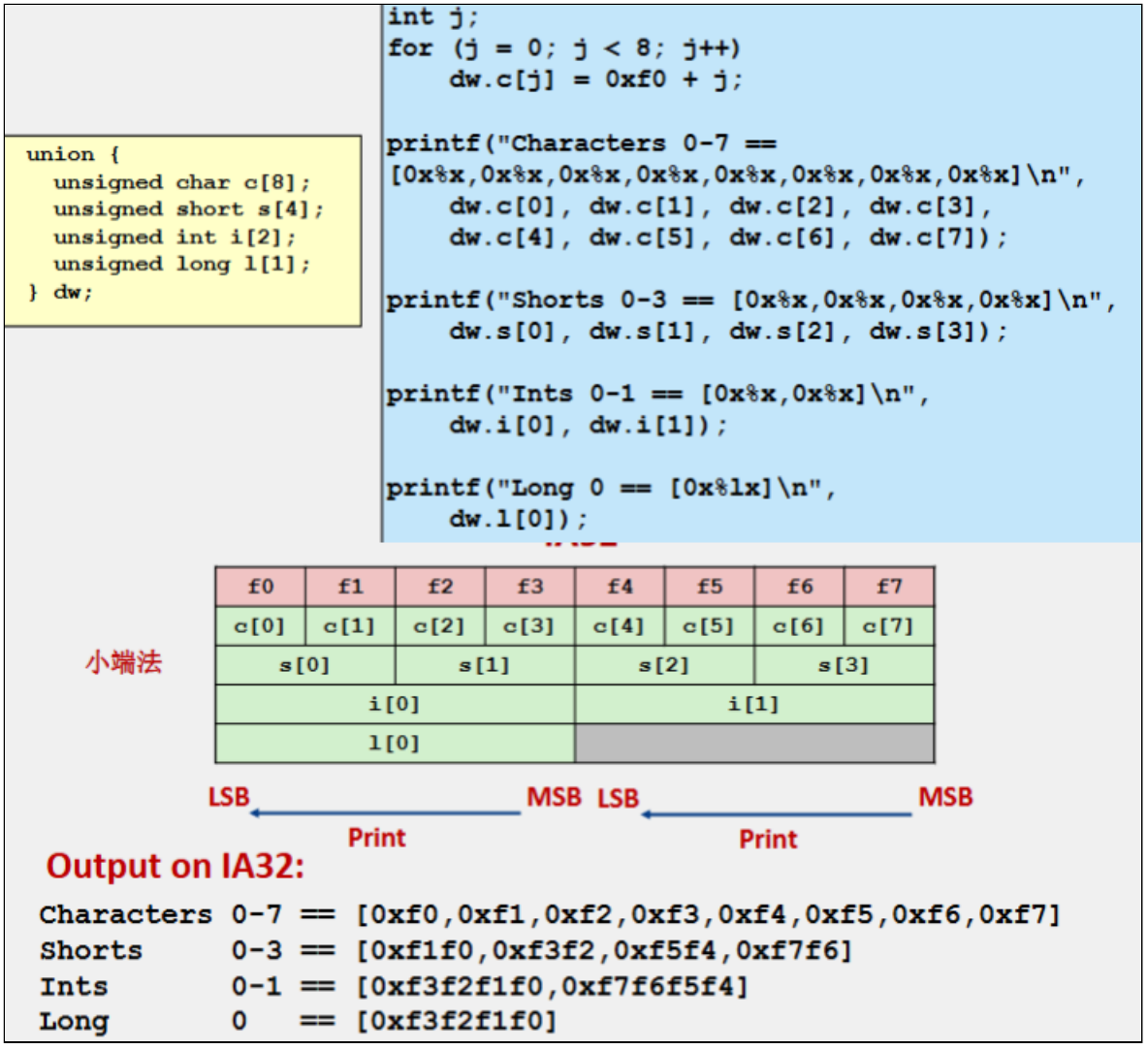
例子:



3.12.2 联合(union)

不同于结构体的对齐式内存分配, 联合的内存分配是**所有成员共享同一块内存**, 联合的大小等于其**最大成员的大小**, 联合的地址就是其最大成员的地址

以下是对联合体的访问的一个剖析



3.13 理解黑客攻击和防御

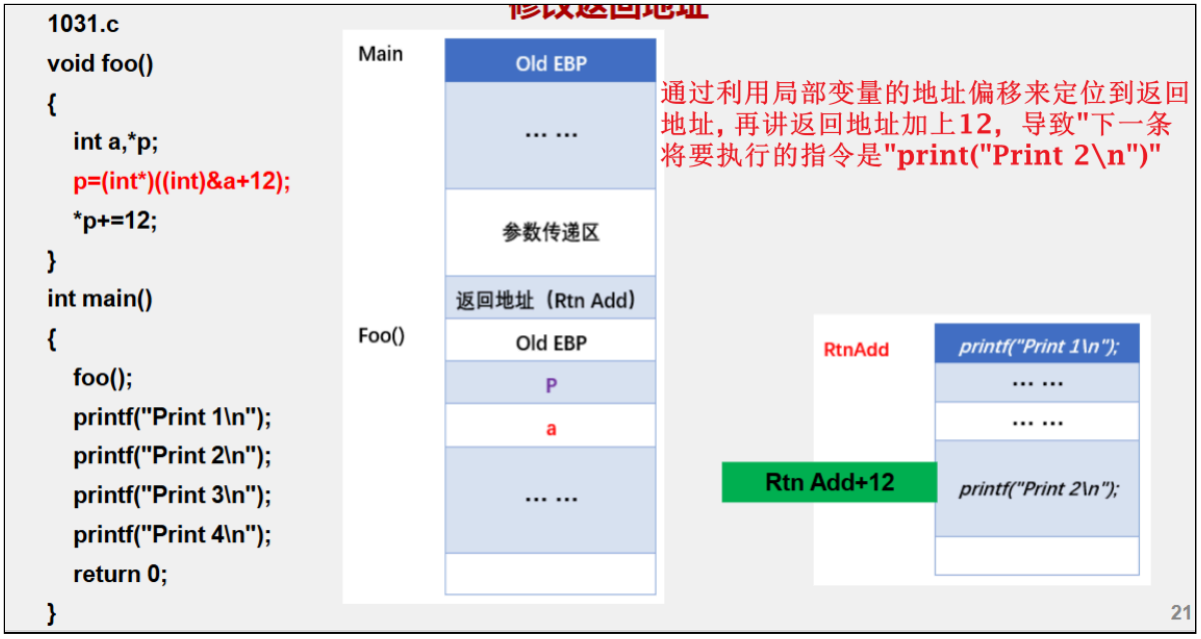
3.13.1 SQL注入攻击

后台接收前台的用户名和密码, 拼接成SQL语句并提交给数据库执行, 如果执行的结果不为空, 则说明用户名和密码正确
但某些用户可能通过构建**含有特殊字符**的用户名, 使得拼接的SQL语句含有攻击信息, 这就是**SQL注入攻击(SQL Injection)**
SQL注入攻击演示网站
除了绕过登录, SQL注入攻击漏洞的危害远不止于此, 例如, 在付出一定的时间成本后, 可以获得任一个用户的密码

3.13.2 直接修改可执行文件



3.13.3 改变程序返回地址



3.13.4 缓冲区溢出攻击

缓冲区溢出是一种非常普遍, 危险的漏洞, 在各种操作系统, 应用软件中广泛存在.
缓冲区溢出攻击是利用缓冲区溢出漏洞所进行的攻击行为

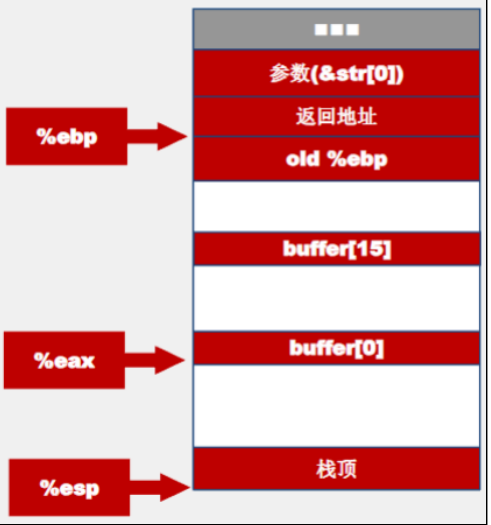
testoverflow.c

```c
#include <stdio.h>
#include "string.h"

void outputs(char *str)
{
        char buffer[16];
        strcpy(buffer,str);//str to buffer
        printf("%s \n",buffer);
}

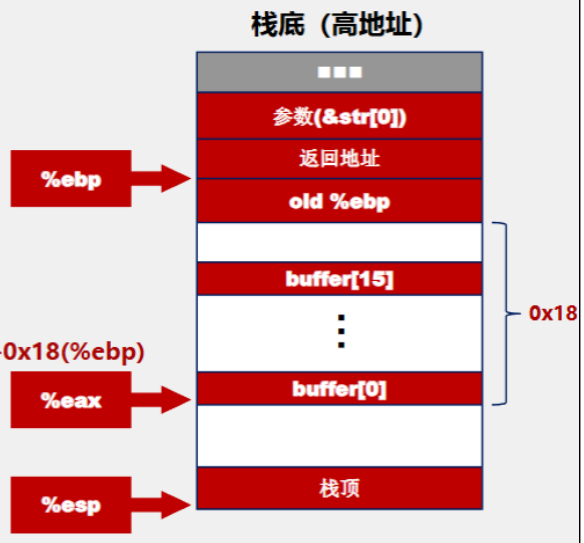
void hacker(void)
{
        printf("being hacked\n");
}

int main(int argc,char *argv[])
{
        outputs("I love HNU");
        /*outputs("12345671234567123456712345671234567\xaa\x84\x04\x08");*/
        return 0;
}
```



testoverflow.c

```
push   %ebp
mov    %esp,%ebp
sub    $0x28,%esp
mov    0x8(%ebp),%eax
mov    %eax,0x4(%esp)
lea    -0x18(%ebp),%eax
mov    %eax,(%esp)
call   8048340 <strcpy@plt>
lea    -0x18(%ebp),%eax
mov    %eax,0x4(%esp)
movl   $0x8048570,(%esp)
call   8048330 <printf@plt>
leave
ret
```



为防止GCC编译时打开缓冲区溢出保护使演示无法完成，
使用如下命令编译：

gcc –fno-stack-protector –z execstack –o a.out testoverflow.c

第四章 链接器

- 4.1 C语言的符号变量定义相关
  - 4.1.1 作用域 vs. 链接属性
  - 4.1.2 变量定义 vs. 声明
- 4.2 链接时的强弱符号
- 4.3 链接器的工作流程
  - 4.3.1 "符号解析"中"符号"
  - 4.3.2 为什么要链接器
- 4.4 ELF
  - 4.4.1 ELF 格式下的文件
  - 4.4.2 可重定位目标文件(`.o`)的格式和内容
- 4.5 链接第一步 -- 符号解析
  - 4.5.1 解析多重定义的符号
  - 4.5.2 链接静态库
- 4.6 链接第二步 -- 重定位
  - 4.6.1 重定位的符号引用过程实例
- 4.7 动态链接
  - 4.7.1 共享库 (Shared Library)
  - 4.7.2 动态链接过程

4.1 C语言的符号变量定义相关

4.1.1 作用域 vs. 链接属性

1. 作用域（Scope）

指**符号在源代码中的可见范围**：

- **块作用域**：函数内部、`{ }` 内定义的变量（局部变量）。
- **文件作用域（全局）**：在所有函数之外定义的变量/函数，从定义处到文件尾可见。

2. 链接属性（Linkage）

指**多个 `.c` 文件编译链接时，同名符号如何关联**：

- external linkage：符号可被其它目标文件访问。
- internal linkage：符号仅在当前目标文件内可见。
- none linkage：局部变量，无链接性。

声明写法	作用域	链接属性	典型场景
函数外 `int g_val;`	文件作用域	**external**	全局变量（可跨文件访问）

声明写法	作用域	链接属性	典型场景
函数外 `static int s_val;`	文件作用域	internal	文件私有全局变量
函数内 `int l_val;`	块作用域	none	普通局部变量
函数内 `static int sl_val;`	块作用域	none*（**但生命周期静态**）	静态局部变量
函数**声明** `void foo();`	—	**默认 external**	跨文件调用函数

注意:

1. 函数内的static变量声明

作用2：静态局部变量（生命周期全局，但作用域在块内）

```c
void counter() {
    static int c = 0;   // 只初始化一次，函数退出值不销毁
    c++;
    printf("%d\n", c);
}
```

链接属性仍然是 **none**，其它函数无法直接访问 `c`。

4.1.2 变量定义 vs. 声明

1. **定义**: 分配内存空间, 例如 `int g = 10` (强符号, 有初始值)
2. **声明 (extern)**: 不分配内存空间, 仅仅告诉编译器 "这个符号在别处定义"

```c
// file1.c
int g = 10;          // 定义（强符号）

// file2.c
extern int g;        // 仅仅声明，不分配空间
```

注意**函数声明**和**函数定义**的区别:

```c
//file1.c
void foo() {
    ...
}

//file2.c
void foo();  // 仅仅声明，不定义
```

4.2 链接时的强弱符号

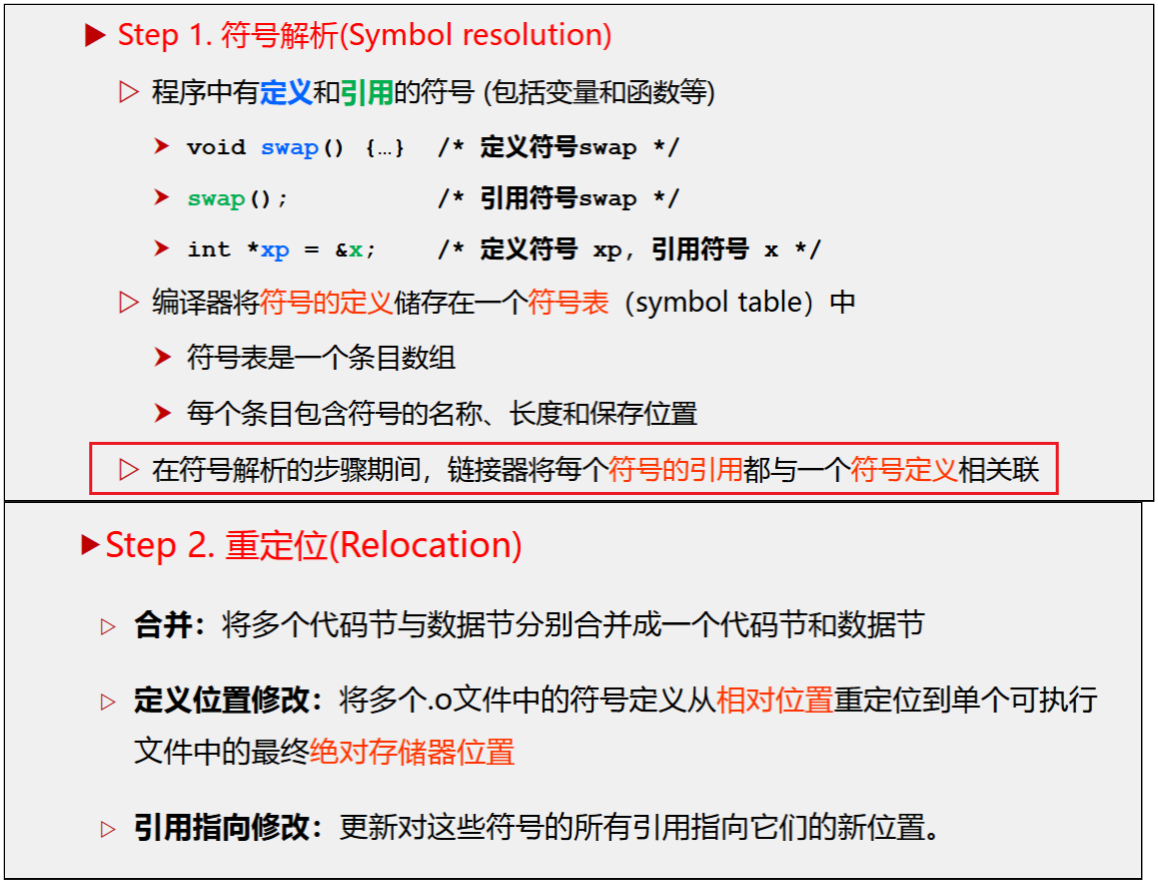
链接器在处理多个目标文件时, 根据符号强弱决定冲突规则:

符号类型	强弱	例子
已初始化的全局变量	强	`int x = 5;`
未初始化的全局变量	弱	`int x;`（常见实现为 common 块）
函数定义	强	`void foo() { ... }`
带 static 的全局变量	不参与链接（局部符号）	`static int y;`

冲突规则

1. 不允许两个强符号（重定义错误）
2. 一强一弱 → 选强符号
3. 两个弱符号 → 选占用空间大的（如果大小不同会警告）

4.3 链接器的工作流程



链接器对**可重定位目标文件**并不是"机械"的合并:

1. 一方面他讲不同目标文件中的各个部分按照代码, 数据等进行分类合并, 尤其对于静态链接, 链接器会将代码和数据都合并到可执行的文件中, 既需要确定各个代码和数据的最终存储位置;
2. 另一方面他需要分析目标文件中**符号定义**的完整性, 对于符号的地址信息进行就解析, 并将**符号的引用和相应符号定义关联起来.**

4.3.1 "符号解析"中"符号"

符号（Symbol）在链接的语境下，是指**在目标文件（.o）的符号表(symbol table)中记录的一个名字**，它代表一个全局可见的地址（代码地址或数据地址）。链接器只关心那些跨目标文件可见或需要重定位的符号。

那些C语言元素会成为"符号"

C语言元素	是否成为链接器符号	说明
全局变量（函数外定义）	✅ 是	无论是否 `static`，都会在符号表中（`static` 标记为 LOCAL）
函数定义	✅ 是	同上，`static` 函数为 LOCAL，非 `static` 为 GLOBAL
未初始化的全局变量	✅ 是	作为弱符号（COMMON 类型）
局部变量（函数内定义）	❌ 不是	存在栈或寄存器中，不进入符号表
静态局部变量	✅ 是	特殊：作用域在函数内，但存储静态，**有符号**（通常 LOCAL，名字修饰后）
`extern` **声明**	❌ 不是定义	仅仅是声明，不产生符号；但会产生**未定义符号引用**（见问题2）

关于"符号引用"和"符号定义"的区分

符号定义：分配了存储空间的符号（在某个目标文件中提供）。

符号引用：使用了该符号但没有定义的地方（通常需要链接器去找到定义）。

```c
// ref.c
extern int ext_var;   // 引用（声明），没有定义
void bar(void);       // 引用（函数声明），没有定义

int main() {
    ext_var = 5;
    bar();
    return 0;
}

// def.c
int ext_var = 10;     // 定义
void bar(void) { }    // 定义
```

```
// ref.o的符号表
Num:    Value  Size Type    Bind   Vis      Ndx Name
  5: 00000000     0 NOTYPE  GLOBAL DEFAULT  UND ext_var
  6: 00000000     0 NOTYPE  GLOBAL DEFAULT  UND bar
  7: 00000000    35 FUNC    GLOBAL DEFAULT    1 main

// def.o的符号表
  5: 00000000     4 OBJECT  GLOBAL DEFAULT    2 ext_var
  6: 00000000    10 FUNC    GLOBAL DEFAULT    1 bar
```

`UND` 表示undefined, 即表示该符号是引用

链接时，链接器会把 ref.o 中的 UND 条目与 def.o 中的 GLOBAL 定义关联起来，完成符号解析。

4.3.2 为什么要链接器

1. **模块化编程**: 允许将程序分成多个源文件，分别编译，最后链接成一个可执行文件
2. **代码重用**: 可以链接库文件（如标准库）来重用已有的功能
3. **分离编译**: 只需要重新编译修改过的源文件，而不需要重新编译整个程序

C语言元素	是否成为链接器符号	说明
全局变量（函数外定义）	✅ 是	无论是否 `static`，都会在符号表中（`static` 标记为 LOCAL）
函数定义	✅ 是	同上，`static` 函数为 LOCAL，非 `static` 为 GLOBAL
未初始化的全局变量	✅ 是	作为弱符号（COMMON 类型）
局部变量（函数内定义）	❌ 不是	存在栈或寄存器中，不进入符号表
静态局部变量	✅ 是	特殊：作用域在函数内，但存储静态，**有符号**（通常 LOCAL，名字修饰后）
`extern` **声明**	❌ 不是定义	仅仅是声明，不产生符号；但会产生**未定义符号引用**（见问题2）

4.4 ELF

ELF (Executable and Linkable Format) Linux / Unix 系统下目标文件的标准二进制格式，规范了如何组织代码、数据、符号表、重定位信息、动态链接信息等。
你看到的所有 `.o`、`.so`、不带后缀的可执行文件、甚至是内核模块 `.ko`，本质上都是 符合 ELF 规范的文件，只是 ELF 头中的类型字段不同。

相当于：**ELF 是一套“文件格式模版”**，而 `.o`、`.so`、可执行文件是这套模版的不同“实例化角色”。

4.4.1 ELF 格式下的文件

文件类型	常见后缀	ELF 类型常量	作用	链接过程角色
可重定位目标文件	`.o`（目标文件）	`ET_REL`	包含机器码和重定位信息，但地址尚未固定	**链接器的输入**：多个 `.o` 经过链接器合并为一个可执行或共享库
可执行文件	无后缀（如 `a.out`，或你指定的 `prog`）或 `.out`（传统）	`ET_EXEC`	包含已链接完成的机器码，地址已固定（或位置相关）	**链接器的输出**：可直接加载运行
共享目标文件	`.so`（shared object）	`ET_DYN`	可被多个程序动态加载，地址无关（`PIC`），支持动态链接	**动态链接器的输入**：运行时加载
核心转储文件	`core`	`ET_CORE`	进程崩溃时内存镜像（调试用）	不参与链接，仅调试

`readelf` **查看文件类型**

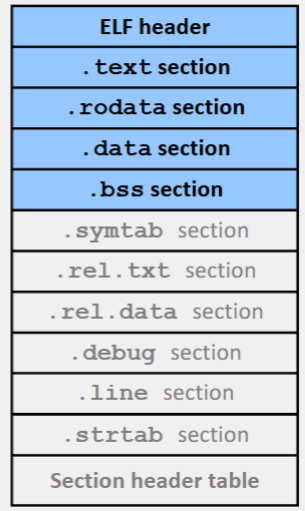
```
readelf -h add.o | grep Type
# Type: REL (Relocatable file)

readelf -h prog | grep Type
# Type: EXEC (Executable file)

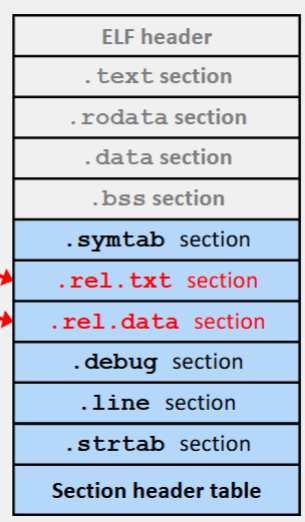
readelf -h libadd.so | grep Type
# Type: DYN (Shared object file)
```

4.4.2 可重定位目标文件(`.o`)的格式和内容

- ELF header
  - 字长、字节序（大端/小端）、文件类型 (.o, exec, .so)、机器类型（如 IA-32）等（readelf -h）
- .text section
  - 已编译的机器代码
- .rodata section
  - 只读数据，如printf格式串、switch跳转表等
- .data section
  - 已初始化的全局变量。注意**局部变量运行时保存在栈中**
- .bss section
  - 未初始化全局变量，仅是占位符
  - 不占用实际磁盘空间
  - 区分初始化和非初始化是为了空间效率
  - Block Started by Symbol (Better Save Space)



- .symtab section
  - 符号表（readelf -s）
  - 函数和全局/静态变量名，不含局部变量条目
- .rel.text section
  - **.text** 节的重定位信息（readelf -r）
  - 用于重新修改代码段的指令中的地址信息
- .rel.data section
  - **.data** 节的重定位信息
  - 用于对被模块引用或定义的全局变量进行重定位的信息
- .debug section (.line section)
  - 调试符号表 (gcc -g)
- . strtab section
  - 包含**.symtab**节中符号及**section**节名
- Section header table（节头表）
  - 每个**section**的名称、偏移和大小（readelf -S）



一个典型的 `ET_REL` 文件包含以下部分：

1. **ELF 头**（ELF Header）—— 文件类型、目标架构、入口点（此处为0）、节表位置等。
2. **节头表**（Section Header Table）—— 描述每个节的名称、类型、大小、偏移量。
3. **各个节的内容**：
   - `.text`：可执行机器码
   - `.data`：已初始化的全局/静态变量
   - `.bss`：未初始化的全局/静态变量（在 .o 中不占实际空间，仅记录大小）
   - `.rodata`：只读数据（如字符串常量）
   - `.symtab`：符号表
   - `.strtab`：字符串表（存储符号名、节名等）
   - `.rel.text` / `.rel.data`：重定位信息（记录哪些位置需要在链接时修改）
   - `.shstrtab`：节名字符串表
4. **程序头表**（Program Header Table）—— 对于 `ET_REL` 文件，通常没有程序头表（用于可执行文件和共享库）。

ELF Header (ELF头)

ELF 头位于文件开头，固定长度 **（32位为52字节，64位为64字节）。** 它描述了文件的基本属性和其他关键数据结构的位置与大小。

```
readelf -h <file>
```

主要字段:

字段	含义	示例（32位可执行文件）
`e_ident`	魔数、类别、字节序等	`7f 45 4c 46`
`e_type`	文件类型	`ET_EXEC` (可执行) 或 `ET_DYN` (PIE)
`e_machine`	目标体系结构	`EM_386` (Intel 80386)
`e_entry`	**入口点虚拟地址**	`0x8048310`
`e_phoff`	程序头表在文件中的偏移	`52` 字节
`e_phnum`	程序头表项数量	`9`
`e_shoff`	节头表在文件中的偏移	`12340` 字节
`e_shnum`	节头表项数量	`30`
`e_phentsize`	每个程序头项大小	`32` 字节
`e_shentsize`	每个节头项大小	`40` 字节

许多关于程序头表节头表的字段

ELF头的作用: 告诉加载器该文件是**何种类型, 入口在哪, 程序头表和节头表在何处**

Program Header Table (程序头表)

程序头表是一个结构体数组, 每个条目描述一个**段(Segment),** 段是**运行时**的视图: 它将多个具有相同权限(读/写/执行)的姐(section)合并在一起, 告知操作系统**如何加载文件的不同部分到内存**

```
readelf -l <file>
```

典型程序头类型（p_type）：

- `PT_LOAD`：可加载段，必须被映射到内存。
- `PT_DYNAMIC`：动态链接信息，由动态链接器处理。
- `PT_INTERP`：指定动态链接器的路径（如 `/ld-linux.so.2`）。
- `PT_NOTE`：辅助信息（如内核版本、ABI标签）。
- `PT_PHDR`：程序头表自身的位置。

每个条目包含的关键信息：

- `p_vaddr`：段应被加载到的**虚拟地址**。
- `p_filesz`：段在文件中的大小。
- `p_memsz`：段在内存中的大小（通常 ≥ `p_filesz`，未初始化的 `.bss` 部分扩展为 0）。
- `p_flags`：权限（R=可读，W=可写，E=可执行）。
- `p_offset`：段在文件中的偏移量。

一个简单可执行文件的程序头表如下:

```
Type           Offset   VirtAddr   PhysAddr   FileSiz  MemSiz   Flg Align
PHDR           0x000034 0x8048034  0x8048034  0x00100  0x00100  R   0x4
INTERP         0x000134 0x8048134  0x8048134  0x00013  0x00013  R   0x1
LOAD           0x000000 0x8048000  0x8048000  0x0069c  0x0069c  R E 0x1000
LOAD           0x000f0c 0x8049f0c  0x8049f0c  0x0010c  0x00124  RW  0x1000
DYNAMIC        0x000f18 0x8049f18  0x8049f18  0x000c8  0x000c8  RW  0x4
...
```

- 第一个 LOAD 段（R E）包含 .text、.rodata 等，只读+可执行。
- 第二个 LOAD 段（RW）包含 .data、.bss，可读写。
- INTERP 段告诉加载器使用动态链接器。

程序头表的作用： 告诉操作系统内核"如何将文件中的内容映射到进程地址空间"，是 **加载可执行文件的核心依据。**

Section Header Table (节头表)

节头表是一个结构体数组，每个条目描述一个节（Section）。**节是链接时的视图：它将不同功能的代码、数据、调试信息等分开，供链接器使用。** 对于可执行文件，**节头表不是运行时必需的（可被 strip 删除）**，但保留它有助于调试和查看元信息。

```
readelf -S <file>
```

使用 `readelf -S <file>` 查看。

常见节（Section）类型（sh_type）：

- `SHT_PROGBITS`：程序定义的内容（代码、数据）。
- `SHT_NOBITS`：不占文件空间的节（如 `.bss`）。
- `SHT_SYMTAB` / `SHT_STRTAB`：符号表 / 字符串表。
- `SHT_REL` / `SHT_RELA`：重定位节（可执行文件中通常已处理，不出现）。

每个条目包含的关键信息：

- `sh_name`：节名字符串在 `.shstrtab` 中的索引。
- `sh_type`：节类型。
- `sh_flags`：属性（如 `SHF_EXECINSTR`、`SHF_WRITE`、`SHF_ALLOC`）。
- `sh_addr`：节在内存中的**虚拟地址**（如果被加载）。
- `sh_offset`：节在文件中的偏移。
- `sh_size`：节的大小。
- `sh_link` / `sh_info`：用于链接的辅助索引。

典型的节:

节名	内容	是否加载到内存
`.text`	可执行机器指令	是（R E）
`.rodata`	只读数据（字符串常量等）	是（R）
`.data`	已初始化的全局/静态变量	是（RW）
`.bss`	未初始化的全局/静态变量（占位，不占文件空间）	是（RW，内存中清零）
`.plt`	过程链接表（动态链接用）	是（R E）
`.got`	全局偏移表	是（RW）
`.symtab`	符号表	否（调试信息）
`.strtab`	字符串表	否
`.shstrtab`	节名字符串表	否
`.debug_*`	调试信息	否

节头表的作用: 供调试器, 链接器, 反汇编工具理解文件结构, 不是运行时加载必需

4.5 链接第一步 -- 符号解析

符号解析就是要确定**每一个符号引用和相应的符号定义之间的关联**

在编译过程层中, 编译器只要找到相应的变量声明, 他就会继续编译过程, 他所产生的`.o`中的**符号表**只表达了两件事:

1. GLOBAL: 本目标文件能够提供给其他文件使用的符号

2. UND: 本目标文件需要其他文件提供的符号;

4.5.1 解析多重定义的符号

链接器将所有符号分为两类：

- **强符号（Strong Symbol）**：已初始化的全局变量、非 `static` 函数定义。
- **弱符号（Weak Symbol）**：未初始化的全局变量（GCC 中默认视为 `COMMON` 弱符号）、显式用 `__attribute__((weak))` 标记的符号。

多重定义时的处理规则（按优先级从高到低）：

1. **多个强符号** → 链接器报错：`multiple definition of 'symbol'`。
   例如：`file1.c` 中 `int x = 1;`，`file2.c` 中 `int x = 2;`。
   结果：链接失败。
2. **一个强符号 + 一个或多个弱符号** → 选择**强符号**，忽略弱符号。
   例如：`file1.c` 中 `int x = 100;`（强），`file2.c` 中 `int x;`（弱，COMMON）。
   结果：所有引用都绑定到 `x = 100`，弱符号的定义被丢弃。
   可能发出警告（若大小不一致时）。
3. **多个弱符号** → 选择**占用空间最大的**那个（若类型/大小不同则选最大的，否则任选一个）。
   例如：`file1.c` 中 `int x;`（4字节），`file2.c` 中 `double x;`（8字节）。
   结果：符号 `x` 最终大小为 8 字节（对应 `double`），所有引用都指向它。
   通常也会发出警告。

4.5.2 链接静态库

静态库的动机

在静态库出现之前，每个程序需要包含所有依赖的 `.o` 文件，导致：

1. **重复存储**：多个程序都用 `printf.o`、`sqrt.o` 等，每个可执行文件都复制一份，浪费磁盘和内存。
2. **维护困难**：库函数更新后，所有程序必须重新链接。
3. **编译繁琐**：命令行需要列出大量 `.o` 文件。

静态库解决：

- 将常用函数打包成一个文件（如 `libc.a`），链接器自动提取需要的 `.o` 模块。
- 链接后的可执行文件**独立无外部依赖**，但仍有冗余（每个程序包含库代码副本）。

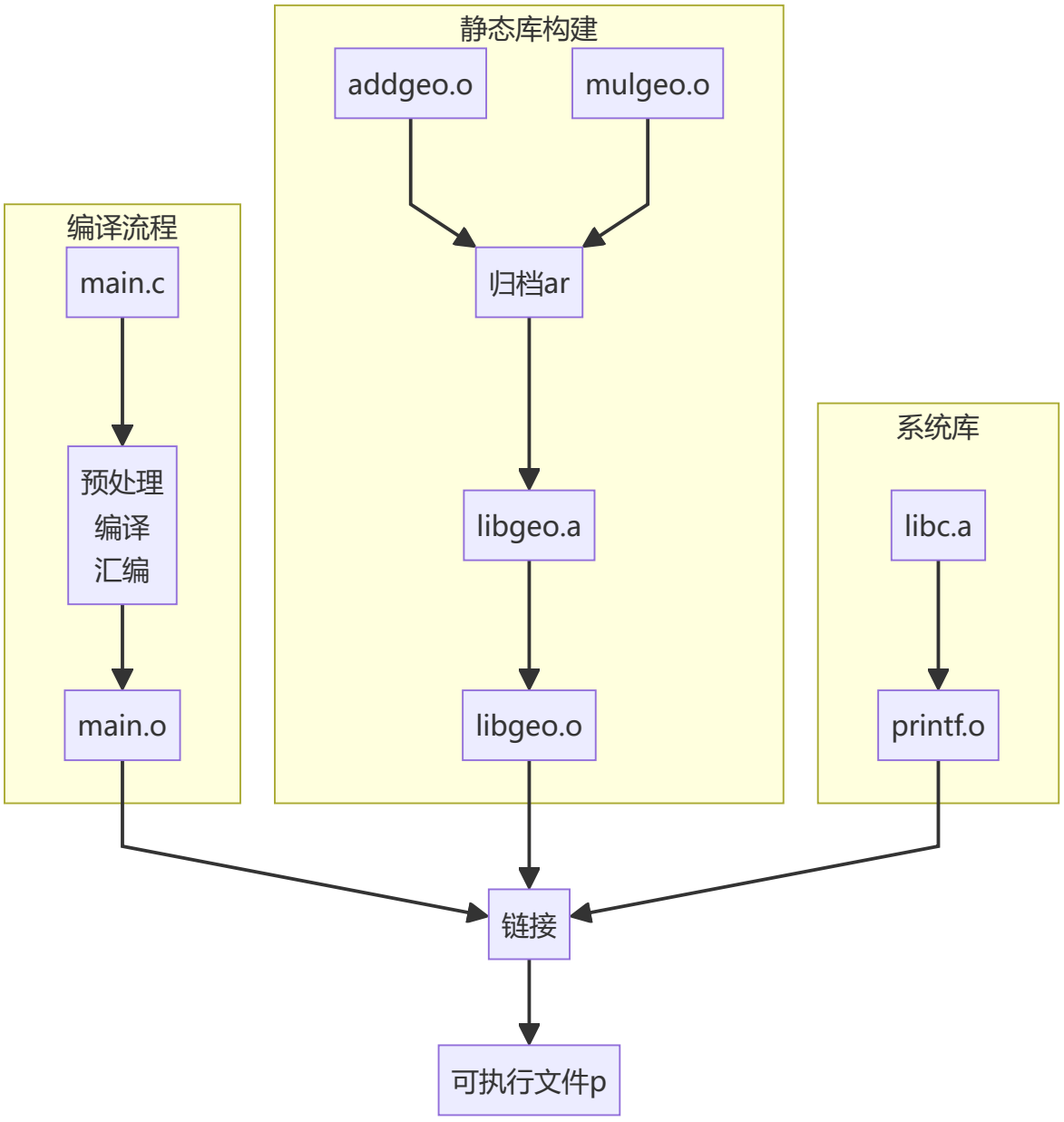
自己建立动态库和与动态库的链接

```
# 1. 编译 addgeo.c 和 mulgeo.c 生成目标文件
$gcc -c addgeo.c mulgeo.c

# 2. 将目标文件打包成静态库 libgeo.a
$ar rcs libgeo.a addgeo.o mulgeo.o

# 3. 编译 main.c 生成目标文件 main.o
$gcc -ol -c main.c

# 4. 静态链接，生成可执行文件 p
$gcc -static -o p main.o ./libgeo.a
```



4.6 链接第二步 -- 重定位

可重定位目标文件（`ET_REL`）中的代码和数据从地址 0 开始编译，各个 `.o` 文件相互独立。链接器需要：

1. 合并相同类型节
2. 符号引用的重定位, 链接器会依据**可重定位表**, 修改订**代码节和数据节**中的每个符号引用, 使他们均指向正确的运行时地址

重定位条目（Relocation Entry）

重定位条目记录了：**在 `.o` 文件的某个位置，需要如何修改以引用某个符号。**每个可重定位节（如 `.text`）可能有对应的 `.rel.text` 节, 这些就是可重定位表的表项条目

条目结构（`typedef struct`）：

- `r_offset`：需要修改的位置在节内的字节偏移（相对于节起始）。

- `r_info`：符号表索引 + 重定位类型（高24位符号索引，低8位类型）。
- `r_addend`（某些格式）：一个加数，用于计算最终地址。

使用 `readelf -r file.o` 查看。

重定位类型

在32位x86架构中，**重定位类型决定了如何计算最终地址：**

类型	描述	计算公式	常用场景
`R_386_32`	绝对地址重定位	`S + A`	引用全局变量的地址（`movl $sym, %eax` 或 `movl sym, %eax` 中的符号地址）
`R_386_PC32`	PC 相对地址重定位	`S + A - P`	调用函数（`call sym`）、跳转（`jmp sym`）
`R_386_GLOB_DAT`	用于 GOT（全局偏移表）	同 `R_386_32`	动态链接中访问全局变量
`R_386_JMP_SLOT`	用于 PLT（过程链接表）	同 `R_386_PC32`	动态链接中调用函数（延迟绑定）

> 其中：`S` = 符号的**实际运行时地址**，`A` = 重定位条目中的加数（通常就是原指令中的常数值），`P` = 被修改位置的实际运行时地址（即 `r_offset` 对应的运行时地址）。

4.6.1 重定位的符号引用过程实例

```c
// main.c
extern int sum(int a, int b);   // 定义在其他文件
int x = 100;                    // 强符号
int main() {
    return sum(x, 200);
}
```

编译成 32 位目标文件（默认未链接）：

```
gcc -m32 -c main.c -o main.o
```

(1) 查看重定位条目

```
readelf -r main.o
```

输出（简化）：

```
Relocation section '.rel.text' at offset 0x3a0 contains 2 entries:
 Offset     Info    Type            Sym.Value  Sym. Name
0000000c  00000802 R_386_PC32       00000000   sum
00000007  00000a01 R_386_32         00000000   x
```

- 偏移 `0x07` 处（`movl x, %eax` 中的 `x` 地址）**使用 `R_386_32` 类型，引用符号 `x`**。
- 偏移 `0x0c` 处（`call sum` 指令中的目标地址）使用 `R_386_PC32` 类型，引用符号 `sum`。

(2) 查看原始指令（反汇编，AT&T 格式）

```
objdump -d -M att main.o
```

输出：

```
00000000 <main>:
   0:   55                      push   %ebp
   1:   89 e5                   mov    %esp,%ebp
   3:   83 ec 04                sub    $0x4,%esp
   6:   a1 00 00 00 00          mov    0x0,%eax      # 偏移 7: 占位地址 0x0, 引用 x
   b:   83 c0 c8                add    $0xffffffc8,%eax
   e:   50                      push   %eax
   f:   e8 fc ff ff ff          call   10 <main+0x10> # 偏移 14: 占位 -4, 引用 sum
  14:   83 c4 04                add    $0x4,%esp
  17:   c9                      leave
  18:   c3                      ret
```

- `mov 0x0, %eax` 中操作数是 `0x0`，这是占位符，需要重定位。
- `call 10` 的相对偏移是 `0xfcffffff`（小端，即 -4），这也是占位符。

(3) 假设最终链接后地址分配

链接器决定：

- `.text` 段起始地址为 `0x8048000`，`main` 函数在偏移 `0x0` 处，所以 `main` 地址 = `0x8048000`。
- `x` 是全局变量，位于 `.data` 段起始地址 `0x804a000`，偏移 `0`，所以 `x` 地址 = `0x804a000`。
- `sum` 函数定义在 `sum.o` 中，假设其起始地址为 `0x80480a0`。

(4) 链接器执行重定位计算

对于 `R_386_32`（绝对地址）：

- 位置：`.text` 节偏移 `0x07`（即指令 `mov 0x0,%eax` 中的 `0x0` 字段）。
- 计算公式：`S + A`，**其中 `S = 0x804a000`（x 的地址），`A = 0`（原始占位值）。**
- 得到 `0x804a000`，以小端 4 字节写入。指令变为 `mov 0x804a000, %eax`。

对于 `R_386_PC32`（PC 相对地址）：

- 位置：`.text` 节偏移 `0x0f`（`call` 指令后的 4 字节）。
- 计算公式：`S + A - P`，其中：
  - `S = 0x80480a0`（sum 地址）
  - `A = -4`（原指令中的相对偏移，即 `0xfcffffff` 的有符号值）

- `P` = **被修改位置的实际运行地址 =** `main` **地址 + 0x0f =** `0x8048000 + 0x0f = 0x804800f`

- 计算：`0x80480a0 + (-4) - 0x804800f = 0x91`（十进制 145）。
- 写入 `0x91`（小端 `91 00 00 00`）。因此 `call` 指令变为 `call 0x80480a0`（实际上在反汇编中会显示为 `call sum`）。

(5) 最后结果

```
8048000:       55                      push   %ebp
8048001:       89 e5                   mov    %esp,%ebp
8048003:       83 ec 04                sub    $0x4,%esp
8048006:       a1 00 a0 04 08          mov    0x804a000,%eax    # 绝对地址重定位
804800b:       83 c0 c8                add    $0xffffffc8,%eax
804800e:       50                      push   %eax
804800f:       e8 8c 00 00 00          call   80480a0 <sum>     # PC相对重定位
8048014:       83 c4 04                add    $0x4,%esp
8048017:       c9                      leave
8048018:       c3                      ret
```

4.7 动态链接

动态链接是现代操作系统将程序与共享库（`.so` 文件）在运行时绑定在一起的技术。它与静态链接（**链接时合并所有代码**）相反，可以**节省磁盘和内存空间，并允许库独立升级。**

4.7.1 共享库 (Shared Library)

共享库是一种特殊的 ELF 目标文件（类型 ET_DYN），包含可被多个进程共享的代码和数据。

特点：

- **位置无关代码（PIC，Position-Independent Code）：** 使用相对寻址，可在内存任意地址加载。
- **文件后缀：** .so（如 libc.so.6）。
- **多个进程共享物理内存：** 内核将同一份共享库的代码段映射到不同进程的虚拟地址空间，但数据段各自独立（写时复制）。
- **独立升级：** 库更新后，无需重新链接程序，只要接口不变。

4.7.2 动态链接过程

动态链接发生在两个阶段：加载时（程序启动前）和运行时（首次调用库函数时，即延迟绑定）。

整个流程分为以下步骤：

1. 编译时（生成可执行文件）

- 编译器遇到外部函数（如 `printf`）时，不生成直接调用地址，而是生成对 **PLT（过程链接表）** 的调用。
- 链接器（静态链接阶段）在可执行文件中添加：
  - `.interp` 节：指定动态链接器的路径（如 `/ld-linux.so.2`）。
  - `.dynamic` 节：记录动态链接所需的信息（依赖库、符号表位置、重定位表等）。
  - `.got` 和 `.plt` 节：用于运行时符号解析。

2. 加载时（程序启动）

- 内核读取 ELF 可执行文件，发现 `.interp` 节，将控制权转交给 **动态链接器**（`ld-linux.so`）。
- 动态链接器执行以下操作：
  - 加载可执行文件本身（已静态链接的部分）。
  - 解析可执行文件的 `.dynamic` 段，获取依赖的共享库列表（如 `libadd.so`、`libc.so.6`）。
  - 查找每个共享库（按照 `LD_LIBRARY_PATH`、`/etc/ld.so.cache`、`/lib`、`/usr/lib` 等路径）。
  - 将共享库的代码段和数据段映射到进程的虚拟地址空间（通过 `mmap`）。
  - 执行共享库中必要的初始化代码（如 `.init` 和 `.ctors` 节）。
  - **处理重定位**：对于共享库中引用的全局变量和函数，动态链接器修改 GOT **（全局偏移表）** 中的条目，使其指向正确的地址。
- 最后，动态链接器将控制权交给可执行文件的入口点（`_start`）。

3. 运行时（延迟绑定）

为了加快启动速度，默认采用 **延迟绑定**（Lazy Binding）：函数地址在第一次被调用时才解析。

- 第一次调用 `printf` 时：
  - 程序通过 PLT 跳转：`call printf@plt`。
  - `printf@plt` 的第一条指令是 `jmp *printf@got`，初始时 `printf@got` 存放的是 PLT 中下一条指令的地址（`push` 指令）。
  - 执行 `push` 压入重定位索引，然后 `jmp` 到 PLT 的公共解析代码。
  - 公共代码调用动态链接器中的 `_dl_runtime_resolve` 函数，该函数根据索引找到实际函数地址，修改 `printf@got` 为目标地址。
  - 最后跳转到真正的 `printf` 函数执行。
- 后续调用：`jmp *printf@got` 直接跳到已解析的地址，无需再调用动态链接器。

第五章 内存系统

- 5.1 计算机的存储
  - 5.1.1 计算机的存储和寻址体系
  - 5.1.2 DRAM 芯片结构和访问
  - 5.1.3 存储器模块
  - 5.1.4 发展趋势对比
- 5.2 存储器层次结构
  - 5.2.1 高速缓存的命中/不命中
  - 5.2.2 高速缓存的结构
  - 5.2.3 存储层次
  - 5.2.4 高速缓存读取
  - 5.2.5 Cache写入
  - 5.2.6 Core i7 Cache结构剖析
  - 5.2.7 编写对Cache友好的程序
  - 5.2.8 存储器山

5.1 计算机的存储

5.1.1 计算机的存储和寻址体系

什么是"行"

“行”（Line）是**数据传输的最小单位**，也是缓存中**实际存储数据的基本容器。**它的标准全称是**缓存行（Cache Line）。**

一个缓存行一般由三部分组成:

1. **有效位(valid bit)**
2. **标记位(Tag)**
3. **数据块(Data)**

> 我们说“缓存行大小为 64 字节”，指的就是这个 **数据块（Data Block）** 的大小，而不是整个行的总大小（加上Tag和Valid位会略大一些）。

"页"和"行"的区别

对比维度	页（Page）	行（Cache Line）
所属层级	**虚拟内存/物理内存**（主存 DRAM 层）	**CPU 缓存（Cache）**（SRAM 层）
管理者	**操作系统（OS）+ MMU（内存管理单元）**	**CPU 硬件（Cache 控制器）**
核心作用	解决“进程虚拟地址”与“物理内存地址”的**映射**问题	解决 CPU 与物理内存之间**速度不匹配**的加速问题

对比维度	页（Page）	行（Cache Line）
典型大小	**4KB**（也有 2MB/1GB 的大页，但基础是 4KB）	**64 字节**（现代 Intel/ARM 标准）

他们在数据传输中的分工

1. 缺页异常时 -- 按"页"搬运

   当程序访问的内存不在物理内存中（缺页）时，操作系统会去**硬盘（磁盘/Swap分区）**，把包含该地址的**整整 4KB（一个页）**的数据，整体搬运到物理内存中。**注意**：硬盘太慢，按“行”（64B）搬会频繁死机，所以必须一次多搬点，按“页”来。

2. CPU 读数据时 -- 按"行"安装

   数据从硬盘搬到物理内存后，CPU 发出读指令。这时，CPU 的缓存控制器**不看“页”，只看“行”**。它把物理内存中这个页里的数据，**只提取出需要的那个 64 字节（一行）**，搬运到 CPU 的 L1/L2/L3 缓存中供 CPU 运算。

"几路组相连"的含义

> 缓存被分为若干组(set), 一个地址通过**Index固定进某一组**; "N 路"表示**每组有N个格子(way)**, 这个地址可以落在**其中任意一格，查找时N路并行比较Tag**
>
> 另外 **Offset**:
>
> - CO(Cache Offset): 缓存行内偏移量, 决定了从一个**缓存行**(64B, 64字节)中取**哪一个字节**
> - VPO/PPO: 页内偏移量 : 决定了从一个**"页"**(4KB, 4k字节)中取出**哪一个缓存行(64字节)**
>
> 在组相连缓存中，一个内存地址会被CPU拆分成三个字段（从高位到低位）：**Tag（标记）** | **Index（组索引）** | **Offset（块内偏移）**。



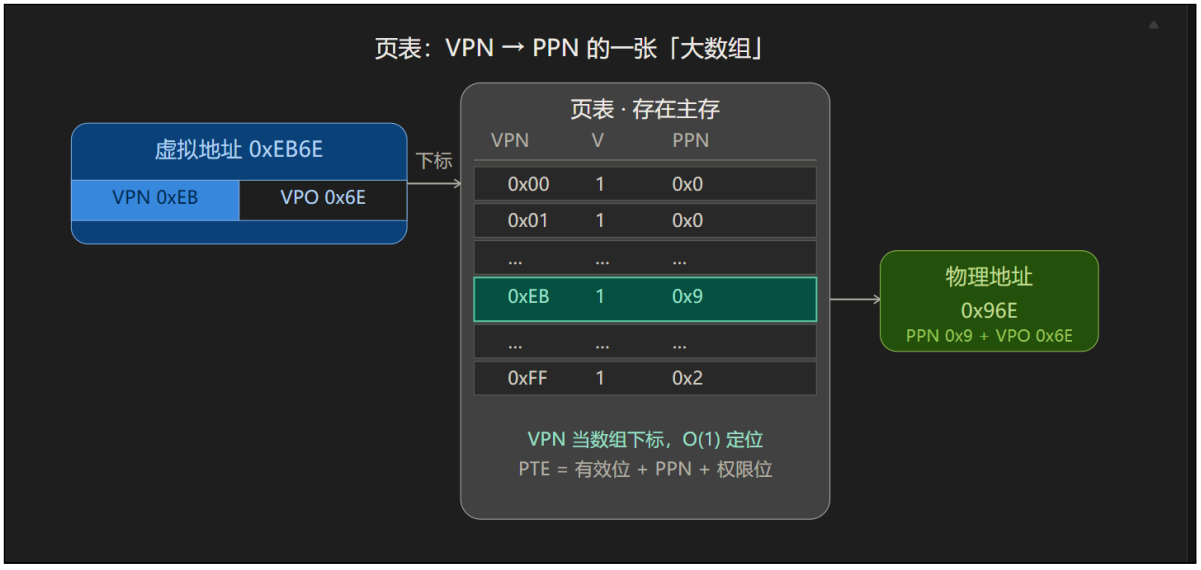
组数, Index位宽(位数), Offset位宽(位数) Tag位宽(位数) 三者之间的公式关系

- `组数 = 总行数 ÷ 路数`（如 24 行 ÷ 3 路 = 8 组 → CI 取 3 位, 这里的行在后面的示意图里表示一个长方形）
- `Index 位数 = log₂(组数)`
- `Offset 位数 = log₂(行大小)`
- `Tag 位数 = 地址总位宽 − Index − Offset`（TLB 里就是 `TLBT = VPN − TLBI`）

核心模块区分

	页表	TLB	Cache
存什么	**VPN→PPN 映射**	**最近用过的 VPN→PPN**	**数据块本体**
在哪	内存	**CPU 内部**	CPU 旁边(高速缓存)
谁索引	VPN（下标）	TLBI 索引 + TLBT 校验	CI（PA 中位）
怎么命中	直接数组访问	TLBT 组内比较	CT 组内比较
未命中	缺页 → 磁盘调页	查页表	去内存取块
规模	很大（256 项）	很小（16 项）	中（24 行）

页表结构



TLB



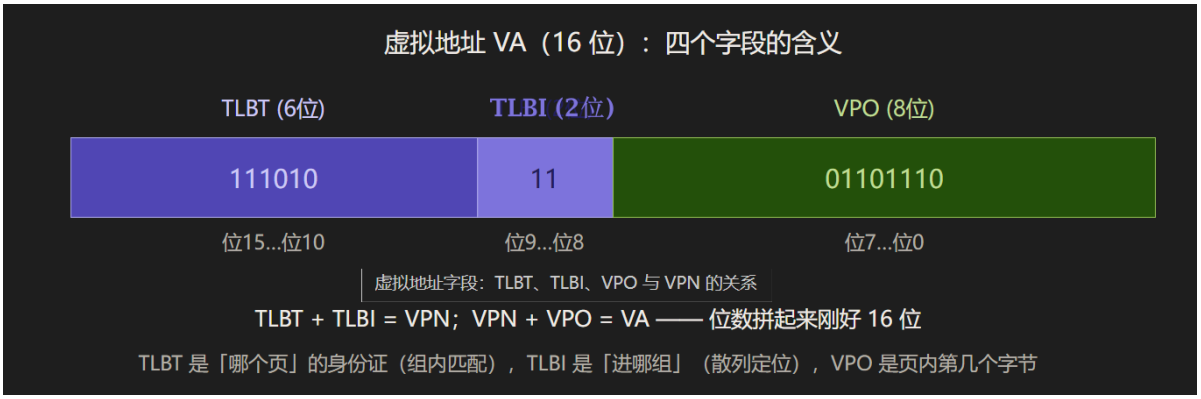
Cache



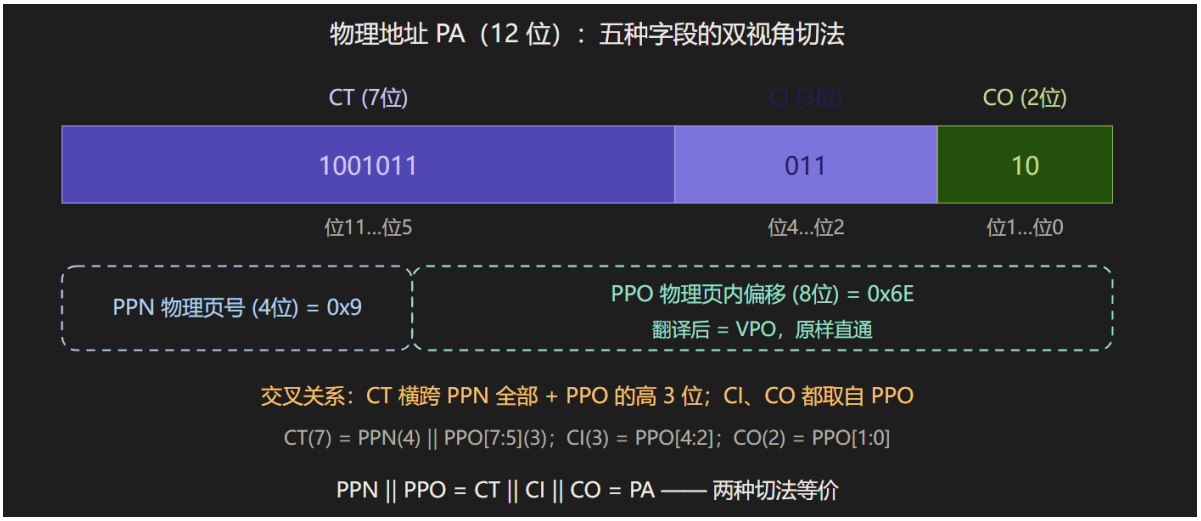
虚拟地址和物理地址拆分

> **总原则:**
>
> 1. 两种地址都有 N 和 O的区分: VPN和VPO, PPN和PPO, 其中PPO和VPO是完全相同的都不参与地址翻译, 都是用作块内偏移
> 2. 页表和TLB是和虚拟地址关联的, 也就是VA的VPN, 和物理地址无关
> 3. Cache 是和物理地址关联的, 也就是PA的PPN, 和虚拟地址无关

虚拟地址VA



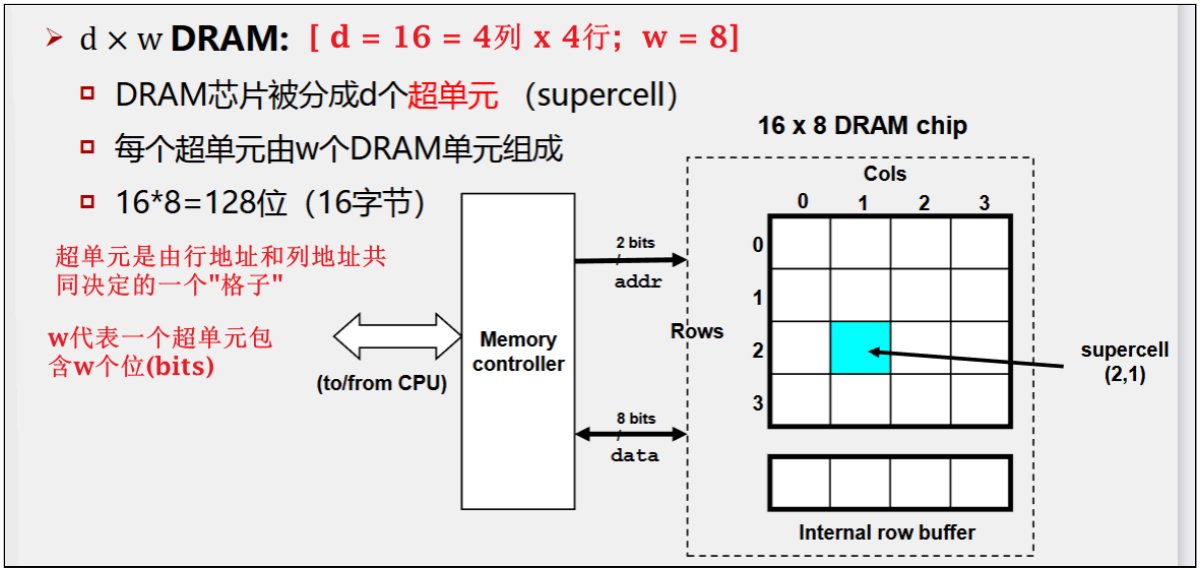
物理地址PA



5.1.2 DRAM 芯片结构和访问

DRAM芯片相比SRAM, **访问时间更长, 成本更低**, 存储密度更高, 因此被广泛使用在**主内存**中.

DRAM结构

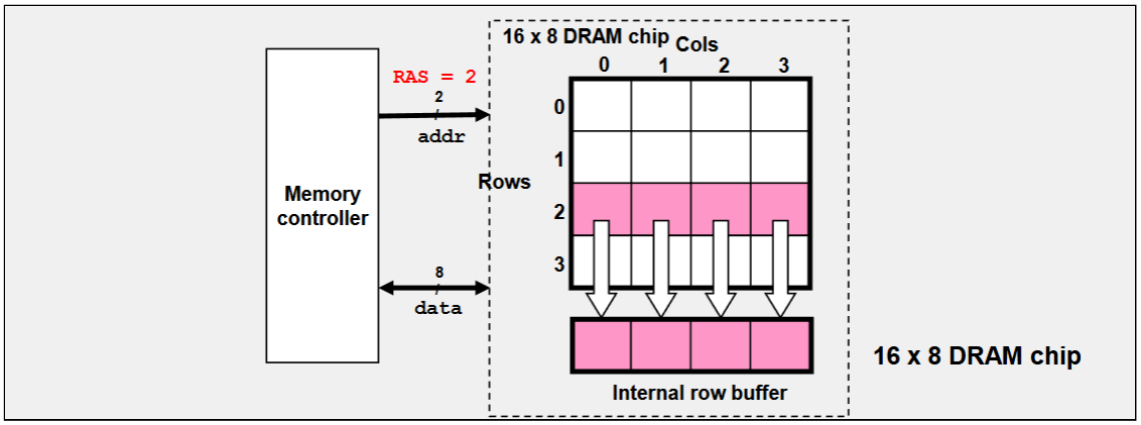


如图, 一个 `8M x 8` 的RAM芯片, 共有8M个超单元, 每个超单元存储8位数据, 因此总容量是64Mb=8MB.

访问DRAM

访问DRAM需要**行地址**和**列地址**, 访问时先提供行地址, 选中对应的行, 然后提供列地址, 选中对应的列, 最后就可以访问到对应的内存单元了.

1. **行激活**: 内存控制器(memory controller)向DRAM芯片提供行地址, DRAM将第i行所有的超单元拷贝到**行缓冲区(row buffer)**中
2. **列选择**: 控制器发送列地址, 从行缓冲区中精准选中对应的超单元, 进行读写操作
3. **输出**: 这个超单元的w位数据通过n引脚发送到CPU.



这样设计的好处是: **极大的减少了地址线的数量**, 是DRAM能够实现高容量的核心原因

二维阵列而不是线性数组

- 降低芯片上地址引脚的数量，简化硬件设计
- 采用线性地址时，地址编码是 0～15，需要 4 比特表示，所以需要 4 个引脚

```
0000  0001  0010  0011  0100  0101  0110  0111
1000  1001  1010  1011  1100  1101  1110  1111
```

- 采用二维方式时，每个单元都有行和列两个地址，行和列是分两次寻址的。例子中的行和列地址都是 0～3，需要 2 比特表示，也就是 2 个引脚（引脚复用）

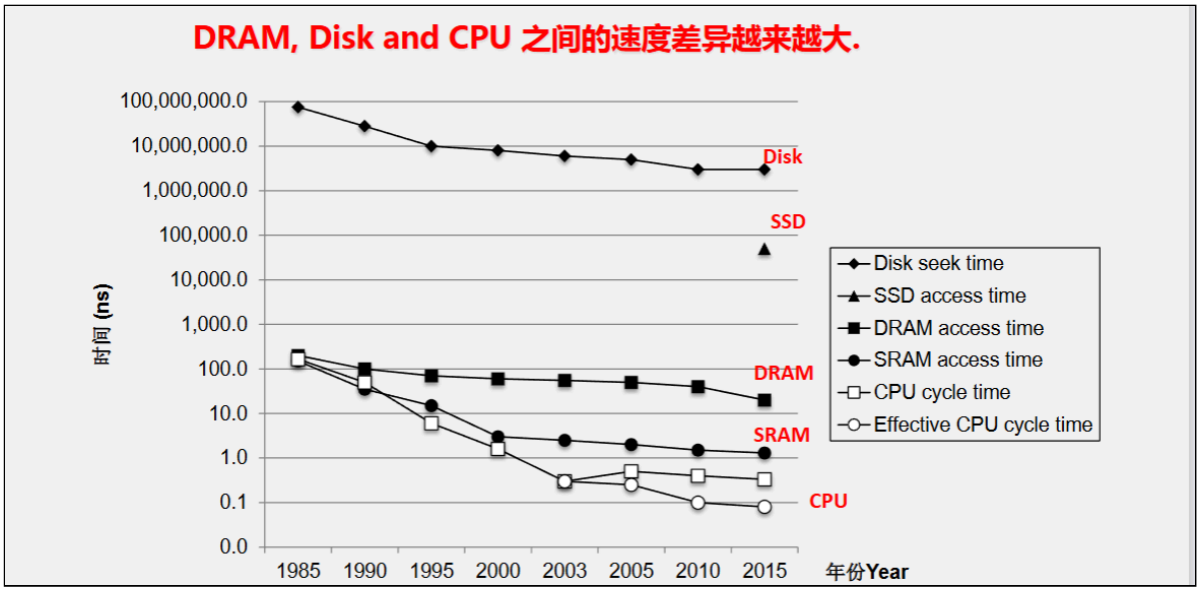
```
[00,00]   [00,01]   [00,10]   [00,11]
[01,00]   [01,01]   [01,10]   [01,11]
[10,00]   [10,01]   [10,10]   [10,11]
[11,00]   [11,01]   [11,10]   [11,11]
```

- 缺点是必须分两步发送地址，这增加了访问时间。

5.1.3 存储器模块

存储器模块(memory module)是由多个DRAM芯片组成的一个物理单元, 通过总线与CPU连接.
主存（通常也直接称为“内存”或“RAM”） 是一个逻辑概念，它是计算机硬件架构中直接受 CPU 寻址并与之交换数据的核心存储空间。

5.1.4 发展趋势对比



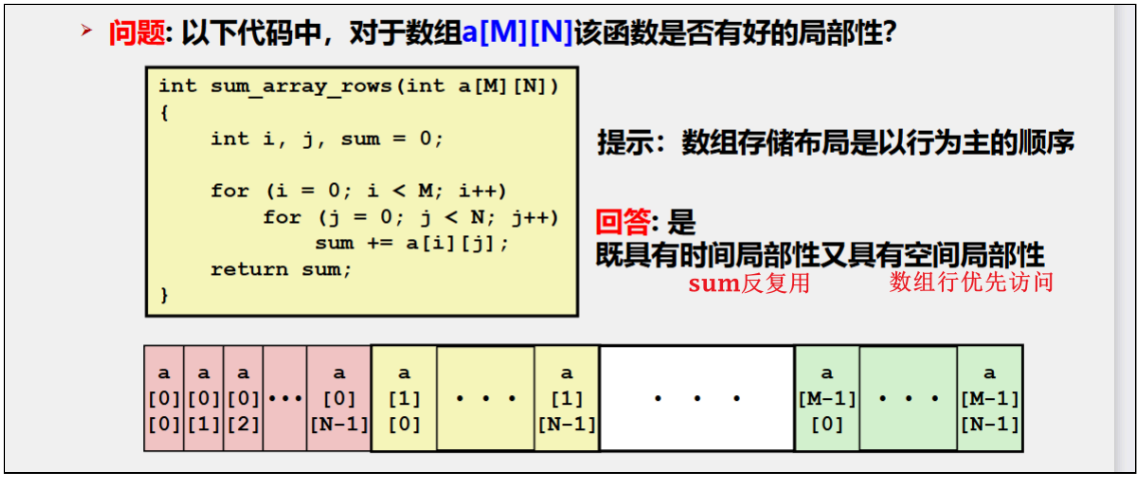
可以看到一开始CPU, RAM的性能差距不大, 他们的访问速度基本相当, 但是**随着CPU性能的提升非常快(频率的增加, 核数的增加), 两者之间的差异越来越大**

局部性原理

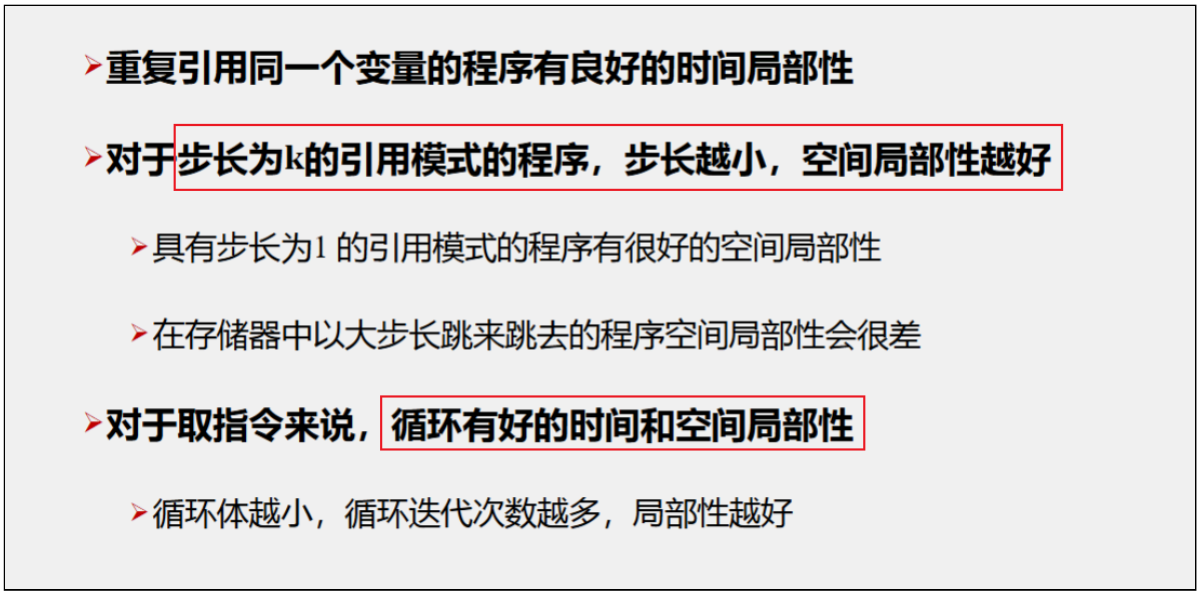
计算机程序具有**局部性(Locality)**, 这一点就能够**缓解CPU和内存之间的这执行速度差异**.

局部性原理: 一个**编写很好**的程序倾向于引用**最近引用过的数据本身**, 或者引用的数据项**邻近于其最近引用过**的数据项
局部性分为两种:

1. **时间局部性**: 被引用过的数据很可能很快会被再次引用
2. **空间局部性**: 一个存储位置被引用了一次, 很可能很快其附近存储位置就会被引用



局部性小结

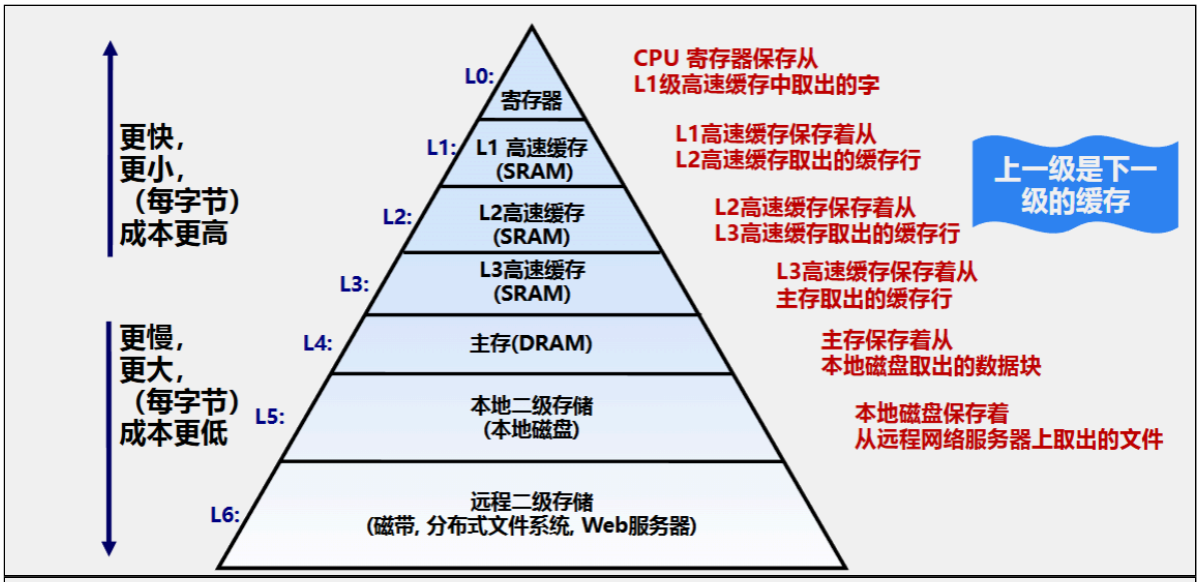


5.2 存储器层次结构

软件和硬件的基本属性**决定了存储器层次结构**设计的必要性

1. 访问速度越快的存储技术每字节成本要比速度慢的技术高, 且容量较小, 也需要更多能耗
2. CPU和主存之间的速度差异日益增大
3. 编写良好的程序具有良好的局部性

上面基本属性基本可以互补, 这样的"互补"可以在**存储器层次结构**中得到体现

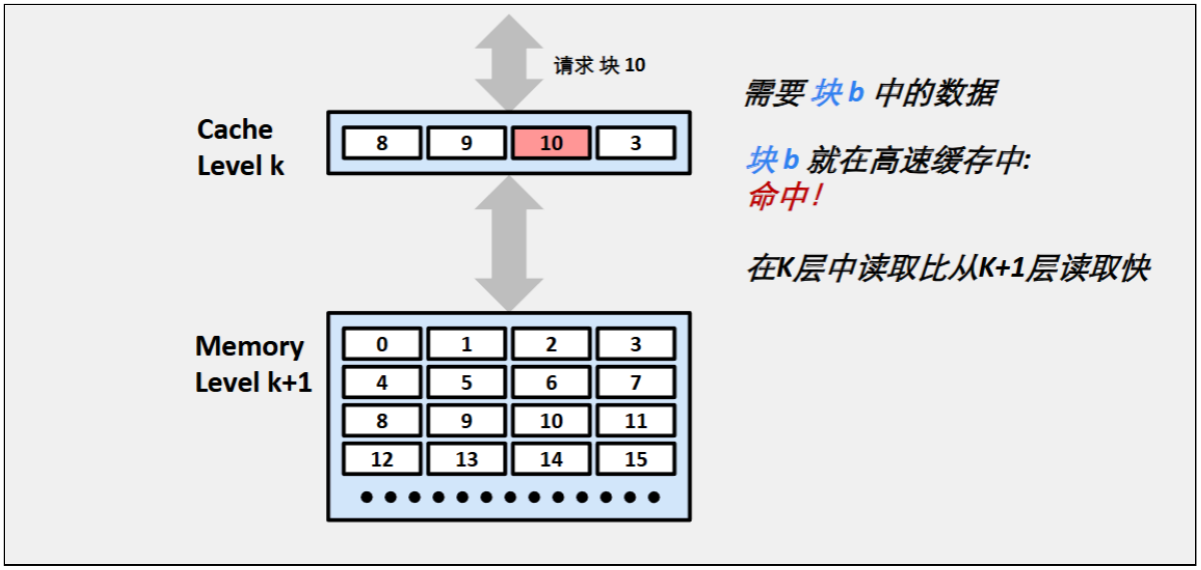


- 高速缓存Cache: 一个小而快的存储设备，它作为存储在更大、更慢的设备中的数据对象的缓冲区域
- 存储层次结构的中心思想:---缓存
  - 对于每个 k，位于k 层的更快、更小的存储设备作为位于 k+1 层的更大、更慢的存储设备的缓存
- 存储层次为什么有效?
  - 由于局部性，相对于层次 k+1 的数据，程序趋向于更频繁访问层次 k 的数据
  - 因此，允许层次 k+1 的存取速度更慢，空间更大，单位价格更便宜
- ***Big Idea:*** 这种存储层次构建了一个价格接近最底层存储层次、大容量存储，而读取数据的速率接近最顶层的存储层次

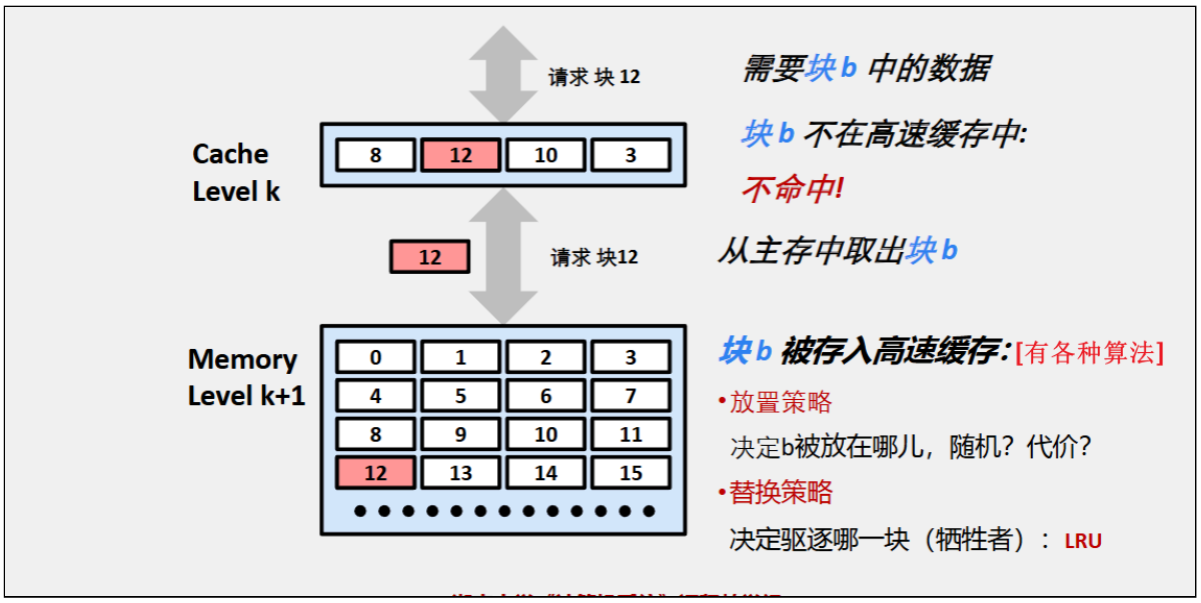
5.2.1 高速缓存的命中/不命中

程序趋向于更频繁访问层次k的数据, 访问高速缓存时, 会有**命中(hit)** 和 **不命中(miss)**两种情况

命中(hit)



不命中(miss)



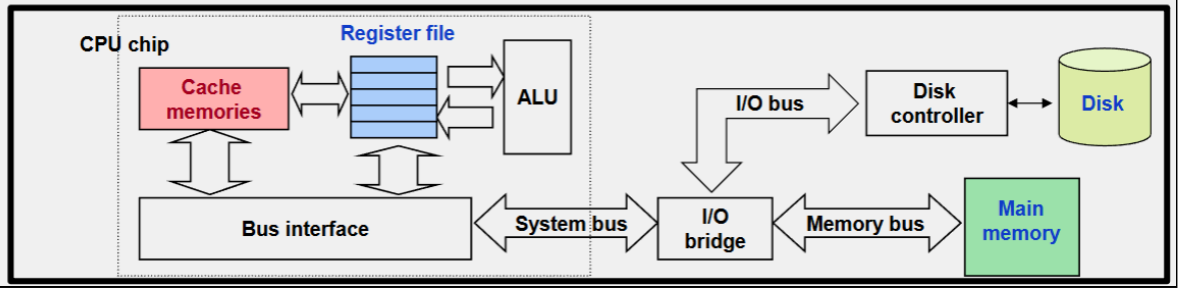
高速缓存的不命中又分为三种类型:

- 冷 (又称 强制) 不命中
  - 如果k层缓存是空的，则对任何数据的访问都是不命中的
  - 属于短暂事件，在缓存暖身后不会出现
- 容量不命中
  - 需访问相对稳定不变的数据集合，这个块的集合称之为工作集 (working set) ，其大小超过缓存大小
- 冲突不命中
  - k层缓存比k +1层缓存空间更小，只能存放k+1层缓存数据块的子集
    - 例如，k+1层的块 i 必须放在 k层 的 (i mod 4) 块中 （严格放置策略）
  - 缓存够大，但所需的多个数据块都被映射到同一个缓存块中，导致一直发生冲突不命中
    - 例如，反复引用块 0, 8, 0, 8, 0, 8, ... 那么每次都会产生冲突

5.2.2 高速缓存的结构

高速缓存存储器

- Cache： 小且快的SRAM存储器，
  - 代替频繁访问主存数据块
  - 保存内容是主存数据的子集
- CPU首先在L1，L2和L3高速缓存中寻找所需数据，若找不到再访问主存
- 使用高速缓存的典型总线结构:



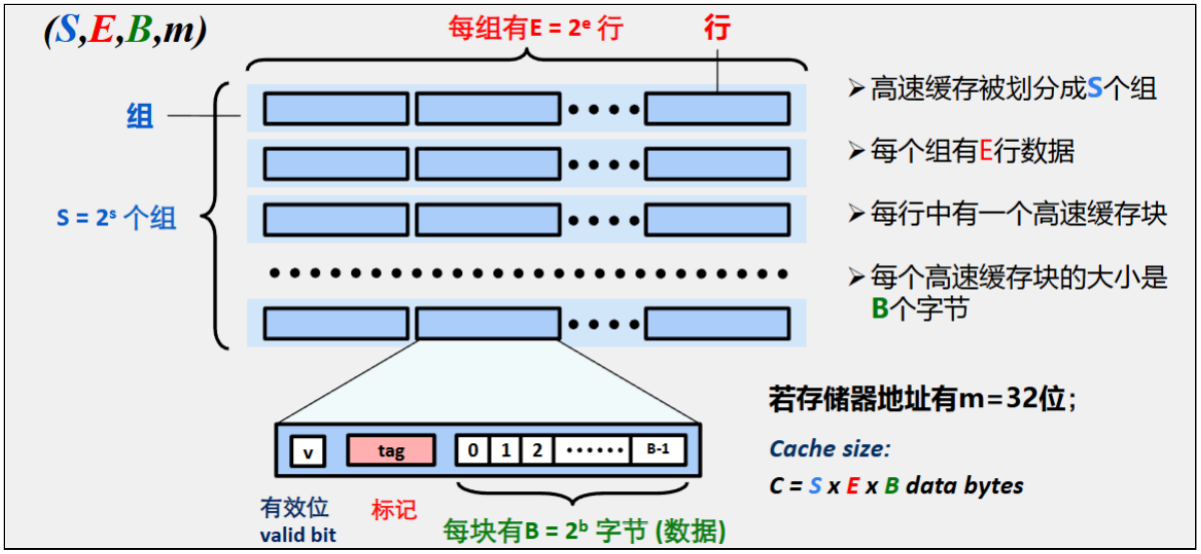
高速缓存存储空间

需要明白**主存**和**高速缓存**存储空间之间的差异

- 32位处理器：32位是指CPU操作总线的数据宽度和地址表示都是32位
  - 32位的数据总线，一次操作最大可执行32位的计算（字长）；
  - 32位的地址总线，能够寻址的地址范围是2^32个地址
- 地址范围从[0x00000000 , 0xFFFFFFFF]，在此范围内的地址空间一共有2^32 = 4,294,967,296个，也就是我们常说的4G（2^32）个地址；
- CPU里处理地址中的值时，每个地址对应一个字节(Byte)大小的存储空间，一个字节由8个位(bit)构成，也就是说，每个地址代表8位存储空间，因此一共有4GB存储空间

- 内存存储空间： 一共2^32个地址，每个地址存储一个字节
- 高速缓存总是比内存小得多得多，L1一般就32-64KB，离4GB远着呢，那么CPU的这32位地址怎么用呢?
- 高速缓存既然小，那一定是被地址空间所共享，4GB中任何一个字节都可能被缓存在L1中，因此，需要有办法进行区分。
- 高速缓存存储空间设计：**存储分组，组中分行，行中分字节**

通用高速缓存存储器结构

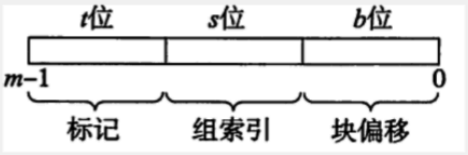


高速缓存存储空间的访问

当需要利用一个比如8位, 32位或者64位的地址范围某个内存/字节时要首先访问高速缓存, 那么如何解读这比如32位地址访问到合适的高速缓存存储的某个地方呢, (若是访问主存, 则直接访问对应的地址就好了), 而高速缓存并不能保证每个地址都能直接访问到, 因此当有一个32位地址是, 我们要怎么解读他:

- 标识组：要在S个组里面找出唯一的一个组信息，需要占用m位地址表示（CPU数据宽度）的多少位呢？如果S=2^s，那我们就需要在m位地址表示中划分出s位来标识组信息；
- 标识行：每组有E行，在m位地址表示中划分t位来标识行信息
- 标识块字节偏移：行中的缓存块字节偏移（每行一个块，每块多个字节），既然有B字节，而B=2^b字节，也就说还要从m位地址表示中分配b位来标识块字节偏移
- 完整的m位地址表示被分成不同的字段，并且要保证刚好够用，可以得到:

$$t = m - (s+b)$$



- t标记位不同于s和b用顺序标识
- t位标记位用于唯一标识组内的行
- 当该行的标记位与内存地址A中的标记位相匹配时，组中这行才包括地址A中存储的字
- 一次存取、加工和传送的数据长度称为字word，一个字通常由多个字节构成

注意 t 的计算只能是
t = m - (s+b)
t 并不等于前面的e.

这里有个小例题来加强理解

[单选]假设高速缓存的结构将32个地址位划分成了t个标记位、s个组索引位和b个块偏移位，已知所有块的大小的和为1024字节，每个组包含1个高速缓存行，每个行是由一个4字节的数据块组成的,确定s、b、t的值()

A. 16, 4, 12
B. 8, 2, 22
C. 4, 4, 24
D. 2, 8, 20

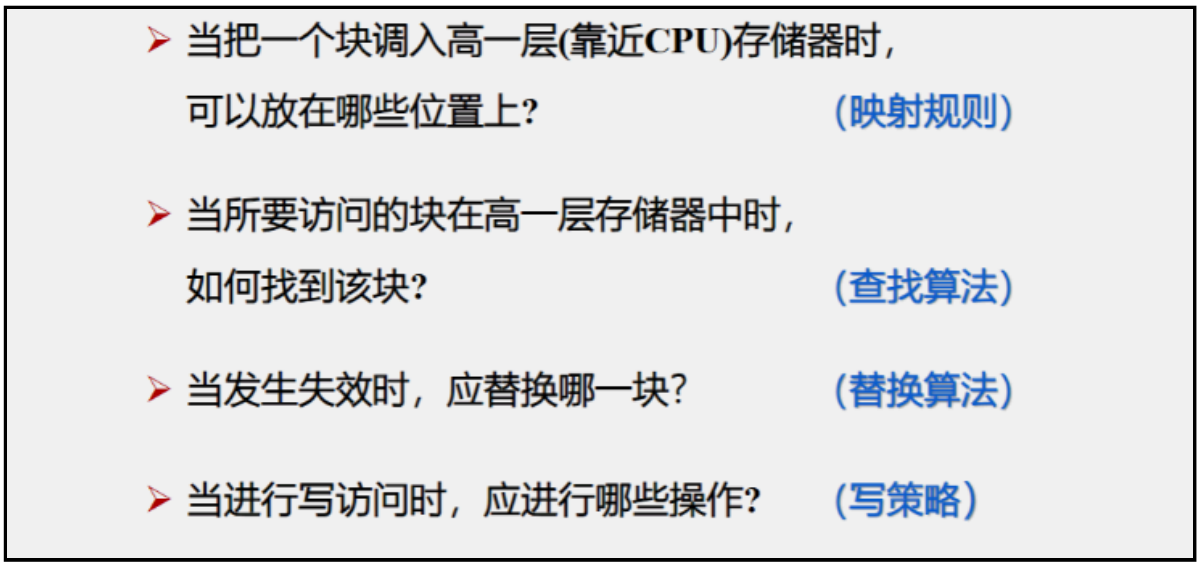
① 每行1个数据块4字节，表示4字节需要2位，$b=\log_2(4)=2$

② 每组1行，每行4字节，总共C=1024字节，S=C/(E*B)，组数：1024/(1*4) =256组，表示256组需要8位，$s=\log_2(256)=8$

③ 剩余的 t=32-2-8=22位用作标记位　t 只能这么算

5.2.3 存储层次

分层次存储需要面对的基本问题有以下这些:



映射规则

映射规则的目的是**确定主存中的一个块(block)应该被放置在高速缓存的哪个位置**. 映射规则有三种:

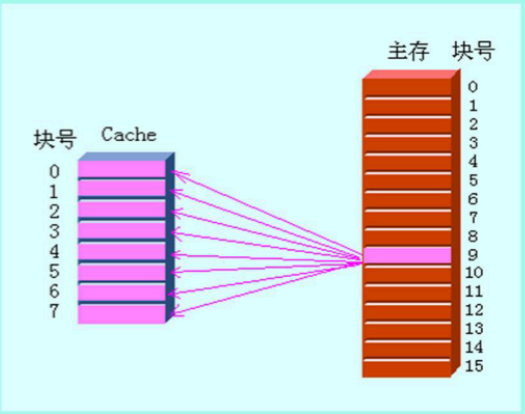
1. **直接映射**(Direct Mapping): 每个主存块只能映射到高速缓存中的**一个特定位置**
2. **全相联映射**(Fully Associative Mapping): 每个主存块可以映射到高速缓存中的**任意位置**
3. **组相联映射**(Set Associative Mapping): Cache被分成若干组, 每个主存块只能映射到高速缓存中的一个**特定组**内的**任意位置**

三个映射规则各有优缺点, 主要考察他们的空间利用率和冲突概率

全相联映射

- 所谓全相联：主存中的任一块可以被放置到Cache中的任意一个位置
- 对比：教学楼位置 —— 每个班自由选择一间教室自习
- 特点：空间利用率最高，冲突概率最低，实现最复杂。

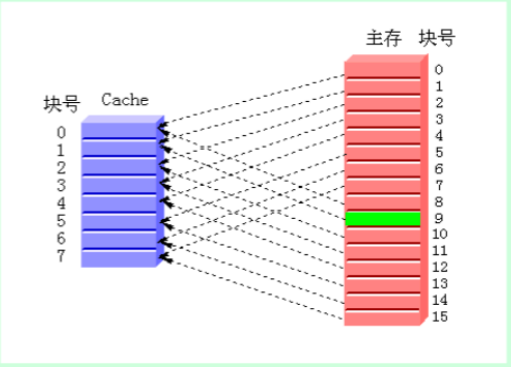
全相联映射
（举例）



直接映射

- 所谓直接映射：主存中的每一块只能被放置到Cache中唯一的一个位置（循环分配）
- 对比：教学楼位置 —— 一个班固定分配一间教室
- 特点：空间利用率最低，冲突概率最高，实现最简单

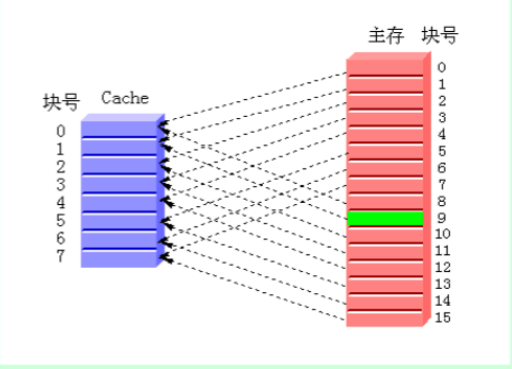
直 接 映 射
（举 例）



直接映射

- 所谓直接映射：主存中的每一块只能被放置到Cache中唯一的一个位置（循环分配）
- 对比：教学楼位置 —— 一个班固定分配一间教室
- 特点：空间利用率最低，冲突概率最高，实现最简单

直 接 映 射
（举 例）

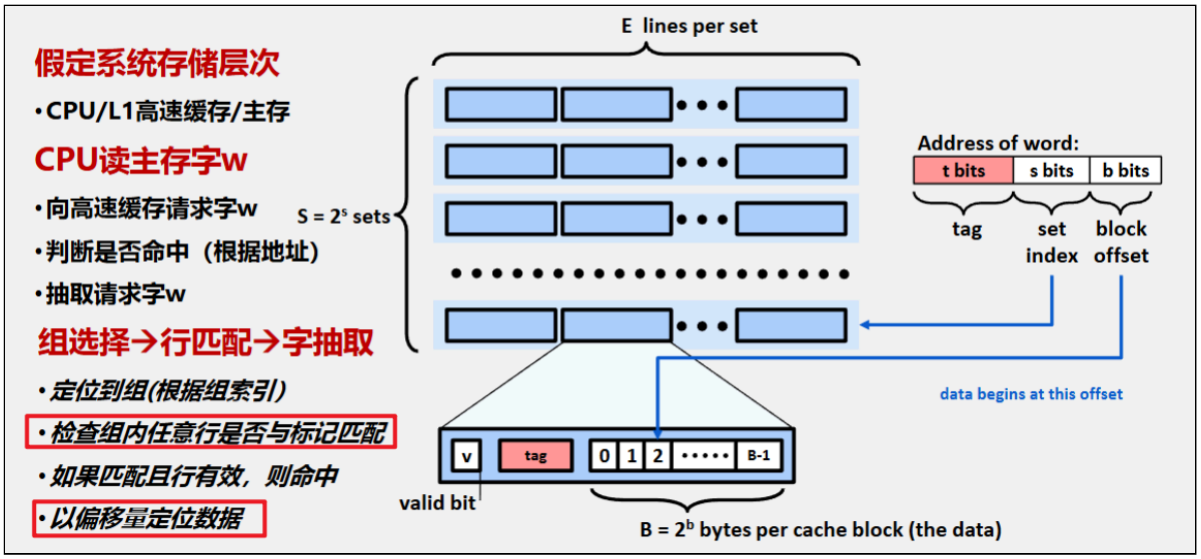


根据E(每个组的高速缓存行数)高速缓存被分为不同的分类

- 全相联高速缓存 -- 只有一个组
- 直接映射 -- 有多个组, 每个组只有一个行
- 组相联高速缓存 -- 有多个组, 每个组有多个行 [最正常]

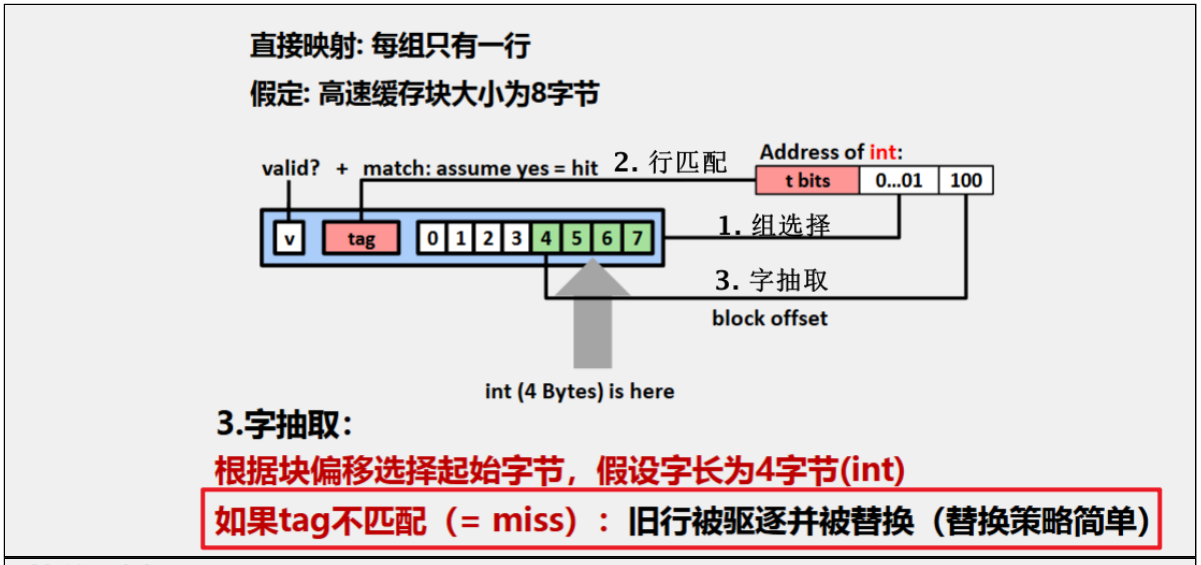
5.2.4 高速缓存读取

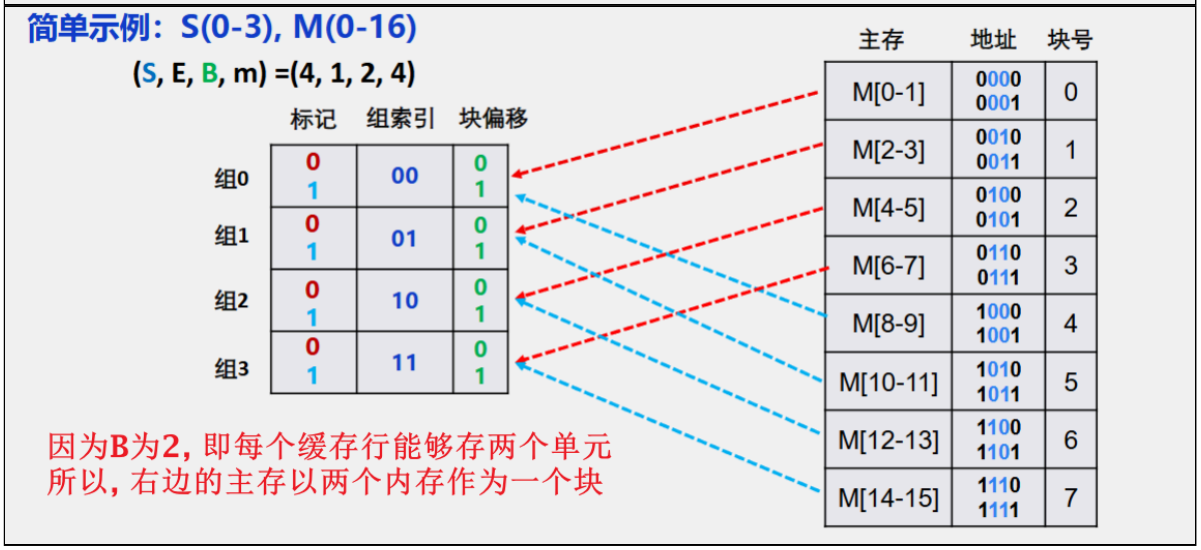
无论怎么映射都基本遵循**组选择 -> 行匹配 -> 字抽取**的步骤



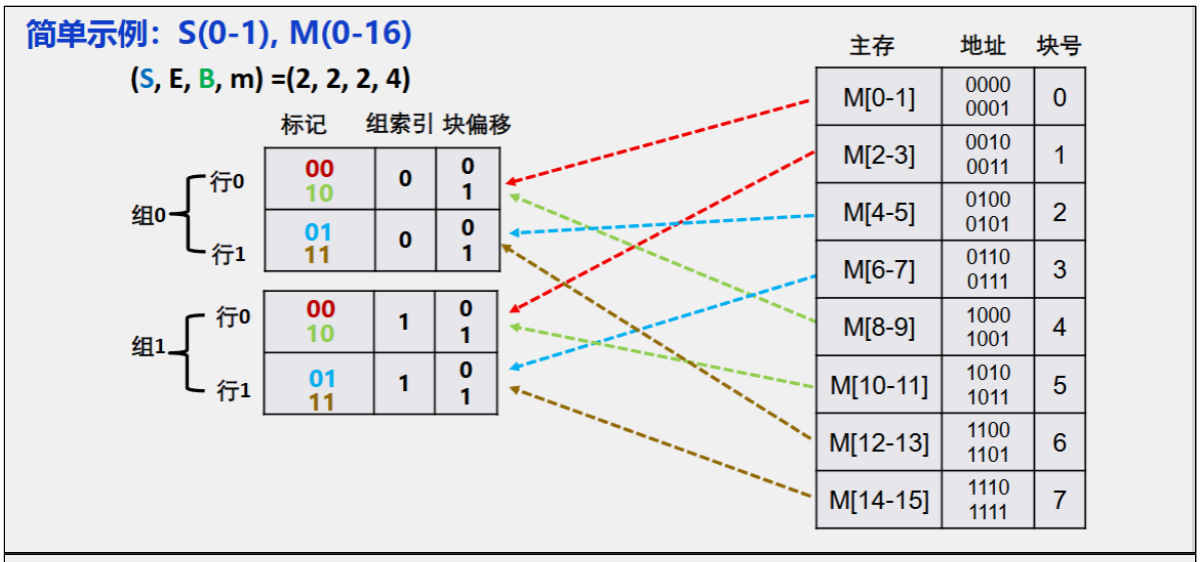
需要重点看的就是**行匹配**阶段, 只要选中组, 对组内的每一个行都要进行匹配的判断: **有效位为1 && 标记为相等**, 只有满足这个条件的行才算命中, 若是**不命中**的话会有不同的策略来实行**替换(冷不命中是放置, 冲突不命中是真的"替换")**

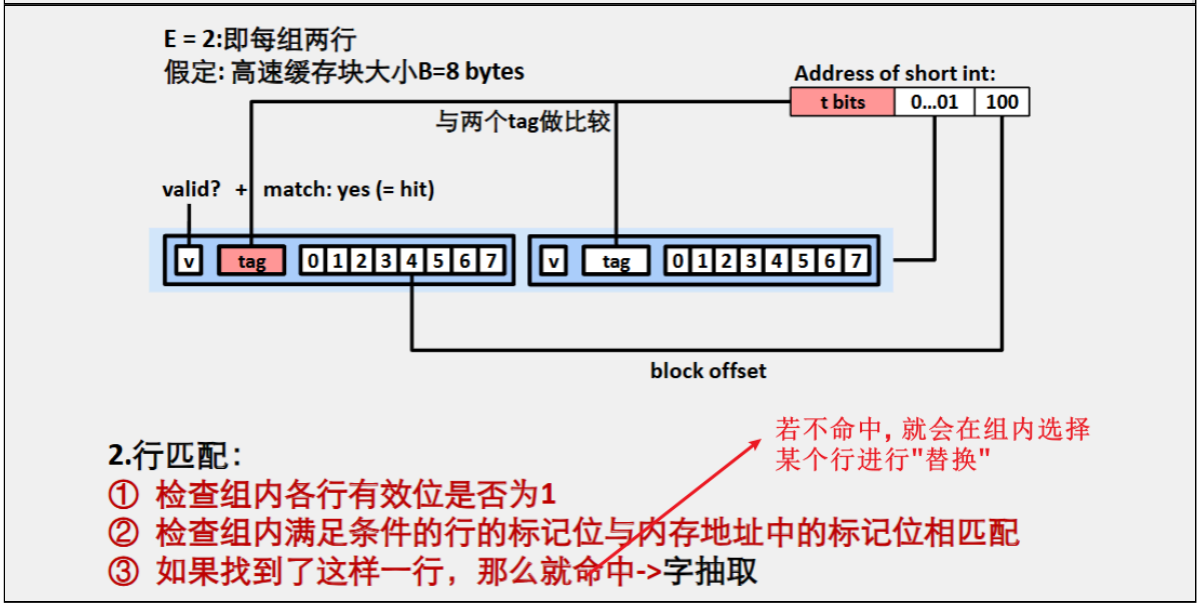
直接映射的读取





组相联映射的读取

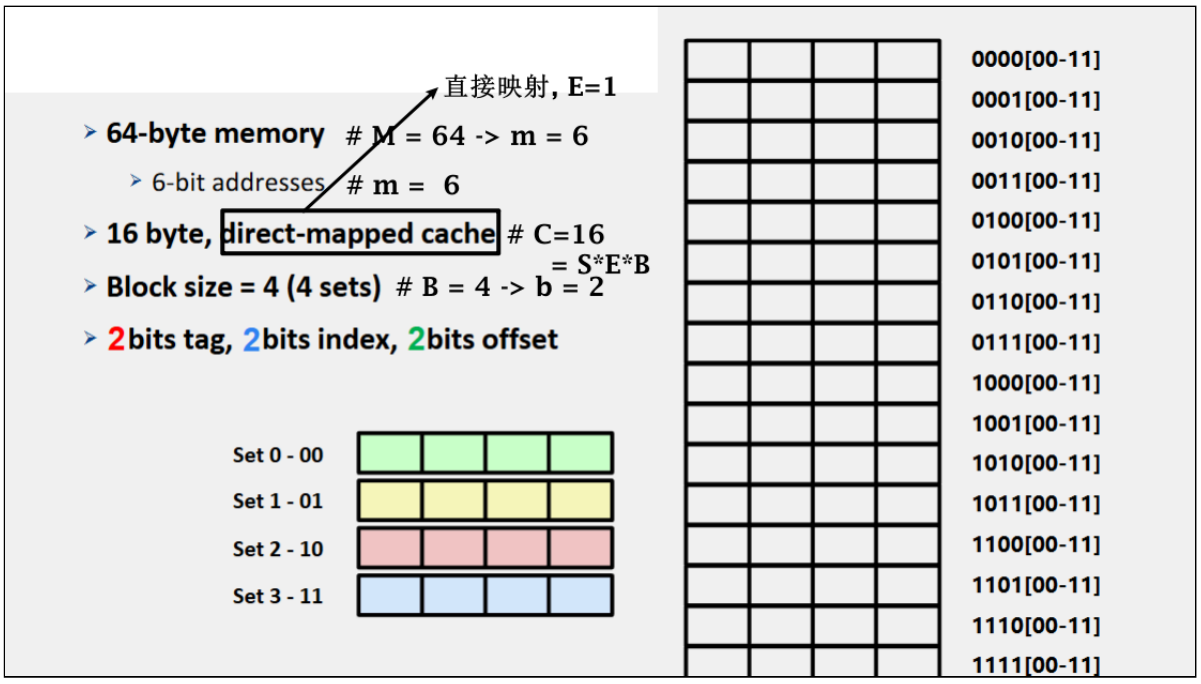




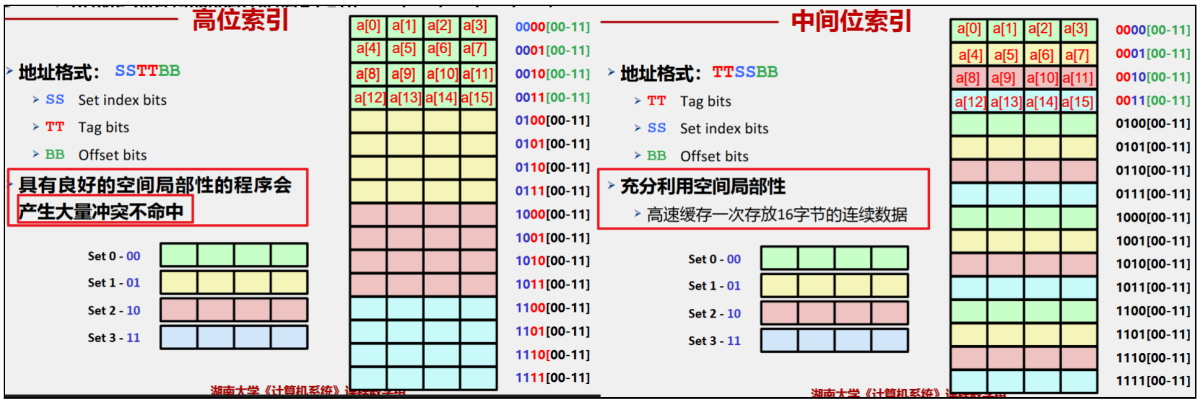
全相联映射的读取

为什么只中间位组选择

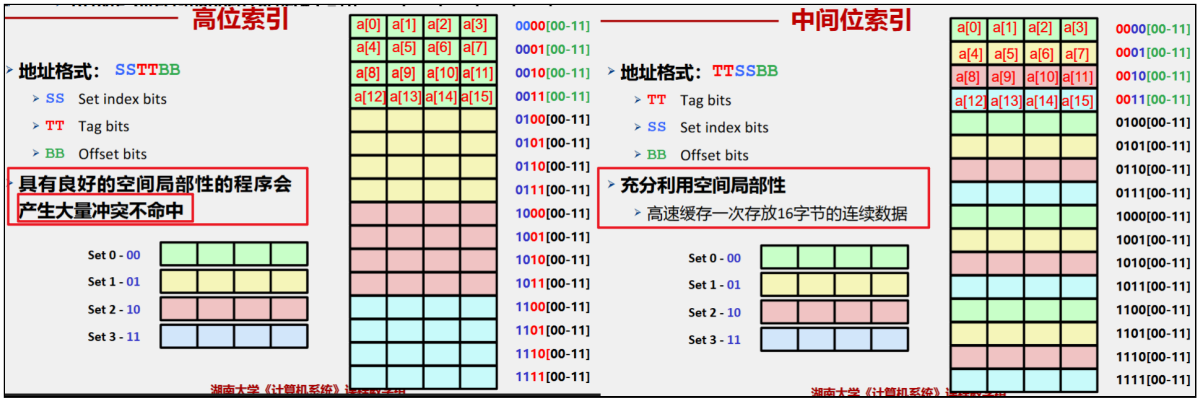
这是前提条件, 也是**计算机系统常见的"题干"**



以下是中间位组选择和高位组选择的对比, 可以看到中间位组选择的命中率更高



5.2.5 Cache写入

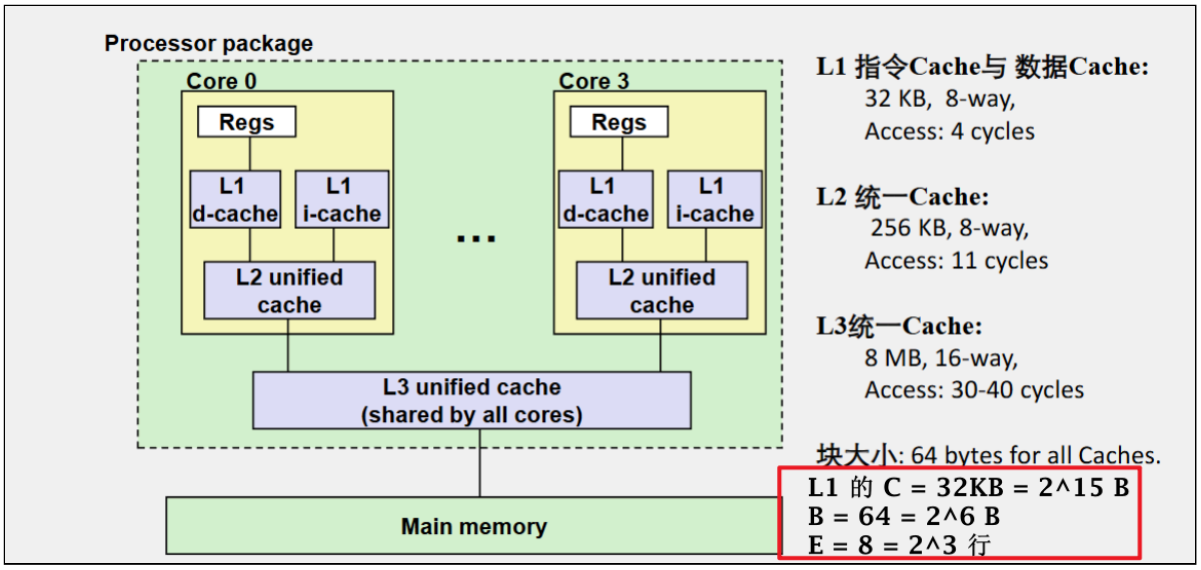


5.2.6 Core i7 Cache结构剖析

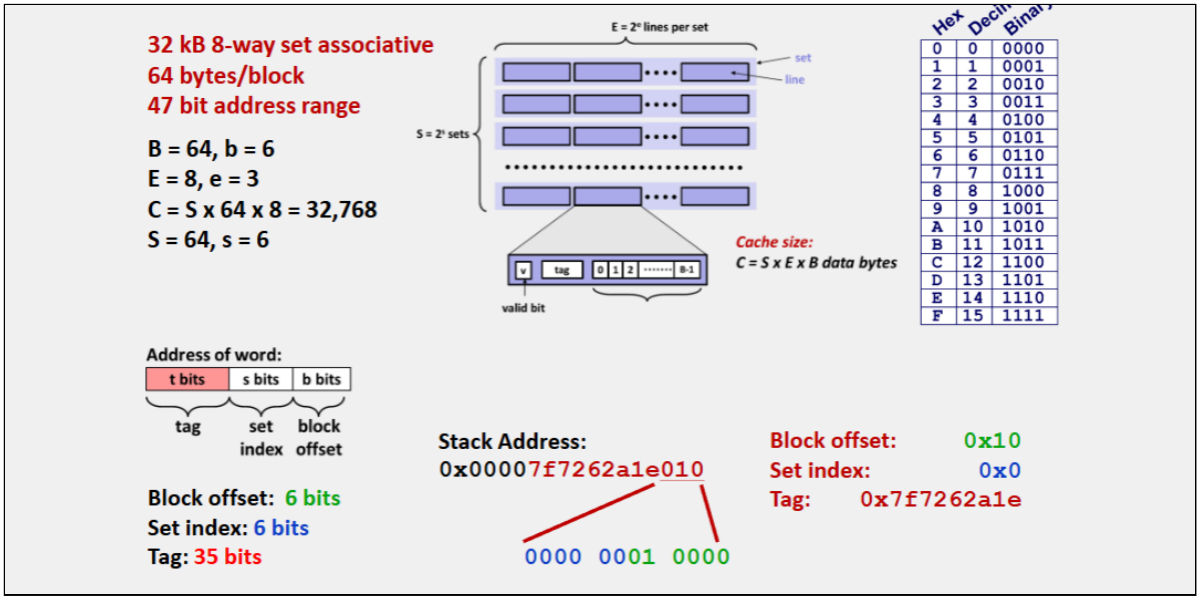


- **数据存在多重拷贝:**
  - L1, L2, 主存, 硬盘
- **如果写命中.....**
  - 直写 (Write-through)：立即将高速缓存数据块写回到低一层
  - 写回 (Write-back)：延迟写回低一层，直到数据被替换时才更新低一层内容
    - 每个高速缓存行维护一个额外的修改位dirty bit
- **如果写不命中.....**
  - 非写分配 (No-write-allocate)：直接写到低一层的主存中
  - 写分配 (Write-allocate)加载相应的低一层中的块到高速缓存，然后更新高速缓存
    - 若空间局部性好，则性能好
- **典型**
  - 直写 + 非写分配（高层次，速度快）
  - 写回 + 写分配（较低层次的缓存更可能使用，主存-->磁盘）

图中的Access: 4 cycles代表的是命中时间, 后面的性能分析会详细讲



cache性能分析



关于命中率, 命中时间, 制造成本, 局部性, 组相联度, 不命中惩罚等考量, 有如下关系, 一般是好的属性没法兼得

- **不命中率**
  - 存储器引用不命中的比率 (misses / accesses)= 1 – 命中率
  - 典型值(百分比):
    - 对 L1 高速缓存一般为3-10%
    - 对 L2 高速缓存不命中率更小 (e.g., < 1%)， 受限于其大小
- **命中时间**
  - 从高速缓存传送一个字到处理器所需时间
    - 包括组选择、行确认和字选择的时间
  - 典型值:
    - 对 L1 高速缓存一般为4 时钟周期
    - 对 L2 高速缓存一般为10 周期，L3一般需要40个周期
- **不命中处罚**
  - 由于不命中所需的额外时间
    - 对于主存一般为 50-200 周期

5.2.7 编写对Cache友好的程序

- **高速缓存大小的影响**（**命中率 vs. 命中时间**）
  - 容量↑,命中率↑， 命中时间↑， L1强调命中时间
- **块大小的影响** （**空间局部性 vs. 时间局部性**）
  - 块↑， 空间局部性↑，传送时间↑， 时间局部性↓，折中32-64字节
- **相联度的影响** （**制造成本 vs. 命中时间 vs. 不命中处罚**）
  - 相联度↑， 冲突不命中↓，成本和时间↑， L1&L2=8， L3=16
- **写策略的影响** (**传送时间 & 传送次数**)
  - 上层-直写；下层-写回
- **注意区别高速缓存中：组 行 块**
  - 直接映射：组=行；行大小=块大小

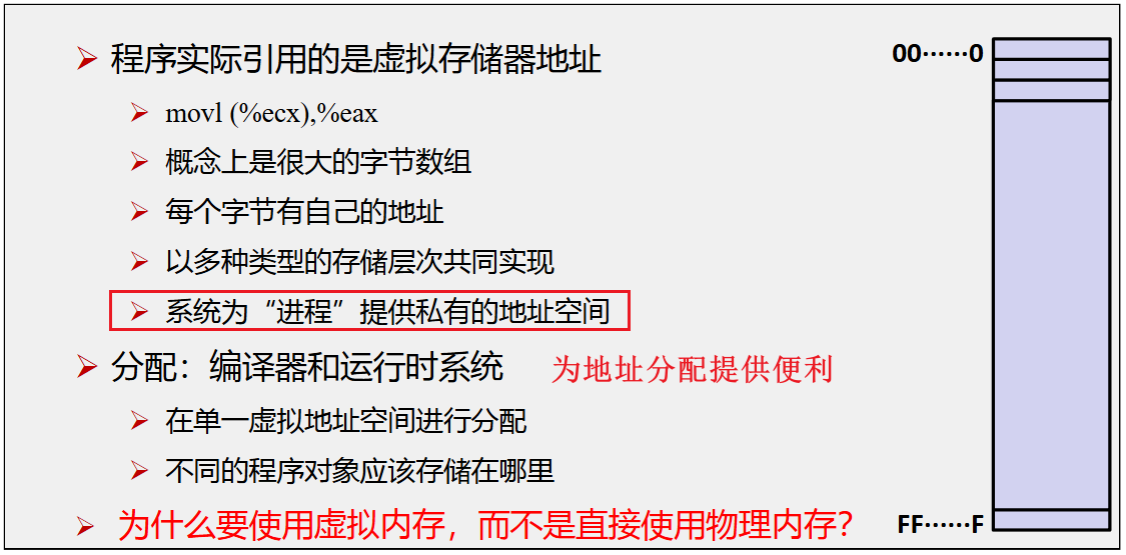
5.2.8 存储器山

存储器山是用来反映时间/空间局部性基于各个不同的存储层次带来的程序性能的影响

第六章 虚拟存储器

- 6.1 虚存抽象
  - 6.1.1 地址空间
  - 6.1.2 三个问题
  - 6.1.3 解决方案
  - 6.1.4 虚拟内存
- 6.2 虚存作为缓存工具
  - 6.2.1 页表
  - 6.2.2 分配页面
  - 6.2.3 异常中的页故障
- 6.3 虚存作为内存管理工具
- 6.4 虚存作为内存保护工具
- 6.5 虚拟内存的地址翻译
  - 6.5.1 地址翻译的符号汇总
  - 6.5.2 地址翻译的页命中
  - 6.5.3 TLB -- 加速地址翻译
  - 6.5.4 TLB 命中
  - 6.5.5 页表的多级结构

6.1 虚存抽象



6.1.1 地址空间

- 线性地址空间：连续的非负整数地址空间

  $\{0, 1, 2, 3 \ldots\}$

- 虚拟地址空间：$N = 2^n$ 虚拟地址

  $\{0, 1, 2, 3, \ldots, N\text{-}1\}$

- 物理地址空间：$M = 2^m$ 物理地址

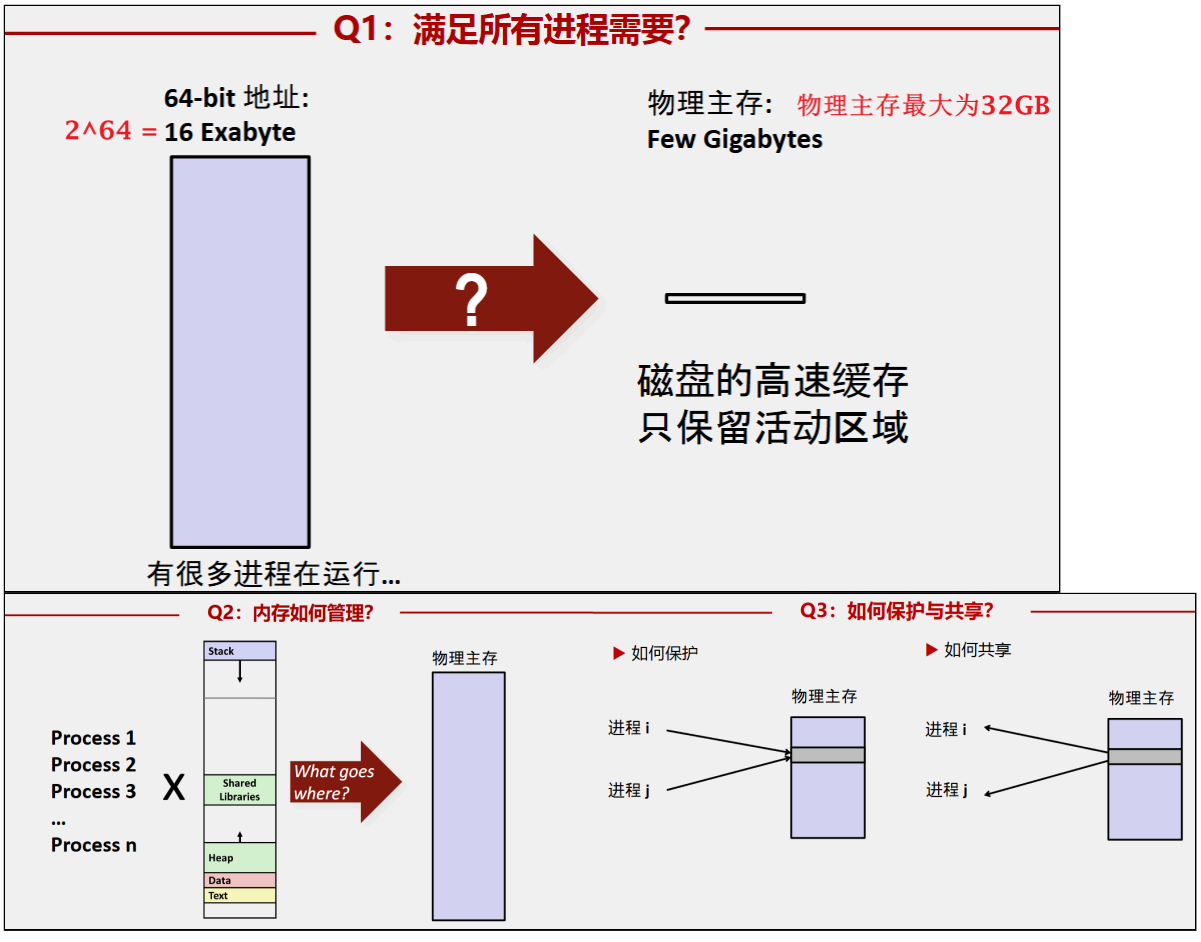
  $\{0, 1, 2, 3, \ldots, M\text{-}1\}$

- 明确区分**数据 (字节)** 与 其**属性 (地址)** [内容和位置]
- 每个对象有多重地址
- 内存的每个字节 具有:

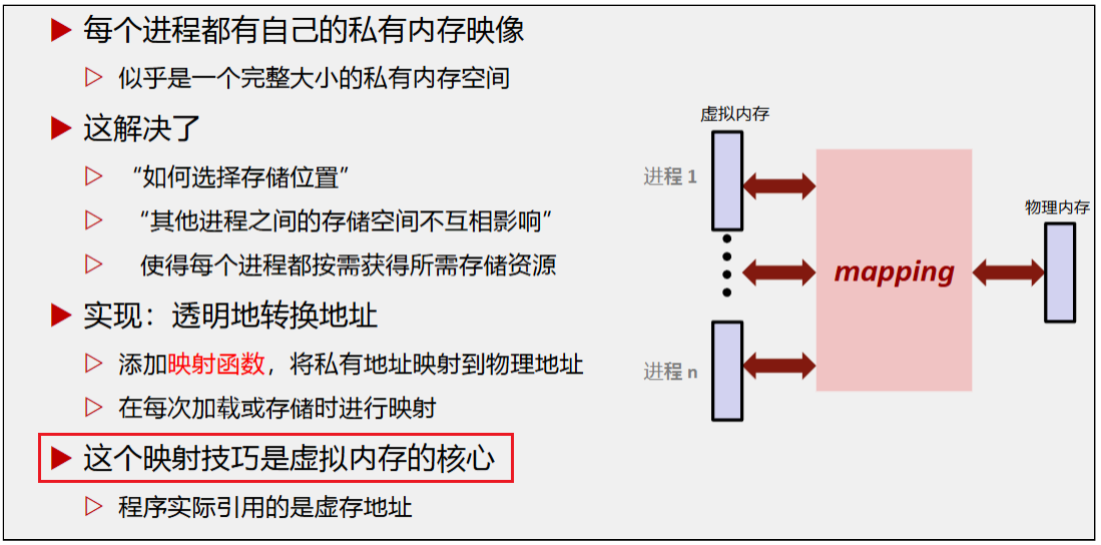
  一个物理地址，一个或多个虚拟地址

  在不同的进程中会映射成不同的虚拟地址

6.1.2 三个问题



6.1.3 解决方案

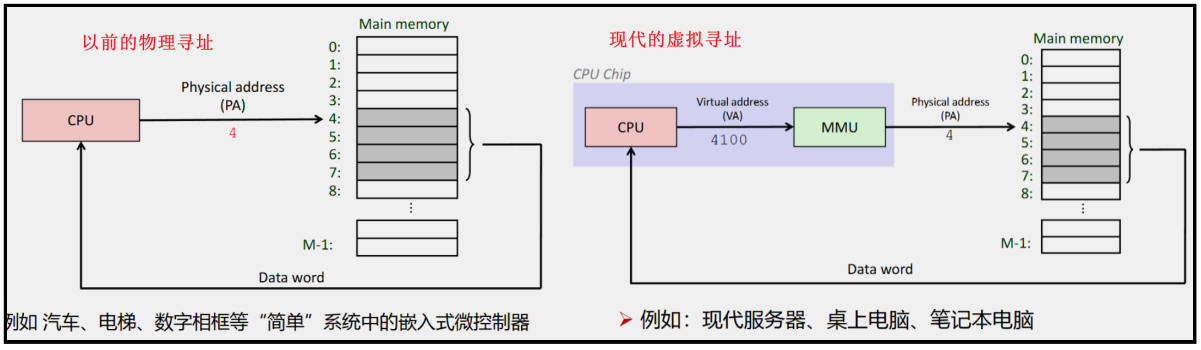


6.1.4 虚拟内存

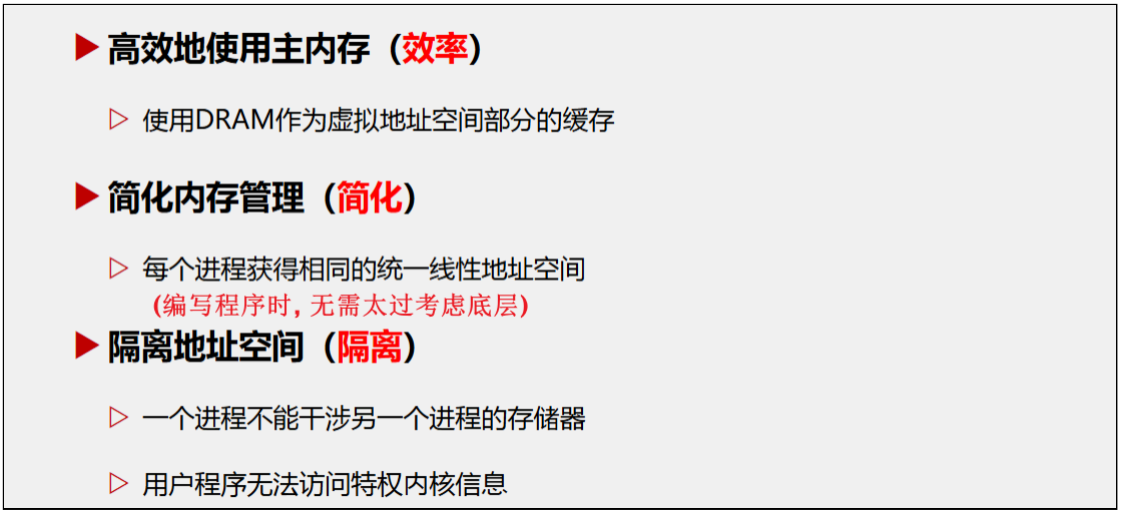
现代处理器使用的是虚拟寻址而不是物理寻址, 也就是说CPU生成的地址是**虚拟地址**, 需要通过**地址翻译**才能得到对应的物理地址, 从而访问内存中的数据

在多任务操作系统中, 每一个正在执行的任务实际上都运行在虚拟地址空间中, 他以 **远大于主存物理空间的大小(多个虚拟地址可以映射到同一个物理地址, 其实多出来的一部分空间需要硬盘提供, 但是通过局部性原理可以实现很高的缓存命中, 不需要访问硬盘)** 进行近似于主存访问速率的存取

处理器上有一个专用硬件--内存管理单元(Memory Management Unit, MMU), 负责将虚拟地址翻译成物理地址, 以及实现虚拟内存的其他功能



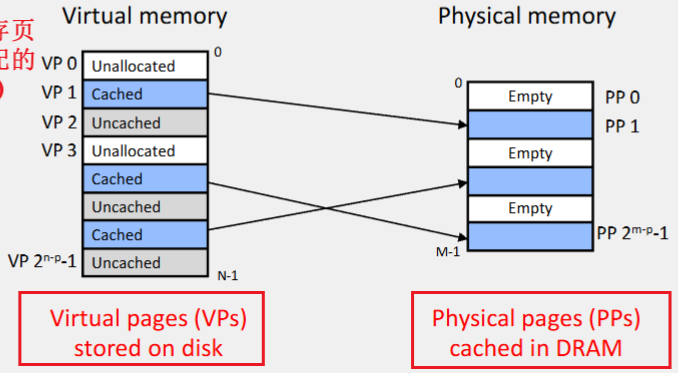
虚拟内存的好处



6.2 虚存作为缓存工具

- 虚拟内存是存储在磁盘上的N个连续字节的数组
- 磁盘上的内容缓存在物理内存（DRAM缓存）中
  - 这些缓存块称为页面（大小为 $P = 2^p$ 字节）(通常为4-8kB)

绝大部分的虚存页其实都是未分配的(unallocated)



Virtual pages (VPs) stored on disk

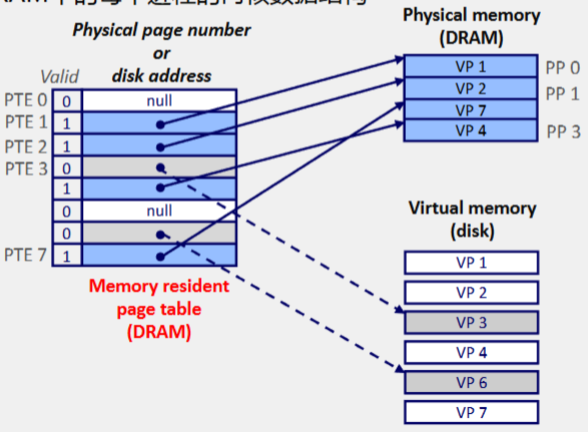
Physical pages (PPs) cached in DRAM

- DRAM缓存的组织结构源于未命中引起的巨大代价
  - DRAM比SRAM慢大约10倍
  - 磁盘比DRAM慢大约10,000倍
- 结论
  - 大页面（块）大小：通常为4-8 KB，甚至达4 MB
  - 全相联
    - 任何虚存页VP都可以放在任何物理页PP中
    - 需要“大”映射函数 - 与CPU缓存不同
  - 高度复杂，昂贵的替换算法
    - 更为复杂精密，超出我们的讨论范畴
  - 写回 而非 直写

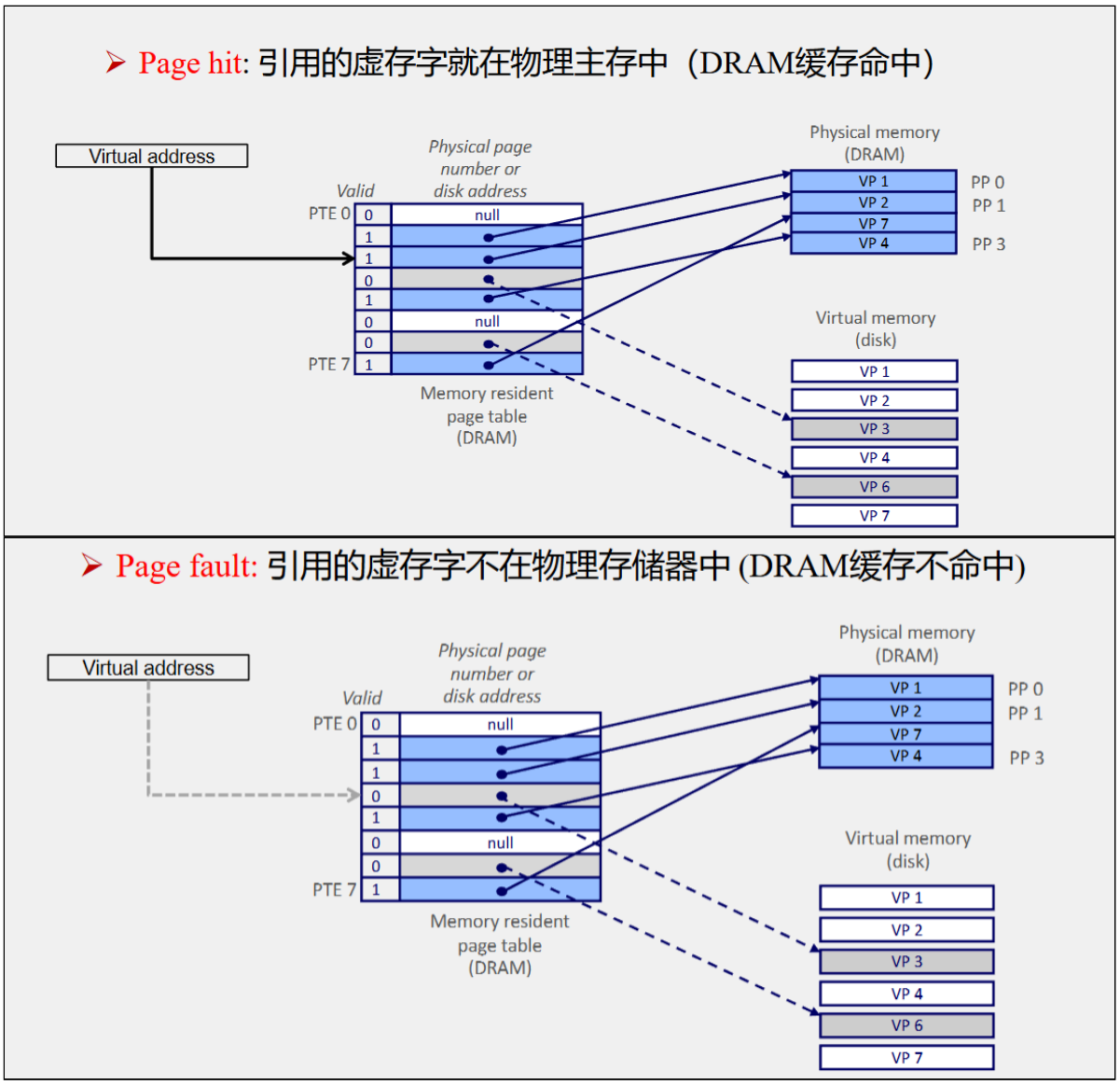
6.2.1 页表

- **页表是将虚拟页映射到物理页的页表项(page table entries,PTEs)的数组。**
  - 存放在DRAM中的每个进程的内核数据结构

该图是某个进程的页表



页表的命中



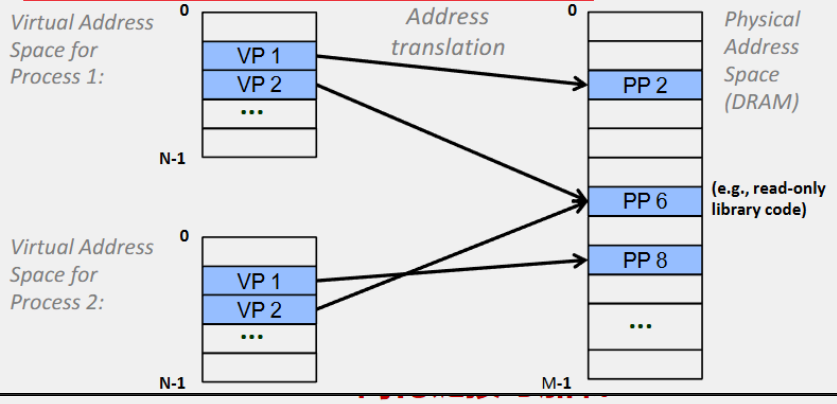
缺页处理程序

6.2.2 分配页面

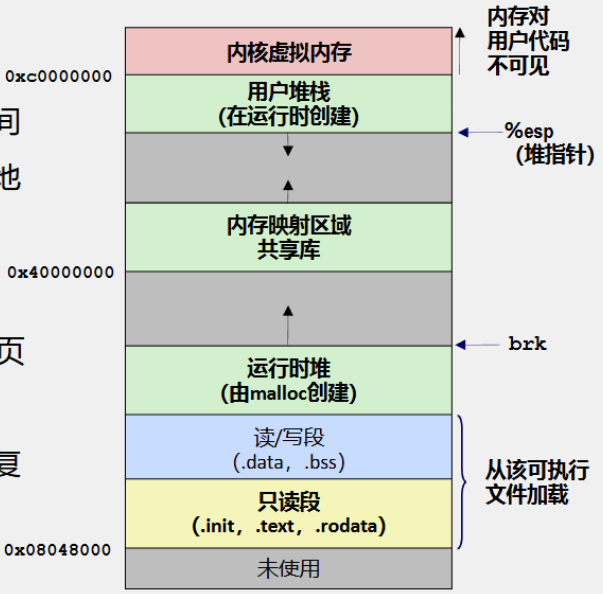
6.2.3 异常中的页故障

6.3 虚存作为内存管理工具

- 关键思想：每个进程都有自己的虚拟地址空间
  - 可以将内存视为简单的线性数组
  - 映射函数通过物理内存分散地址
    - 精心选择的映射可以改善局部性
  - 简化主存分配
    - 每一个虚存页映射到任意物理页面
    - 一个虚页在不同时刻存储于不同物理页
  - 进程之间共享代码和数据
    - 映射多个虚拟页到相同物理页 (如下图示: PP 6)

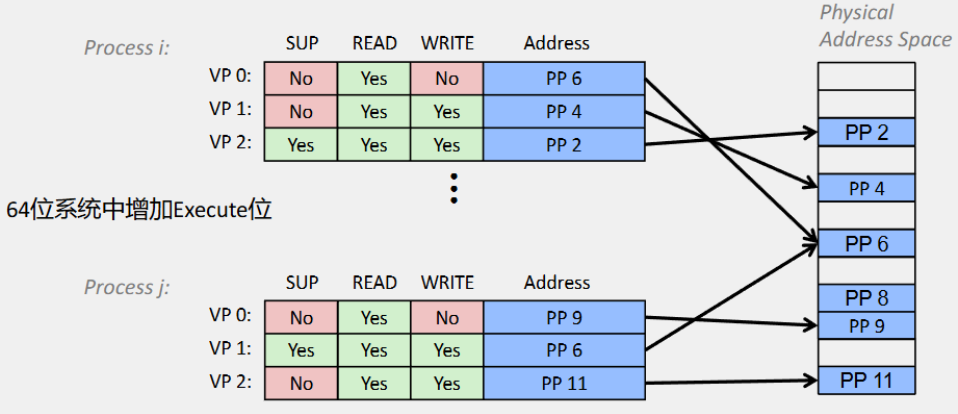


- 链接
  - 每个程序都有类似的虚拟地址空间
  - 代码，堆栈和共享库始终从相同地址开始
- 加载
  - execve()为.text和.data段分配虚拟页面（创建标记为无效的PTE）
  - 根据虚拟内存系统的需求，逐页复制.text和.data段



6.4 虚存作为内存保护工具

- 以许可位（permission bits）扩展页表条目（PTE）
- 页故障处理程序在再映射前重新检查
  - 若违反许可条件向进程发送信号SIGSEGV (段故障)



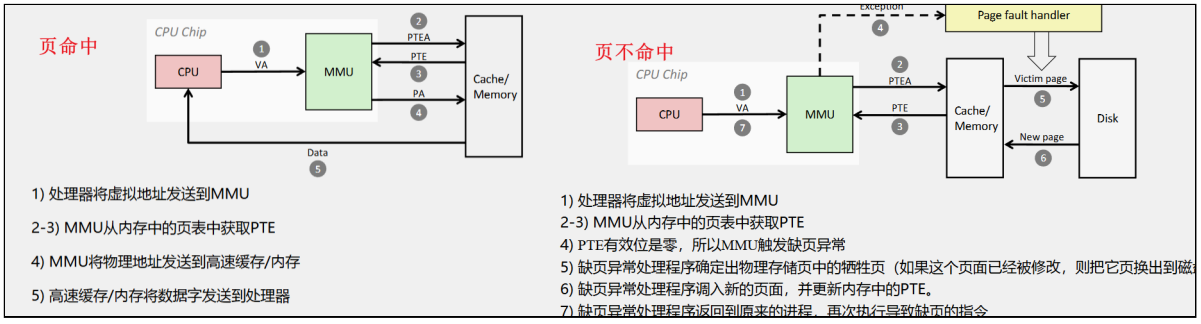
6.5 虚拟内存的地址翻译

- **虚拟地址空间**
  - $V = \{0, 1, \ldots, N-1\}$
- **物理地址空间**
  - $P = \{0, 1, \ldots, M-1\}$
- **地址翻译**
  - $MAP: V \rightarrow P \cup \{\varnothing\}$
  - 对于虚拟地址 a：
    - $MAP(a) = a'$ 如果虚拟地址 a 处的数据在 P 中的物理地址 a' 处
    - $MAP(a) = \varnothing$ 如果虚拟地址 a 处的数据不在物理内存中
      - 无效或存储在磁盘上

6.5.1 地址翻译的符号汇总

- **基本参数**
  - $N = 2^n$：虚拟地址空间中的地址数量
  - $M = 2^m$：物理地址空间中的地址数量
  - $P = 2^p$：页面大小（字节）
- **虚拟地址（VA）的组件**
  - VPO：虚拟页偏移量（字节）
  - VPN：虚拟页号
- **物理地址（PA）的组件**
  - PPO：物理页面偏移量（与VPO相同）
  - PPN：物理页号

6.5.2 地址翻译的页命中



1) 处理器将虚拟地址发送到MMU

2-3) MMU从内存中的页表中获取PTE

4) MMU将物理地址发送到高速缓存/内存

5) 高速缓存/内存将数据字发送到处理器

1) 处理器将虚拟地址发送到MMU

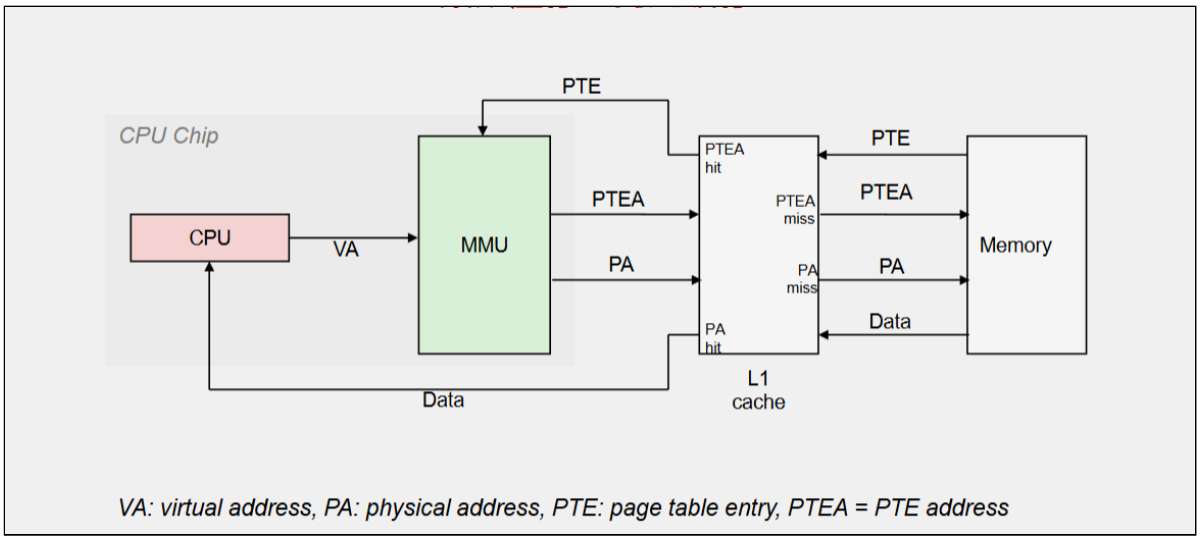
2-3) MMU从内存中的页表中获取PTE

4) PTE有效位是零，所以MMU触发缺页异常

5) 缺页异常处理程序确定出物理存储页中的牺牲页（如果这个页面已经被修改，则把它页换出到磁

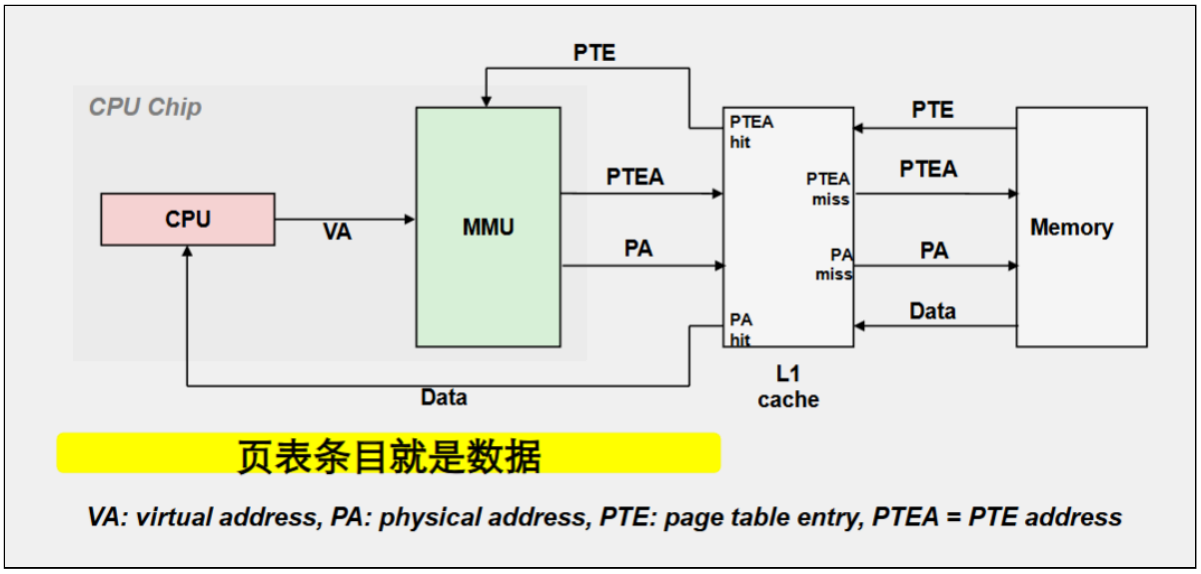
6) 缺页异常处理程序调入新的页面，并更新内存中的PTE。

7) 缺页异常处理程序返回到原来的进程，再次执行导致缺页的指令



6.5.3 TLB -- 加速地址翻译

接下来介绍虚拟存储器的具体运行

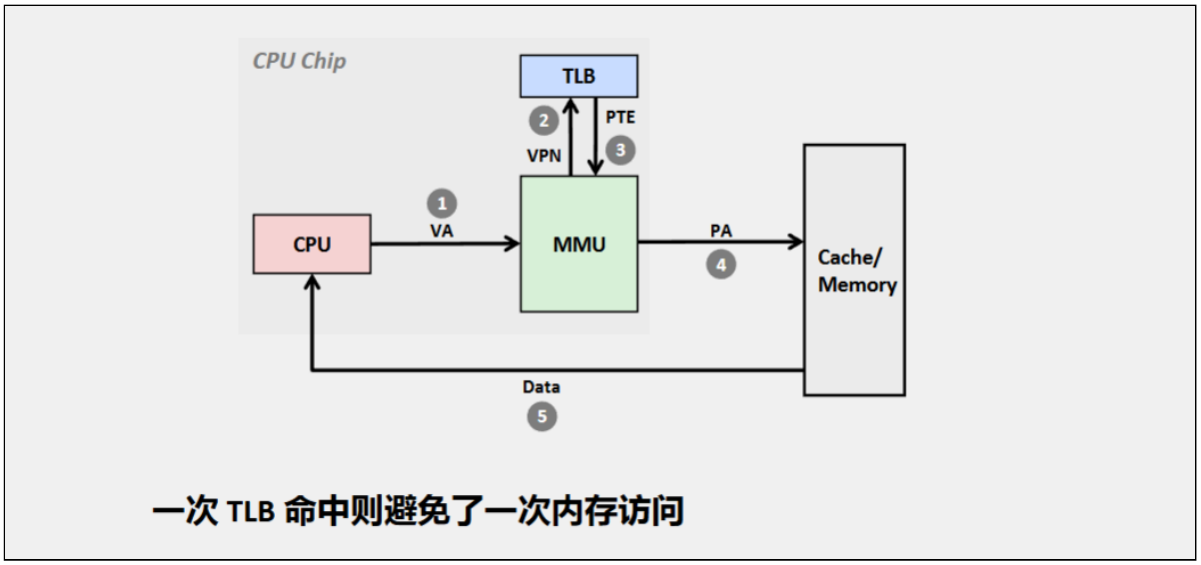


所有的PTE(Page Table Entry, 页表项) 保存在内存(RAM)中的.



TLB(Translation Lookaside Buffer) 是一个小型的高速缓存，专门用于存储最近使用的页表项。当CPU需要进行地址翻译时，首先会检查TLB，如果TLB中有对应的页表项（TLB命中），就可以直接得到物理地址；如果没有（TLB未命中），则需要访问内存中的页表来获取页表项，然后将其加载到TLB中以加速后续的地址翻译。

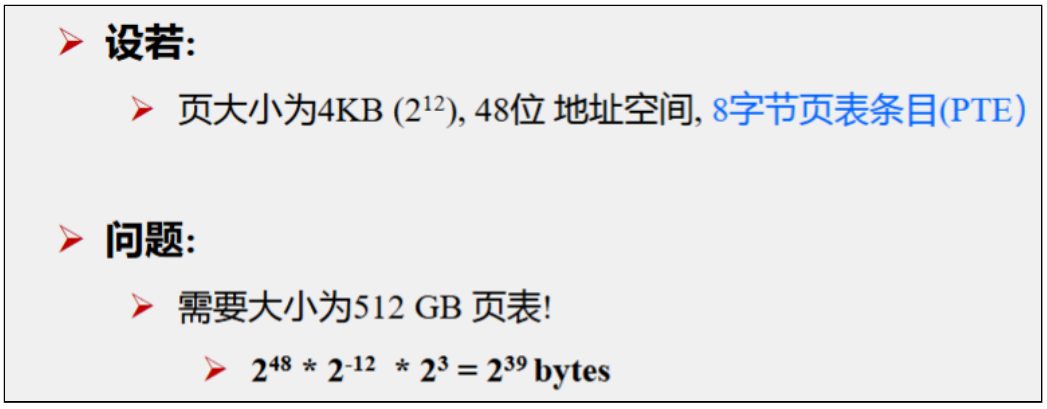
6.5.4 TLB 命中



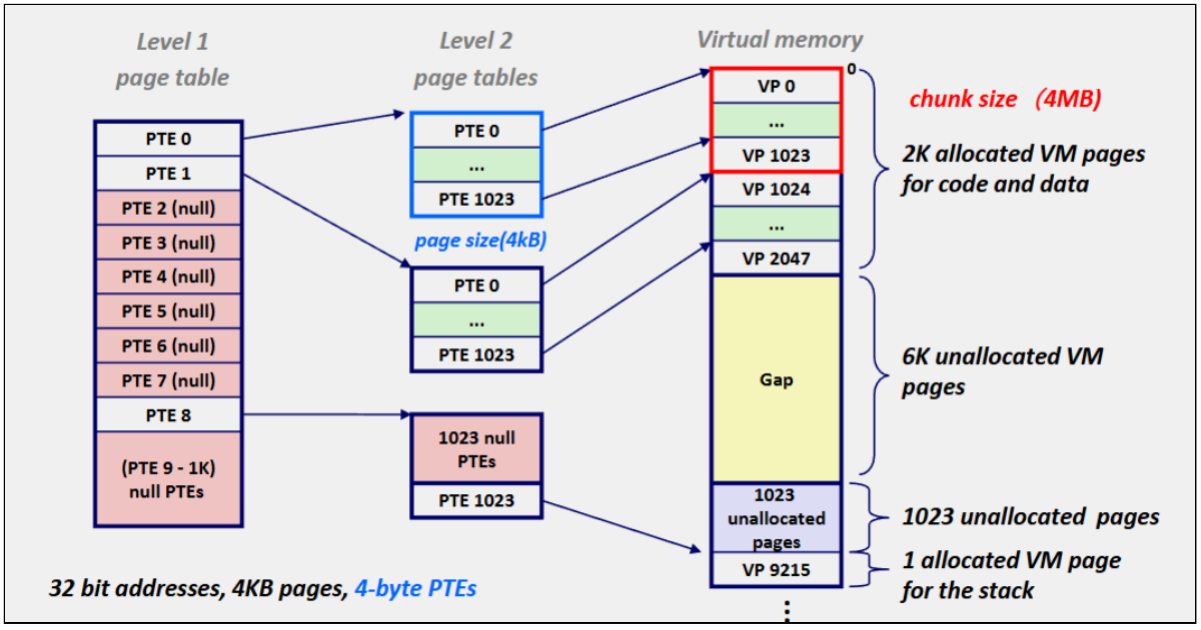
TLB 不命中的概率通常较低，因为程序的局部性原理（temporal and spatial locality）使得最近使用的页表项很可能在TLB中。

6.5.5 页表的多级结构

页表很大:



为了节省内存，现代操作系统通常使用多级页表结构，例如两级或三级页表。这种结构将页表分成多个层次，每层页表只包含指向下一层页表的指针，只有在需要访问某个页时才会分配对应的页表项，从而节省了大量的内存空间。



例如两级页表:

- 一级页表: 每个PTE指向一个页表。（**总是驻留内存**）
- 二级页表: 每个PTE指向一页（像其他任何数据一样换页：**page in&page out**）

第七章 异常与进程

- 7.1 异常
  - 7.1.1 异常处理
  - 7.1.2 异常分类
- 7.2 进程
  - 7.2.1 什么是进程
  - 7.2.2 进程的核心部分
  - 7.2.3 并发进程(Concurrent Processes)
- 7.3 进程的执行
  - 7.3.1 进程的生命周期与状态

程序执行是一个“指令流”，正常情况下 PC（程序计数器）依次递增。但遇到**分支、跳转、函数调用**时，指令流会发生“自愿”的偏离。还有一种**非自愿**的偏离——当 CPU 检测到异常事件（如除零、缺页、系统调用）时，会强制跳转到操作系统预先设置好的**异常处理程序**。这种机制就是**异常控制流（ECF, Exception Control Flow）**。

7.1 异常

7.1.1 异常处理

异常是 CPU 在执行指令过程中，检测到某个“需要操作系统介入”的事件，而触发的**处理器到操作系统（内核）的受控转移。**
每个异常被分配一个**异常号**（0～255，部分保留），CPU 利用异常号在**中断描述符表（IDT, Interrupt Descriptor Table）** 中找到对应的**处理程序入口。**

> 在 Linux/IA-32 中，IDT 的每个表项包含处理程序的段选择子和偏移，以及权限等信息。

异常与普通函数调用的区别：

- 异常由硬件触发，不是代码主动调用。
- 异常可能发生在指令执行中间，处理完后可能返回原指令继续，也可能终止程序。
- 异常的返回地址可能是触发异常的指令，也可能是下一条指令，取决于类型。

操作系统如何响应异常?

当 CPU 遇到异常时，执行以下硬件动作（简化）：

1. **保存当前上下文**：将 `eflags`、`cs`、`eip`（返回地址）以及可能的错误代码压入内核栈。
2. **查 IDT**：根据异常号找到处理程序所在的段和偏移。
3. **权限切换**：如果从用户态陷入内核，切换堆栈（从用户栈切换到内核栈）。
4. **跳转**：`call` 到处理程序。

处理程序执行完毕后，通过 `iret` 指令恢复上下文，并返回用户态。

> **注意**：故障类异常返回时，重新执行 **导致异常的指令**；陷阱类返回下一条指令；中止不会返回。

7.1.2 异常分类

类别	原因	返回行为	典型例子
陷阱（Trap）	故意触发（如**系统调用** `int $0x80`）	返回到**下一条指令**	`syscall`、断点调试 `int 3`
故障（Fault）	可恢复的错误	可能重新执行**当前指令**	缺页（Page Fault）、除零（Divide Error）
中止（Abort）	不可恢复的严重错误	不会返回，进程终止	硬件错误、双重故障（Double Fault）
中断（Interrupt）	来自 CPU 外部（I/O 设备）	返回到**下一条指令**	键盘输入、时钟中断

> 注意：广义"异常"有时包括外部中断，但狭义仅指 CPU 内部事件。这里我们按教材狭义，但会提中断以作对比。

异步异常--中断(Interrupt)

同步异常

缺页异常（Page Fault，异常号 14）

- 属于 **故障**，但可恢复。
- 发生条件：访问的虚拟页不在物理内存中（页表项有效位=0）或权限不够。
- CPU 行为：压入错误代码（指明访问类型：读/写/执行、用户/内核模式等），跳转到 `do_page_fault`。
- 操作系统行为：如果是合法访问但页面未分配（如写时复制、栈自动增长、换入），则分配物理页或从磁盘读入，修改页表，然后**重新执行触发故障的指令**；若是非法访问（如空指针解引用），则发送 `SIGSEGV`。

系统调用（通过 `int $0x80` 或 `sysenter`）

- 属于 **陷阱。**
- 用户程序主动调用 `int $0x80`（或其他快速系统调用指令），将系统调用号放入 `eax`，参数放入其他寄存器。
- CPU 行为：切换到内核模式，跳转到 IDT[0x80] 对应的 `system_call` 处理程序。
- 操作系统处理：根据 `eax` 调用对应的内核函数，完成后返回 **下一条指令**（`iret`）。
- 这是应用程序请求内核服务的唯一入口（例如 `read`, `write`, `open`）。

7.2 进程

7.2.1 什么是进程

进程是操作系统对**正在运行的程序**的抽象。它包含：

- 一个**地址空间**（代码、数据、堆、栈）
- 至少一个**执行线程**（通常就是主线程）
- 一组**资源**（打开的文件、信号处理函数、环境变量等）

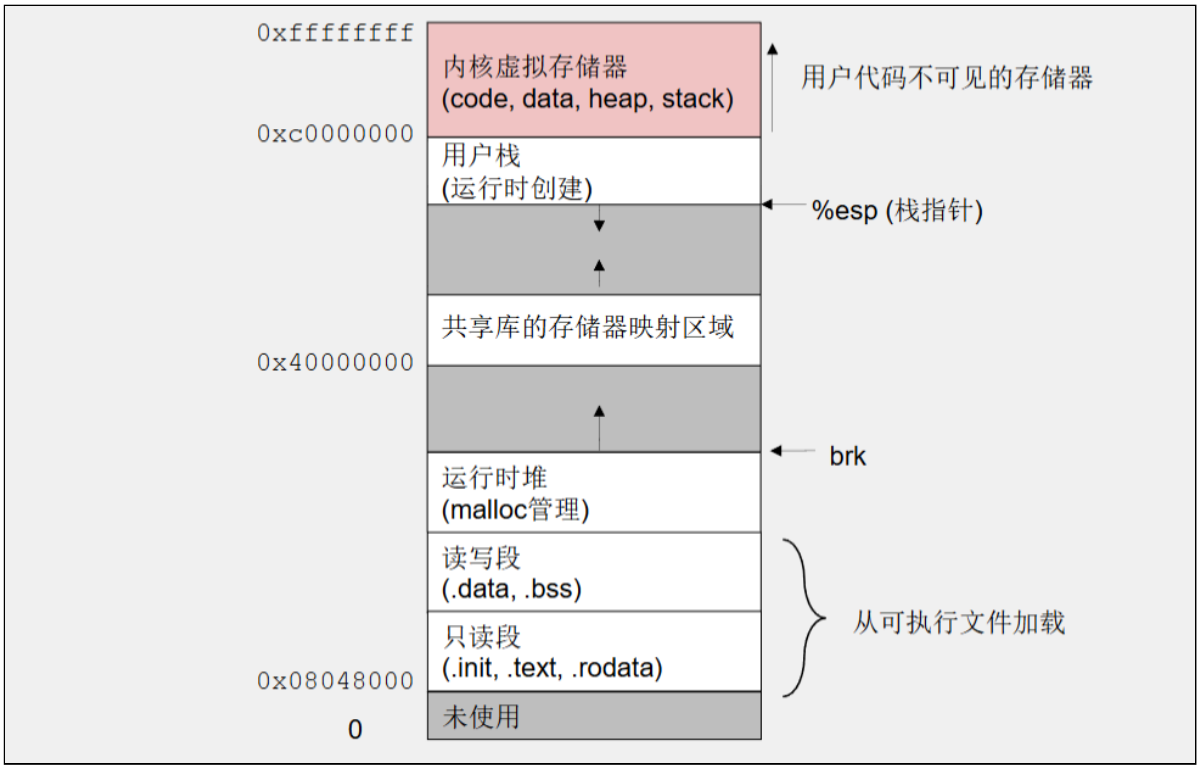
- 运行状态（就绪、运行、阻塞等）

> 程序和进程的区别：程序是静态的可执行文件（如 `prog`），而进程是动态的执行实例。同一个程序可以对应多个进程（比如同时打开两个终端运行 `ls`）。

7.2.2 进程的核心部分

虚拟地址空间

每个进程拥有**独立的虚拟地址空间**（32 位下 4GB），由操作系统通过**页表**映射到物理内存。



- **内核空间**：所有进程共享同一个内核映射，但用户态不能访问（通过特权级隔离）。
- **栈**：存储局部变量、函数调用帧。
- **堆**：动态分配的内存（`malloc`/`new`）。
- **数据段**：已初始化（.data）和未初始化（.bss）的全局变量。
- **代码段**：可执行指令（.text）。

进程控制块(PCB)

内核中维护**每个进程的数据结构（`struct task_struct` 在 Linux 中）**，包含：

- **进程 ID（PID）**：**每个进程的唯一标识**
- **程序计数器（PC）、寄存器上下文**（保存调度时的状态）
- **页表基址**（CR3 寄存器值）
- **内存映射信息**（代码、数据、堆、栈的界限）
- **打开文件表**
- **信号处理信息**
- **进程状态**（运行、就绪、等待、僵尸等）
- **调度信息**（优先级、时间片等）
- **统计信息**（CPU 时间、缺页次数等）

上下文 (Context)

进程的**上下文**包括：

- 用户态寄存器（`eax, ebx, ecx, edx, esi, edi, esp, ebp, eip, eflags`）
- 段寄存器（`cs, ds, es, ss`）
- 页表基址（CR3）
- 内核栈内容（当陷入内核时）

当进程被切换出去时，**内核将当前上下文保存到 PCB**；恢复另一个进程时，加载其 PCB 中的上下文到 CPU。

7.2.3 并发进程(Concurrent Processes)

在操作系统中，**并发**指多个进程（或线程）在**同一时间段内**交错执行。它们可能共享 CPU 时间（单核）或真正同时运行（多核），但从宏观上看仿佛"同时进行"。

并发 vs 并行

概念	定义	硬件要求	关键点
并发	多个进程**交替**执行，在时间上重叠	单核或多核均可	逻辑上的"同时"，微观上是切换
并行	多个进程**真正同时**执行	必须多核/多处理器	物理上的"同时"

- **并发**是操作系统提供的一种"假象"，通过**进程调度**快速切换进程（例如时间片轮转）实现。
- **并行**是硬件能力的体现。

> 回忆之前讲过的**异常**：时钟中断是进程调度的常见触发器；调度器在中断处理中可能将当前进程换出，选择另一就绪进程运行 → 这就是并发的底层实现。

为什么需要并发?

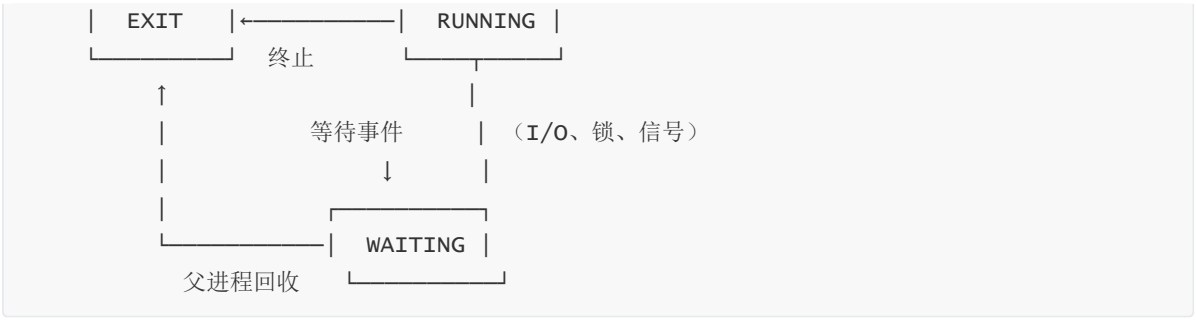
1. **提高资源利用率**：当一个进程等待 I/O（如读磁盘、网络）时，CPU 可以执行另一个进程。
2. **响应及时**：用户交互程序需要保持响应（如编辑文档同时下载文件）。
3. **多任务需求**：现代用户同时运行多个应用（浏览器、音乐播放器、编译器）。

7.3 进程的执行

7.3.1 进程的生命周期与状态

常见状态 (简化)





- **NEW**：进程正在被创建（`fork` 后尚未调度）。
- **READY**：可运行，等待 CPU。
- **RUNNING**：正在 CPU 上执行。
- **WAITING**：阻塞（如等待 I/O 完成）。
- **EXIT**：终止，等待父进程回收（僵尸状态）。

状态转换由**调度器**和**异常/中断**驱动。例如：

- 进程执行系统调用（陷阱）可能主动进入等待（`read` 等）。
- 时钟中断可能触发调度，将当前进程从 RUNNING 变为 READY，选择另一个 READY 进程。
- 缺页异常（故障）可能使进程进入等待（等待磁盘 I/O 换页），但也可能瞬间完成并重新执行指令（仍然 RUNNING）。